

The present work was submitted to:

Chair of Process and Data Science

RWTH Aachen University

Determining Material Waste in Object-Centric Event Logs using Input and Output Mining

Bachelor Thesis

presented by

Joshua Prieth

446803

First Examiner: Prof. Dr. Wil M. P. van der Aalst

Second Examiner: Prof. Dr. Sander Leemans

Advisor: Nina Graves

Aachen, February 15, 2026

Abstract

Life Cycle Assessment is an environmental reporting framework in which the input and output flows of a process are quantified in the Life Cycle Inventory analysis (LCI analysis) phase. Waste flows are of particular interest for sustainability.

LCI analysis has the potential to be automated using object-centric process mining, but existing process mining algorithms focus on the control flow of a process, in which events represent synchronization points in the life cycle of objects. An LCI analysis, however, requires knowledge about the material flow, which interprets events as transformations of input objects to output objects.

To discover a model of the material flow from an object-centric event log, which we identify with the object flow, we propose the Object Flow Miner, a discovery algorithm that constructs Object Flow Nets (OFNs). These visualize the object flow through quantified transformations of input objects into output objects for each activity. Knowledge about the inputs and outputs of events can be used to compute waste flows, and to detect implicit waste by calculating the difference between input and output mass.

The algorithm was evaluated on public object-centric event logs, where it discovered accurate as-is models of the object flow or derived suitable process models through filtering. We demonstrated our approach on the hinge production log, where it semi-automated the LCI analysis by providing a quantified OFN containing information about the waste flows.

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Problem Statement	4
1.3	Research Questions	7
1.4	Research Goals	9
1.5	Contributions	9
1.6	Thesis Structure	9
2	Preliminaries	10
2.1	Mathematical Foundations	10
2.2	Life Cycle Assessment	11
2.2.1	Example	12
2.3	Material Flow Analysis	13
2.3.1	Example	14
2.4	Object-Centric Process Mining	14
2.4.1	Object-Centric Event Logs	15
2.4.2	Example	17
2.5	Petri Nets	18
2.6	Resources and Handling Units	19
3	Related Work	20
3.1	Automated Modeling in Life Cycle Assessment	20
3.2	Sustainability Analysis Using Process Mining	21
3.3	Object-Centric Process Discovery	23
4	Method	24
4.1	Object Flow Nets	24
4.2	Role Functions	26
4.3	Object Flow Miner	27
4.3.1	Aggregation of Events to the Activity Level	28
4.3.2	Creating Place Candidates	30
4.3.3	Detecting Self-Loops	32
4.3.4	Combining Equal Place Candidates	33
4.3.5	Removing Subplaces	35
4.3.6	Adding Missing Configurations	35
4.3.7	Creating the Model	37

4.3.8	Reducing Model Complexity by Filtering	41
4.4	Annotating the Model with Waste	44
4.4.1	Waste in Object-Centric Event Logs	44
4.4.2	Annotating the Model with Waste	46
4.4.3	Example	48
5	Implementation	50
5.1	Discovering the Role Functions Automatically	50
5.2	Ocelescope Plugin	52
6	Evaluation	57
6.1	Detecting Basic Constructs	57
6.1.1	Exclusive choice	57
6.1.2	Concurrency	58
6.1.3	Loops	58
6.1.4	Non-free Choice Constructs	59
6.1.5	Conclusion	61
6.2	Discovered Models on Public Logs	61
6.2.1	Quantitative Results	62
6.2.2	Container Logistics	64
6.2.3	Procure to Pay	68
6.2.4	Order Management	68
6.2.5	Conclusion about the Model Discovery	71
6.2.6	The Role of Resources	74
6.3	Waste Detection	74
6.3.1	Discovering the Model	75
6.3.2	Waste Annotation	77
6.4	Runtime Performance	81
6.5	Applicability to LCA	82
7	Discussion	85
7.1	Object Flow Nets	85
7.2	Object Flow Miner	86
7.2.1	Limitations of the Object Flow Miner	86
7.2.2	Limitations of the Heuristic Discovery of the Role Functions	87
7.3	Waste Annotation	88
7.4	Applicability to LCA	89
7.4.1	Limitations	89
8	Conclusion	91
8.1	Future Work	92

List of Figures

1.1	The production process of plastic cases in our fictional example, discovered by our proposed method.	5
1.2	The annotated OFN of the plastic production process.	6
1.3	The production process of our fictional example, discovered by the object-centric inductive miner.	8
2.1	Life cycle perspective of biofuel production, taken from [9].	12
2.2	MFA of global phosphorus use, taken from [12].	14
4.1	The workflow of our method. Created with Whimsical (https://whimsical.com).	25
4.2	The configurations of our running example.	30
4.3	The configurations and their associated place candidates of our running example.	32
4.4	The detection and creation of a place candidate for the self-loop in our running example.	33
4.5	The combination of two pairs of equal place candidates in our running example.	34
4.6	The merger of three subplaces into their superplaces in our running example.	36
4.7	The creation of a missing configuration in our running example.	38
4.8	The OFN with weighted arcs discovered by the object flow miner for our running example.	39
4.9	The OFNs with weighted and with variable arcs discovered by the object flow miner for a modified running example with multiple initial inputs for <i>Inject</i>	40
4.10	Using variable arcs and filtering by 10% on configurations in the container logistics log.	41
4.11	Filtering with $\alpha = 0.1$ in our modified running example.	43
4.12	Filtering with $\beta = 0.1$ in our modified running example.	45
4.13	The model of our running example annotated with waste information.	49
5.1	The available methods in our Ocelescope plugin.	53
5.2	The available parameters for the algorithm with weighted arcs and without annotations in Ocelescope.	54
5.3	The output of Ocelescope for the parameters given in Figure 5.2.	55

5.4	The available parameters for the algorithm with weighted arcs and with annotations in Oceleoscope.	56
6.1	The discovered model, containing an exclusive choice between B and C	58
6.2	A model with a split but no join.	58
6.3	The discovered model, containing concurrent activities B and C	58
6.4	The discovered model of a length-2 loop involving B and C	59
6.5	The discovered model for a loop involving B , C and D , with a final B	59
6.6	The discovered model using the object-centric inductive miner for a loop involving B , C and D , without a final B . We modified the visualization of the Petri net to use the same style as OFNs to make comparison easier.	60
6.7	The discovered model using the object flow miner for a loop involving B , C and D , without a final B	60
6.8	The discovered model which does not respect the long-term dependency.	60
6.9	The discovered model for the non-free choice construct.	61
6.10	The workflow of our evaluation. Created with Whimsical (https://whimsical.com).	63
6.11	Output of the object flow miner on the container logistics log, with 10% filtering on configurations.	66
6.12	Output of the object flow miner on the container logistics log, with 10% filtering on configurations and filtering out <i>Forklift</i> , <i>Vehicle</i> and <i>Truck</i>	67
6.13	Output of the object flow miner on the procure-to-pay log, without filtering.	69
6.14	Output of the object flow miner on the procure-to-pay log, using 10% filtering on configurations.	70
6.15	Output of the object flow miner on the order management log, filtering out <i>products</i> , <i>employees</i> and <i>customers</i> and using 10% filtering on configurations.	72
6.16	Output of the object flow miner on the order management log, filtering out <i>products</i> , <i>employees</i> and <i>customers</i> , and using 10% filtering on configurations and 20% on preceding and succeeding activities.	73
6.17	Conceptual model of the hinge production log [21].	75
6.18	Output of the object flow miner on the hinge production log, without resources <i>Facility</i> , <i>Workstation</i> , <i>Machine</i> and <i>Worker</i>	76
6.19	Output of the object flow miner on the hinge production log, without resources and with hints and MFA applied.	78
6.20	The waste annotated model of the hinge production log.	80
6.21	Runtime of the algorithm and setup using the unmodified public OCELS.	82
6.22	Runtime of the algorithm and setup using realistic parameters as described in Section 6.2 for each OCEL.	83

List of Tables

1.1	Events of the plastic production example.	3
1.2	Objects of the plastic production example.	4
5.1	The different cases in our classification.	51
5.2	The initial inputs and final outputs in the different cases.	51
5.3	Illustrations of the structures that trigger each case, and what the assumed underlying structure is.	52
6.1	The results of the evaluation on public OCELS.	65

1 Introduction

A widespread method to determine various environmental impacts of a company process is Life Cycle Assessment (LCA), where a product is analyzed regarding environmental aspects over the course of its entire life cycle, from the extraction of the necessary raw materials to the manufacture, use, and disposal. However, this comprehensive nature makes LCA time-consuming, as many materials and sources are taken into account. This complexity, combined with the need for domain experts, makes LCA expensive [37]. Furthermore, an LCA only captures a snapshot in time, and the work has to be repeated if the process is changed [41]. Another issue is that the process is often simplified in order to make the necessary calculations easier [29], which disregards some details.

We suggest addressing these issues with traditional LCA by utilizing process mining, which provides a set of methods to handle data consisting of fine-grained process events to find patterns and bottlenecks. Object-Centric Process Mining (OCPM) is a well-suited technology for this purpose, as it provides Object-Centric Event Logs (OCELs), which model objects that interact through events, both of which can also have attributes. This makes them powerful enough to capture the interactions of complex life cycle processes, and provide the data for algorithms. More specifically, we apply OCPM to the second phase of an LCA, the Life Cycle Inventory analysis (LCI analysis), a critical step because the subsequent phases depend on it [4]. It is also the most data-intensive and time-consuming phase [32]. Therefore, it benefits from automation. In this phase, the inventory is discovered, that is, the quantified inputs and outputs of the process, including a model of their interactions. Process inputs and outputs include raw materials, products, energy, emissions, and waste [32]. Often, a Mass Flow Analysis (MFA) is used to discover the model, which uses information about the specific masses of materials to quantify the entire system.

In order to compute the inventory in the LCI analysis, it is necessary to know the inputs and outputs of the entire process and of each activity. In the OCEL 2.0 standard [7], events can be related to objects, but the standard does not define whether an object is an input, output, or both in an event. Current process mining algorithms mine the control flow, in which events provide a synchronization point in the life cycle of objects. In the material flow perspective, however, events are transformations of some input objects into some output objects. As OCPM uses objects to represent materials, we also call this the *object flow*. Therefore, to discover the object flow, information about the inputs and outputs of events is required. Thus, to achieve the goal of automatically discovering a process model suitable for LCI analysis, an algorithm that mines the object flow is

1 Introduction

needed. In this thesis, we develop such an algorithm, which we call the **Object Flow Miner**.

It discovers an **Object Flow Net** (OFN), an adapted Petri net model based on the role that objects play in an activity, such as input or output. The main utility of this model is to visualize and quantify which input object types are transformed into which output object types in activities, therefore visualizing the object flow. It also shows the inputs and outputs of the process itself. If in addition, information about the mass of objects is given, we can perform an MFA that quantifies the inputs and outputs of the process. This is necessary for the LCI analysis.

A special case of process output is waste. Waste detection can be an important use case for LCA, since the reduction of waste helps the environment and improves the efficiency of a manufacturing process [6]. Knowledge of the inputs and outputs of the process allows calculating the waste produced by each output object type. In addition, a difference between the input and output mass or object count of an individual event can reveal implicit waste. This is particularly powerful because this type of waste is not directly given in the data. We extend our method with waste annotations on the OFN, which shows where waste occurs in the object flow and, if there is sufficient data, how much.

Thus, our algorithm serves to perform the LCI analysis step of an LCA, by providing a quantified process model, and is especially suited for waste detection. Leveraging this for automating LCA speeds up the process, uses less manpower and money, and with automated workflows in place, changes to the process can be incorporated more easily. In addition, calculations can be done exactly by the program without the need for abstraction. Therefore, our thesis contributes to the field of sustainability by addressing a gap in environmental reporting. Apart from this, the object flow miner can also be used outside the LCA context without waste annotation as a general process discovery algorithm. Therefore, in this thesis, we also contribute a new method of discovering a process model.

We evaluate our approach, where we take into consideration how it handles basic process model constructs in isolation, as well as complete public OCEs. In this regard, we find that the algorithm can mostly handle common process constructs and discovers a useful model of the object flow. Since our goal is to apply this method to aid in LCA, we also examine how useful the discovered model and the waste detection are for the application in LCA. We also measure the runtime, which is acceptable.

1.1 Motivation

Suppose a company producing plastic casings wants to investigate their manufacturing process, especially the emitted waste. They do not analyze the entire life cycle, in particular, not the production of the precursors and the use of the product, since these

are outside the scope of the company. The company logs production events in the OCEL format and includes an attribute for the weight of every input or output of the process.

The process is visualized in Figure 1.1, where the octagon-shaped boxes are process inputs and the octagons with double borders are process outputs. In the process, first, a *Fiber* is mixed together with some *Resin* and injected into the machine to form a *Mold*, which is colored orange. The engineers are experimenting with an optional *Degas* step, which removes air bubbles from the mold but increases cost. Since this step is optional, the model includes a silent transition. After the mold has cooled, it is *Trimmed*, where impurities in the plastic are removed, which end up as *Waste*. Two molds that went through this process are packed into a single output, which is why the upper *Pack* consumes only the mold, and the lower transition takes the created product and another mold to give a new *Product*. Our algorithm can handle links between cases, such as the *Pack* transition that takes molds from two production cycles, which would not be possible in a case-based log without objects.

For this example, we only consider a single produced *Product*, with one mold that underwent the optional degassing and one that did not. The OCEL is conceptually given in Table 1.1, which shows the events, and Table 1.2, which shows the objects. In the events table, the events are identified by IDs in the first column, and their event type (which we call *activity* in this thesis) in the second column. Instead of creating a second table for the *E2O*-relations, we show all related objects with their IDs in the third column, without qualifiers. The objects also have a mass that the company logs.

Event ID	Activity	Related Objects
inject_1_1	Inject	fiber_1, resin_1, mold_1
degas_1_2	Degas	mold_1
trim_1_3_1	Trim	mold_1, waste_1_1
pack_3	Pack	mold_1, product_1
inject_2_1	Inject	fiber_2, resin_2, mold_2
trim_2_3_1	Trim	mold_2, waste_2_1
trim_2_3_2	Trim	mold_2, waste_2_2
trim_2_3_3	Trim	mold_2, waste_2_3
pack_4	Pack	mold_2, product_1, product_2

Table 1.1: Events of the plastic production example.

The proposed object flow miner generates the model shown in Figure 1.1, the same figure that we used as a process visualization. Note that this is *not* an Object-Centric Petri Net (OCPN), as it does not fire with objects but tokens, and the place colors are visualization hints. Instead, it is an object flow net, which is a special Petri net in which the single-bordered octagon places represent process inputs, while double-bordered octagons represent process outputs. The defining principle of an OFN is that incoming

1 Introduction

Object ID	Object Type	Mass attribute
fiber_1	Fiber	11
resin_1	Resin	12
mold_1	Mold	?
waste_1_1	Waste	2
product_1	Product	20
fiber_2	Fiber	13
resin_2	Resin	14
mold_2	Mold	?
waste_2_1	Waste	3
waste_2_2	Waste	5
waste_2_3	Waste	2
product_2	Product	35

Table 1.2: Objects of the plastic production example.

arcs in a transition correspond directly to input object types, with arc weights indicating the specific quantity of each object type consumed. The same holds for outgoing arcs and outputs. Thus, the *Inject* transition represents that a single *Fiber* and a single *Resin* are transformed into a single *Mold* (orange).

As the company recorded the mass, we annotate the discovered OFN with information about the quantities of the inputs and outputs in Figure 1.2. This calculation is performed automatically. The company can use the result as the LCI analysis step in an LCA. The quantity of waste emitted is given on the *Waste* object type. In addition, the total input mass in the process is 50, while the total output mass is 47. Therefore, the company knows that a waste of 3 units was created and not recorded. This is because the implicit waste on the *Inject* transition is 3, which means that there is a discrepancy between the input and output mass in *Inject* events.

1.2 Problem Statement

Currently, the LCI analysis phase in an LCA, which creates a model of the process and its inputs and outputs, is done by hand or with supporting software. Process mining has the potential to automate it, but existing process mining algorithms only discover the control flow instead of the material flow. For analyzing the material flow, it is necessary to know which objects are inputs and which objects are outputs in an event. This also means that there is no current method that uses OCPM for waste detection.

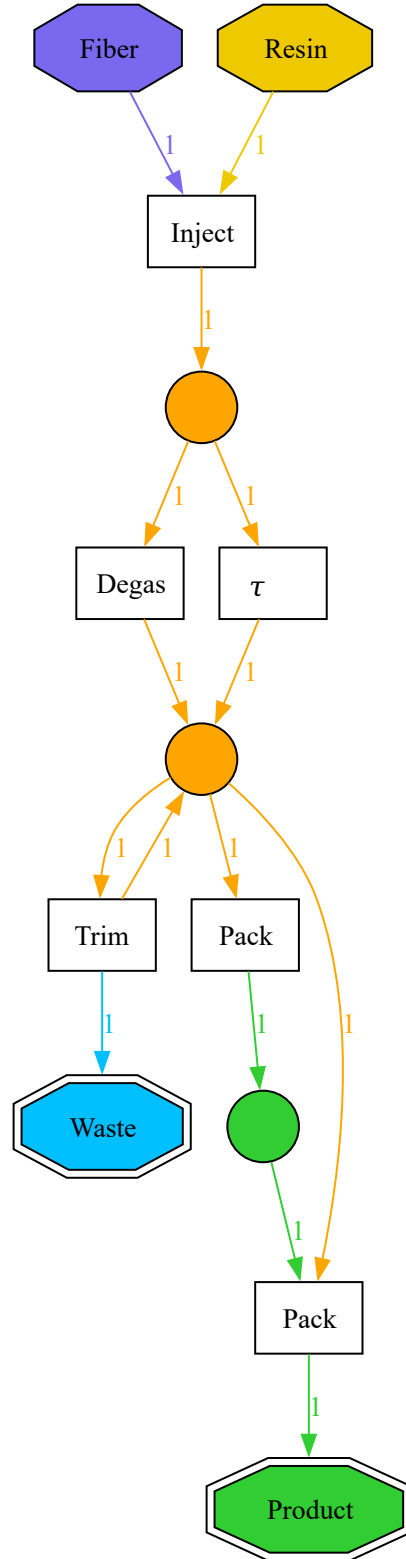


Figure 1.1: The production process of plastic cases in our fictional example, discovered by our proposed method.

1 Introduction

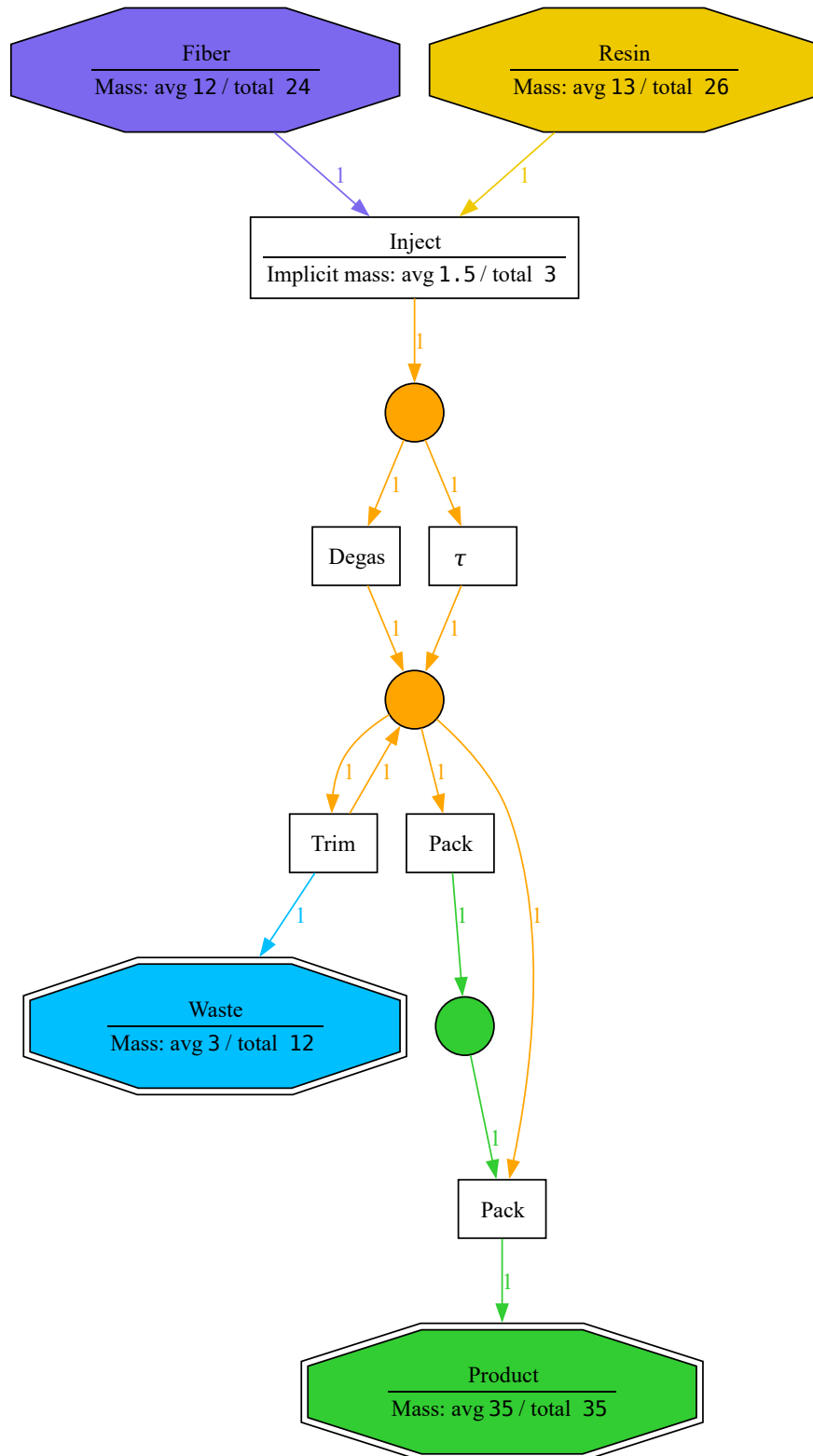


Figure 1.2: The annotated OFN of the plastic production process.

To illustrate this issue, we apply the object-centric inductive miner [2] implemented in `pm4py` [8] to the OCEL from the previous section. The resulting OCPN in Figure 1.3 does not show whether the object types are process inputs, process outputs or neither of those, as each object type has precisely one initial and final place. There are more silent transitions representing loops than in the OFN model, which hinders readability. The OCPN generated by the inductive miner is less precise than the OFN, since *Pack* can be looped with instances of *Product*, although in the log this is not the case; in reality, *Pack* combines the previous *Product* with a new mold coming from *Trim*, and puts it into another *Product*.

Since the Inductive Miner is based on the control flow, and does not know the inputs and outputs of an event, the model does not differentiate between the case where *Pack* takes a single *Mold* and gives a *Product*, and the case where *Pack* takes a *Mold* together with a *Product* as input. Moreover, *Pack* contains a variable arc for *Product*, and therefore does not show the exact quantity of products associated with it. In contrast, the OFN that our algorithm produces contains two transitions of the *Pack* activity, both showing the quantity of *Product* consumed and produced. This ability of the object flow miner is essential for distinguishing the roles of transitions with the same activity in the object flow.

The major conceptual difference between an OCPN and an OFN is that transitions serve a different purpose. In an OCPN, a transition represents a synchronization point in the life cycle of objects, which possess an identifier and exist before and after the transition. In contrast, in an OFN, there are regular, non-identifier tokens, and a transition represents a transformation of the input tokens into the output tokens. Thus, in an OFN, tokens representing objects are consumed or produced by transitions and might not exist before or after a transition is fired.

Moreover, as the Inductive Miner does not know which objects the inputs and outputs of each activity are, it is not able to quantify them and, thereby, unable to determine the waste in this process.

1.3 Research Questions

In the domain of using OCPM, specifically automating LCI analysis and detecting waste, we investigate the following research questions:

1. How can a model of the object flow in an event log be discovered, including the inputs and outputs at the process level?
2. How is waste recorded in an OCEL?
3. How can this waste be read from an OCEL and visualized in the model?
4. To what extent is the LCI analysis phase automated by the OCPM approach?

1 Introduction

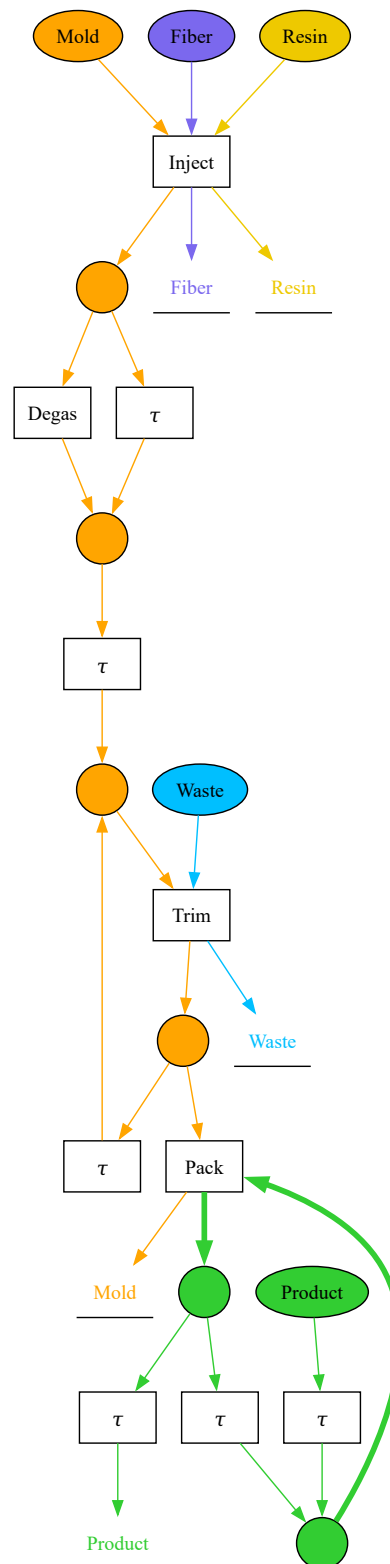


Figure 1.3: The production process of our fictional example, discovered by the object-centric inductive miner.

1.4 Research Goals

To answer the aforementioned research questions, our research goals are as follows:

- Develop a process discovery algorithm that mines the object flow in an event log, including the inputs and outputs on the process level, and yields a process model based on that.
- Determine the types of waste that can occur in an event log.
- Show how information about this waste is integrated into the discovered process model.
- Demonstrate how this approach is used to automate LCI analysis in an LCA.

1.5 Contributions

In an effort to achieve the research goals, we contribute the following to the field of sustainability using process mining:

- A review of the literature on the use of process mining to automate LCA.
- A process discovery algorithm that mines a model based on Petri nets, where each transition describes a combination of input and output object types that occur with this activity.
- A summary of the representations of waste that can occur in an event log.
- Methods that extract the different types of waste from a log, and visualize it in the model generated by the discovery algorithm.
- An automatic procedure to perform the LCI analysis phase in LCA.

1.6 Thesis Structure

The remainder of this thesis is structured as follows. In Chapter 2, we present the concepts of LCA and OCPM as they are used throughout this thesis. In Chapter 3, we discuss related work in the domain of applying process mining to LCA and other techniques to improve sustainability. Subsequently, we show the method we developed in Chapter 4. For further specification, we present a possible implementation of a prerequisite of our method in Chapter 5, which also contains an explanation of a user interface for our method. We evaluate the approach in Chapter 6. Afterward, in Chapter 7, we discuss the design choices we made and their implications, and also the expressivity of our evaluation. The conclusion of the thesis is given in Chapter 8, together with some directions for future work.

2 Preliminaries

In this chapter, we give the mathematical concepts used in this thesis, and in addition explain the concepts Life Cycle Assessment, Material Flow Analysis, and object-centric process mining. We first introduce the mathematical foundations. As our work aims to support LCA, we present LCA second, followed by MFA. Subsequently, we define the OCPM essentials that we use, including the event log format and Petri nets, which are both slightly modified for our purposes in this thesis.

2.1 Mathematical Foundations

We use the usual notation for sets, such as union $\cup$, subsets $\subseteq$, and equality $=$. We also use functions $f : X \rightarrow Y$ that map $x \in X$ to $f(x) \in Y$. We extend partial functions to total functions by mapping inputs that are not in the domain to $\perp$ for a simpler notation. Tuples $(a, b, c) \in \{a, b, c\}^3$ provide a fixed number of values in order. To obtain the element at index i from a tuple t , the notation $\pi_i(t)$ is used.

Given a set S , a multiset is a function $M : S \rightarrow \mathbb{N}$, where each element can occur multiple times, which we call the multiplicity. We denote them in brackets with the multiplicity of each element in superscript, such as $M = [a^1, b^2, c^3]$. The set of possible multisets of a set S , which differ in multiplicity of the elements, is $\mathcal{B}(S)$. In this example, $M \in \mathcal{B}(\{a, b, c\})$. The multiplicity of an element $x \in M$ is obtained through the multiplicity function $m_M(b) = 2$. To obtain the support set of a multiset, we use $\text{supp}(M) = \{m \mid m \in M\}$, so, for example $\text{supp}(M) = \{a, b, c\}$. The additive union of two multisets adds the multiplicities of the elements, as in $M \uplus M = [a^2, b^4, c^6]$. Equality $M = N$ holds if the multiplicity of each element is equal in both M and N . The absolute value $|M|$ of a multiset is the sum of the multiplicities of all elements.

We also define a strict total order as a binary relation that is irreflexive, asymmetric, transitive and connected. An equivalence relation is a binary relation that is reflexive, transitive, and symmetric. It partitions the universe into equivalence classes, of which any element can function as the representative.

2.2 Life Cycle Assessment

A Life Cycle Assessment (LCA) is a study on any specific product with respect to its environmental impacts, beginning with the extraction of the resources it requires, its manufacture, use and disposal, as well as all logistical operations necessary for these steps [9]. The goal is to gain an understanding of the holistic process and its environmental impact, which is then used to identify opportunities for improvement [18]. The life cycle approach is essential for sustainability [18], and LCAs are commonly used and supported by commercial software [41]. Because of this holistic view, LCA is also called cradle-to-grave analysis. A restricted form can also be applied, for instance, to a gate-to-gate analysis, where *gate* refers to the entrance or exit of a factory, since tracking the prerequisites and use of a product can be difficult. The measured environmental impact can be of a diverse nature, such as natural resources, human health, and ecological consequences [23].

The well-known carbon footprint is an example of an LCA study in which the environmental impact measured is global warming potential [44]. The newly acquired information on environmental impacts can be used to improve the impact, inform decision makers, or promote eco-friendliness to customers [24]. In the context of LCA, waste is defined as “substances or objects which the holder intends or is required to dispose of” [24], which is the understanding of waste that we use throughout this thesis.

The principles of LCA were standardized in 2006 by the *International Organization for Standardization* in ISO14040 [24], which specifies the main principles, and ISO14044 [25], which details the guidelines for applying LCA. Since then, it has become one of the most important methods for measuring environmental impacts, and is widely applied [32]. We summarize what is necessary from the standard for this thesis. According to the ISO standard, an LCA consists of the following four phases:

1. In the **Goal and Scope Definition**, the intended application, motivation, and target audience of the study are given. The assumptions taken, data requirements, and limitations are presented as well. In addition, the system to be studied, including its boundary and function, and the environmental impacts of interest are defined.
2. Afterwards, in the **Life Cycle Inventory Analysis** (LCI analysis), the data necessary for the assessment is collected, validated, and aggregated. This includes the inflows and outflows of the system of different types, such as resources, goods, products, waste and emissions. After this step, all inputs and outputs of the process are known. The LCI analysis is the most time-consuming step and its outcome depends on the quality of available data [32].
3. Subsequently, in the **Life Cycle Impact Assessment**, the results of the LCI analysis are assigned to some chosen impacts, which are used to calculate a number that quantifies them.

2 Preliminaries

4. Then, in the **Life Cycle Interpretation**, an evaluation of the previous process is performed. This considers completeness, limitations and identification of significant issues.

In the standard, a concrete list of methods specifying how to perform LCA is intentionally not given, as it is supposed to be open and widely applicable in different scenarios, even for new approaches [24], for which process mining is an example.

To simplify computations and comparisons, an LCA calculates impacts with respect to a single unit of the process output, the functional unit (FU). This is done for simplification purposes, and allows one to see the impact contribution of an intermediate product to the final product. However, a process can be too complex to be broken down in relation to a single FU, especially processes as large as a whole life cycle. If there are multiple or varying outputs, for instance, if waste is produced throughout the process or if a production process gives two distinct products, the attribution of emissions to a single FU is not always obvious [29].

Due to the complex nature of many life cycles, processes that contribute less than 1% to the impacts are often excluded from consideration [29]. The actual environmental impact of actions is often not measured, and instead existing LCA databases are used that provide common values for emissions based on industry and location [40]. As a result, an LCA can have inaccuracies in its calculated impacts. For these reasons, an LCA can be considered as giving a “best estimate” of the average impacts of the process [9].

2.2.1 Example

Suppose that biofuel, fuel made by converting biomass [14], should be used as an energy source for cars. Without in-depth consideration, with regard to the ecological impact, this fuel appears to be a much better source of energy than natural gas, because all carbon dioxide emitted by burning the biofuel has already been absorbed by the plant matter before, and therefore it does not represent an addition of CO₂ to the atmosphere. The life cycle of the fuel is visualized in Figure 2.1.

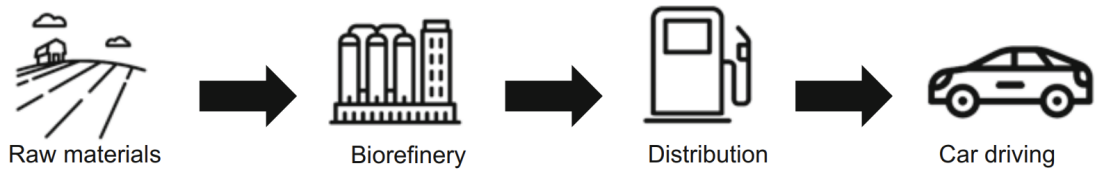


Figure 2.1: Life cycle perspective of biofuel production, taken from [9].

However, by performing an LCA the issue becomes clear. For that, we set the system boundary as a cradle-to-grave process, from the cultivation of the crops to the disposal

by burning the biofuel in a consumer car. The goal is to find out whether biofuels are actually “climate neutral”, so greenhouse gas emissions are chosen as the impact of interest. We perform the LCI analysis for some quantity biofuel as the FU. The inventory includes energy inputs, and we see that the harvest and transport of the biomass, like crops, requires energy. The biomass then has to be converted to biofuel, which also requires energy, and afterward the biofuel has to be distributed before being burned and emitting CO_2 into the atmosphere. Suppose that in our example, these processes use fossil fuels, for example the distribution to a gas station. In the life cycle impact assessment phase, we assign the emissions resulting from the energy consumed by the production and transportation steps to the FU, the biofuel. During the life cycle interpretation, after calculating the total energy input and because biomass extraction cannot power itself, it becomes clear that due to the production process, the biofuel is not climate neutral [9], and for every burned liter of biofuel that came from biomass that bound CO_2 from the atmosphere, there are significant additional emissions coming from fossil fuels that were not previously in the atmosphere.

2.3 Material Flow Analysis

Since this thesis is focused on the LCI analysis phase, because that is where the underlying data is used to find the quantities of inputs and outputs of the process [29], we discuss it in more detail now. In addition to LCA, another environmental analysis framework called *Material Flow Analysis* (MFA) is often used in practice to perform the LCI analysis phase [12]. In an MFA, a model of the inflows and outflows, and of the inner material flow of the system is built, and each material is quantified by some metric, such as weight or volume [12]. Thus, by performing an MFA, one obtains a detailed model of the process supported by quantitative data. This model serves the LCI analysis.

Like an LCA, an MFA requires a system boundary. Within the system boundary, materials are transported, transformed, or stored. These materials can be of a diverse nature, such as chemical substances, economic goods, electricity, and emitted gas. The flow of the materials in the process is investigated and put into relation with each other. This creates a model of material flows between stocks of those materials [12].

The power of an MFA derives from the mass-balance principle, which states that the sum of the mass flowing inside the system must equal the sum of the mass flowing outside. If that principle holds in the empirical data, this ensures that the model of the process is consistent. This is critical for LCI analysis, as otherwise the subsequent impact assessment is done on faulty data. It is also possible to determine the input and output flows exactly, for instance the amount of waste material that leaves the system [12].

2.3.1 Example

The chemical element phosphorus is essential for modern life, but is also harvested from a finite and non-renewable supply of minerals that could be exhausted in 60 to 100 years [42]. This makes the role of phosphorus particularly interesting to analyze because savings can have a great effect on future generations.

So, we consider the usage of phosphorus as our system, with only the direct mass of phosphorus to be considered. We set the system boundary to the entire world, which on the one hand of course comes with some practical issues, but on the other hand brings so much abstraction that the MFA is easy to understand. For our exemplary purposes, the data is given by [42] as percentages. Using them for a primitive MFA, we get Figure 2.2, which shows that 80% of all mined phosphorus is dedicated to agriculture, and only 8% to private households. The numbers are given in $\frac{\text{kg}}{\text{capita} \cdot \text{year}}$. Note that the sum of the masses leaving the system boundary is equal to the mass entering, so the mass balance principle is fulfilled.

In terms of LCI analysis, we see that the process inputs are $5 \frac{\text{kg}}{\text{capita} \cdot \text{year}}$, and $4 \frac{\text{kg}}{\text{capita} \cdot \text{year}}$, $0.6 \frac{\text{kg}}{\text{capita} \cdot \text{year}}$ and $0.4 \frac{\text{kg}}{\text{capita} \cdot \text{year}}$ are the outputs.

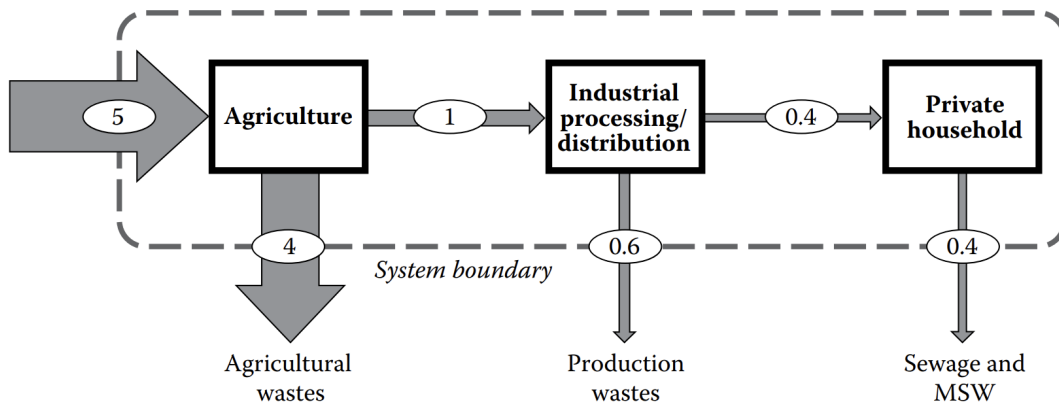


Figure 2.2: MFA of global phosphorus use, taken from [12].

2.4 Object-Centric Process Mining

Process mining is a data-driven and bottom-up set of techniques in Business Process Analysis for analyzing processes based on event logs, which document relevant business events. There are various methods, including process discovery, where a model of the underlying process is automatically created, conformance checking, which checks how

well a given model fits the data, performance analysis of a process execution, and mining of organizational structures [1]. In this thesis, we work on process discovery. Process discovery tools are useful because in the real world, there are often discrepancies between what the desired model looks like, for example the production process in a company, and what actually happens in reality, so simply creating a to-be model is not sufficient. By analyzing the process models found based on the empirical data, understanding of the process is deepened, deviations can be identified, and the process can be improved. Process mining therefore provides a framework for dealing with real-world processes algorithmically. While classic process mining works on event logs where each event belongs to exactly one execution of the process, this notion has recently been extended to object-centric event logs, which relate events with a number of objects, and these objects can be related to other events as well. This extension possesses the ability to represent data from any modern information system [2]. The ecosystem around OCELs is Object-Centric Process Mining (OCPM).

2.4.1 Object-Centric Event Logs

Object-Centric Event Logs (OCELs) are the backbone of OCPM, since they supply the data on which the algorithms work. In OCELs, an event is not necessarily related to a single case as in classical process mining, and indeed there is no case notion. Instead, an event can be related to any number of objects and an object can be related to any number of events, and both relations can have qualifiers describing them in more detail. Both events and objects are of specific types and are further characterized by attributes with arbitrary values.

Let us first define the components that the OCEL uses [7]:

Definition 2.4.1 (Universes). Let $\mathbb{U}_\Sigma$ be the universe of strings. We define the following pairwise disjoint universes:

- $\mathbb{U}_{ev} \subseteq \mathbb{U}_\Sigma$ is the universe of events.
- $\mathbb{U}_{etype} \subseteq \mathbb{U}_\Sigma$ is the universe of event types (i.e., activities).
- $\mathbb{U}_{obj} \subseteq \mathbb{U}_\Sigma$ is the universe of objects.
- $\mathbb{U}_{otype} \subseteq \mathbb{U}_\Sigma$ is the universe of object types.
- $\mathbb{U}_{attr} \subseteq \mathbb{U}_\Sigma$ is the universe of attribute names.
- $\mathbb{U}_{val}$ is the universe of attribute values.
- $\mathbb{U}_{time}$ is the universe of timestamps (with $0 \in \mathbb{U}_{time}$ as the smallest element and $\infty \in \mathbb{U}_{time}$ as the largest element).

2 Preliminaries

In this thesis, we use a modified definition of OCEL 2.0 event logs with multiple modifications: qualifiers are not used, there is a total order of events, and object attribute changes do not occur. Qualifiers are omitted because our algorithm simply needs any relation to exist, so the definitions are simplified if there is at most one element in the relation. The algorithm also compares timestamps between events, so if two different events have the same timestamp, it is not trivial which event should be considered first. Therefore, we require a total order of events. We also assume that there are no object attribute changes. As our algorithm uses object attributes, additional complexity would be introduced by the decision of which attribute is valid at which time. To make a log satisfy this assumption, object attribute changes shall be replaced by creating a new object when an attribute changes. We specify our modified OCEs, based on the original definition [7]:

Definition 2.4.2 (Modified OCEL). A modified Object-Centric Event Log is a tuple

$$L = (E, O, EA, OA, evtype, time, objtype, eatype, oatype, eaval, oaval, E2O)$$

with

- $E \subseteq \mathbb{U}_{ev}$ is the set of events.
- $O \subseteq \mathbb{U}_{obj}$ is the set of objects.
- $evtype : E \rightarrow \mathbb{U}_{etype}$ assigns types to events.
- $time : E \rightarrow \mathbb{U}_{time}$ assigns timestamps to events.
- $EA \subseteq \mathbb{U}_{attr}$ is the set of event attributes.
- $eatype : EA \rightarrow \mathbb{U}_{etype}$ assigns event types to event attributes.
- $eaval : (E \times EA) \not\rightarrow \mathbb{U}_{val}$ assigns values to event attributes (not all attributes are mapped for each event).
- $objtype : O \rightarrow \mathbb{U}_{otype}$ assigns types to objects.
- $OA \subseteq \mathbb{U}_{attr}$ is the set of object attributes.
- $oatype : OA \rightarrow \mathbb{U}_{otype}$ assigns object types to object attributes.
- $oaval : (O \times OA) \not\rightarrow \mathbb{U}_{val}$ assigns values to object attributes.
- $E2O \subseteq E \times O$ are the event-to-object relations, without qualifiers.

such that

- $\text{dom}(eaval) \subseteq \{(e, ea) \in E \times EA \mid evtype(e) = eatype(ea)\}$ to ensure that only existing event attributes can have values.
- $\text{dom}(oaval) \subseteq \{(o, oa) \in O \times OA \mid objtype(o) = oatype(oa)\}$ to ensure that only existing object attributes can have values.

- For the events $e \in E$, the $time(e)$ form a total order. We then also say that there is an order between $e \in E$ and $e' \in E$, determined by their timestamps.

We also use the following notation from the specification of OCELs [7]:

- $ET(L) = \{evtype(e) \mid e \in E\}$ is the set of event types.
- $OT(L) = \{objtype(o) \mid o \in O\}$ is the set of object types.
- $oaval_{oa}(o) = oaval(o, oa)$ provides the object attribute oa of o .

Since there is a total order of events, we introduce two functions that provide the previous and the next event that an object is related to.

Definition 2.4.3 (Previous and next event). Let $(e, o) \in E2O$. Let $prev : O \times E \not\rightarrow E$ be the event related to o that occurs directly before e , if it exists. Let $next : O \times E \not\rightarrow E$ be the event related to o that occurs directly after e , if it exists.

2.4.2 Example

We use the plastic production from our motivation in Section 1.1 here and as the running example in our method. To represent the total order of events based on timestamp, we assume that the rows in the Table 1.1 are sorted by ascending timestamps. In the object table Table 1.2, each object identified by ID in the first column shows its object type in the second column, and a custom attribute *mass* that takes an integer that stores the weight of the object. *Mold* objects do not have a known *mass* attribute, as they are purely inside the reaction vessel and cannot be measured like the other objects that enter or leave the system. There are no *O2O*-relations.

In our running example, there are two main process executions, from *Inject* to *Pack*. In the first, *Degas* is executed, while this is not the case in the second execution. Additionally, in the first execution *Trim* is only executed once and only a single waste object is created, while there are three such events in the second execution. Moreover, the first *Pack*, namely **pack_3**, only uses the mold created from the first execution to create a *Product*, while the second *Pack*, **pack_4**, combines that product with the mold from the second execution into another *Product*. This entanglement of the process executions is also why an object-centric log is necessary here.

The mass serves as an example for the annotation of waste that we present later in this thesis, and is not relevant for the discovery of the process model.

2.5 Petri Nets

A common process model is a Petri net. A Petri net is a graph that consists of two types of nodes, places and transitions. They keep tokens in places and move them using transitions. The edges indicate which places and transitions are connected, where every edge outgoing from a place must lead to a transition, and every outgoing edge from a transition must lead to a place. When a transition fires, it moves a token from each input place to each output place.

We use labeled Petri nets with and without weighted arcs. We define Petri nets with weighted arcs as follows [13]:

Definition 2.5.1 (Petri net with weighted arcs). A *labeled Petri net with weighted arcs* is a tuple $N = (P, T, W, l)$ where P is the finite set of places, T is the finite set of transitions, and $W : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$ is the weighted flow relation. Let $W(p, t) = 0$, or $W(t, p) = 0$, if the arc from p to t or t to p , respectively, does not exist. $l : T \not\rightarrow \mathbb{U}_{act}$ is a labeling partial function, where transitions that are not mapped are silent transitions.

To represent the tokens in the places, we use markings:

Definition 2.5.2 (Marking). A *Marking* of a labeled Petri net with weighted arcs is a multiset of places $M \in \mathcal{B}(P)$. $M[p]$ is the number of tokens in $p \in P$.

To fire a transition $t \in T$ given a marking M , it must be enabled, which is the case if each input place p with $W(p, t) > 0$ contains at least as many tokens as the arc weight requires, or for all $p \in P$ it is $M[p] \geq W(p, t)$. Firing it results in the updated marking $M'[p] = M[p] - W(p, t) + W(t, p)$ for each $p \in P$, where input tokens are consumed and output tokens are produced according to the weights.

A variable arc can consume or produce an arbitrary number of tokens, but at least one. Petri nets with such arcs are defined as follows:

Definition 2.5.3 (Petri net with variable arcs). A *labeled Petri net with variable arcs* is a tuple $N = (P, T, F, V, l)$ where P is the finite set of places, T is the finite set of transitions, and $F, V \subseteq (P \times T) \cup (T \times P)$ define the regular and variable arcs such that $F \cap V = \emptyset$. $l : T \not\rightarrow \mathbb{U}_{act}$ is a labeling partial function, where transitions that are not mapped are silent transitions.

In these nets, given a marking M , a transition $t \in T$ is enabled if $M[p] \geq 1$ for each $p \in P$ with $(p, t) \in F \cup V$. When t fires, the marking in place $p \in P$ is updated as $M'[p] = M[p] - \hat{c} + \hat{p}$, where $\hat{c}$ and $\hat{p}$ are given by the following cases:

Non-variable arcs. If $(p, t) \in F$, then $\hat{c} = 1$, and if $(t, p) \in F$ then $\hat{p} = 1$.

Variable arcs. If $(p, t) \in V$, then $\hat{c}$ is chosen from 1 to $M[p]$, and if $(t, p) \in V$ then $\hat{p} \geq 1$ is chosen.

No arcs. If there is no arc, that is $(p, t) \notin F \cup V$, then $\hat{c} = 0$, and similarly $\hat{p} = 0$ if $(t, p) \notin F \cup V$.

2.6 Resources and Handling Units

OCEs aim to holistically capture a company process. As part of these processes, we differentiate between objects that act as resources or handling units. On the one hand, a resource is an object that describes what an event works *with*, while it is not directly the target. On the other hand, a handling unit describes what the event operates *on* [22]. Assume that we added objects for the machines that performed the actions in our running example, such as *Trimmer* that performs the *Trim* activity. As one *Trimmer* performs many *Trim* events, it represents a resource that works on the handling units *Mold* and *Waste*. A linear programming approach can be used to separate object types into multiple levels, where types on a level act as a resource for lower levels but are handling units for higher levels[35].

3 Related Work

As this thesis provides a bridge between the fields of Life Cycle Assessment and process mining, we present papers from both fields. We first present existing approaches to automate modeling and LCI analysis in the general LCA literature. Subsequently, we investigate papers exploring the link between sustainability and LCA and process mining. As our method aims to automate LCI analysis by discovering the object flow, we also present existing discovery algorithms from the OCPM literature and show why they are not adequate for this purpose.

3.1 Automated Modeling in Life Cycle Assessment

Various LCA steps can be supported by automation. In this thesis, we focus on automatic modeling of the process and the LCI analysis phase, since this is where process mining can be used. There are not many approaches to this general problem: Haun et al. [20] state that “there are no approaches to improve LCI modelling within automated LCAs, at least not for such complex product systems such as entire vehicles with the aspiration of an accurate LCI modelling on component level”. While some concepts exist, the authors find that none of the publications examined in their literature review gives a detailed strategy. Most approaches are in civil engineering, where automatically generated models of buildings are used for LCI analysis of the construction materials used in that building [41]. This modeling is applied in the design phase of a building, instead of relying on live event data like our approach. Another review of automation in LCA by Köck et al. [32] finds various LCA workflows in which automation is employed, from data extraction to process simulation and updating databases. They introduce *process synthesis*, which is the automation field that is closest to automatically modeling the process. It describes methods in chemical engineering that use solutions of optimization problems to find more efficient means of producing chemicals [26]. These techniques source data from LCA databases and use the mass-balance principle, but this is run as a simulation before the process is executed, whereas our method utilizes event log data generated during the process execution. In fact, automated calculation of impacts and improved reporting are specifically mentioned as fields that future work can improve in the same review. In our approach, we automatically calculate the impact of waste and discover a process model. Therefore, we consider this thesis to contribute to filling this research gap.

3.2 Sustainability Analysis Using Process Mining

As previously discussed, current approaches are model-driven, leaving a gap when it comes to data-driven analysis of process executions. Process mining is a tool to address this gap and assist in automated LCA. Through expert interviews, Brehm et al. [11] found that process mining can generally help in the field of carbon accounting, by making emissions visible and allowing the simulation of alternative processes. Although the literature on process mining has recognized its possibilities in the domain of sustainability, most approaches have only been high-level descriptions [17]. Multiple general issues that regular LCAs have, in particular that they are time consuming, costly, difficult to update, and only capture a snapshot of the system, can be tackled using process mining [37].

A concrete framework for supporting an LCA through the use of process mining is Sustainability-Oriented Process Analysis (SOPA) [27, 28]. It defines a workflow of manually creating or modifying a generated process model and assigning environmental impacts to activities or process executions, which can be summed or averaged to obtain emissions, so that a single holistic sustainability indicator can be calculated. Through simulations, changes in the indicator that would result from process changes can be observed. The SOPA framework operates on classic event logs, and in the future work section, the authors suggest using OCPM. The idea of the framework is similar to the approach presented in this thesis, but our focus is the automatic, as-is annotated model generation, instead of needing a process expert to manually create a should-be process first. Particularly for waste detection, it is often useful to investigate the actual process log to find irregular and unexpected waste, instead of relying on an idealized model that does not adequately capture the unintended waste generation. In addition, in SOPA, impacts are assigned to activities, whereas in our model, they can also be associated with objects.

Graves et al. [19] give a review of specifically the possibilities of using OCPM in sustainability analysis, and suggest applications in discovery, conformance checking, process analysis, and predictive process mining. According to the authors, many aspects of a product life cycle can be analyzed and optimized using OCPM. In this thesis, we focus on discovery and process analysis. For LCAs that study logistic processes, the material flow of objects rather than the control flow is of particular interest. The authors therefore suggest that it is necessary to build a model based on the inputs and outputs of each activity, which is the primary research gap we address in this thesis. Fritsch et al. [17] agree that the data flow perspective, that is, the input and output objects relevant to sustainability, is essential for LCI analysis. Furthermore, processes that work on different materials and do not follow a case notion are common [17]. This represents a limitation of classic process mining, where OCPM can be applied instead. Therefore, our focus is the development of an object-centric process discovery algorithm that analyzes the material flow, that is, which object types are inputs and outputs in activities, and also discovers inputs and outputs at the process level. It mines a Petri net in such a way

3 Related Work

that the inputs and outputs of each transition represent the inputs and outputs from a material flow perspective, which is identical to the object flow or data flow. This algorithm is then used for waste detection, which annotates this Petri net with information about waste.

Various problems that arise in LCA can be improved by using process mining. An LCA is about a specific product of a specific organization and therefore needs to be created individually by domain experts. This requires time and money [12]. Using an automated OCPM tool, the process can be streamlined and carried out more efficiently.

Similarly, due to this manual creation process, LCAs only capture a specific period of time, and changes in the environment can go unnoticed. In general, LCAs struggle to react to dynamic changes [33]. Using our automated workflow, newly gathered data can immediately be incorporated into the model to give a more accurate picture of reality. The ease of splitting the data and discovering models on subsets also enables OCPM to see trends in the data over time.

LCA studies often neglect the consistency of the data, as stated by Ayres et al. [5]. In fact, the authors mention ignoring waste streams as an example where data consistency is commonly ignored, which shows why the method proposed in this thesis is useful. Using our detection of implicit waste, waste streams that are ignored can be quantified.

Because an LCA works with simplified calculations, problems can arise if it is applied to a large operation, such as an entire production plant. This makes abstractions necessary, and information about fine-grained events is lost. Because OCPM has the algorithmic ability to handle each event on an individual level, it is better suited than classic LCA to handle such data. Using objects and thus being able to precisely track each object's life cycle eliminates the need to calculate all impacts relative to a single output. In addition, the algorithms are designed to work on large amounts of real, fine-grained data, and thereby do not require abstracting away steps to make the calculations feasible by hand.

A challenge in applying process mining to LCA is that the underlying data must provide sufficient details about sustainability, which is rare [40]. Consequently, LCA databases are often used to populate the model, but these contain reference values instead of real values. According to Schäfer et al. [40], while extracting emission data from existing ERP systems is difficult, the necessary data to perform waste analysis does exist in current data sets already. For this reason, we believe that the application of process mining to waste detection is a reasonable focus for this thesis. In particular, objects that are explicitly marked as waste and objects that exist unintentionally are the two types of waste mentioned. Both types can be discovered by the method we present in this thesis.

3.3 Object-Centric Process Discovery

In this section, we present object-centric discovery algorithms and models to explain why a novel algorithm and model are needed for the object flow. We do not review mining techniques for classic event logs, since our method is fundamentally based on exploiting the existence of individual objects and does not require a fixed case identifier.

As discussed in the motivation in Section 1.1, the object-centric Petri nets created from the object-centric inductive miner do not satisfy our requirements. An object-centric miner that discovers an Object-Centric Process Tree is the DF² miner [43]. The resulting object-centric process tree shows the object types related to an activity, but does not show cardinalities of the object types or transformations of objects through activities. Due to the tree structure, two nodes at different heights can follow each other in the control flow perspective, which also makes object-centric process trees unsuitable for showing the material flow. A strength of the algorithm is that for every object-centric process tree, a live and identifier-sound object-centric Petri net is created.

Another approach is Object-Centric Dynamic Condition Response Graphs [15]. These are graphs in which activities are nodes. The model is then composed of subgraphs that represent the life cycles of objects of an object type, which overlap with activities that the object is related to. Between activities, there can be a conditional, response, and excluding relationship. Therefore, these models are based on the relations between activities, which are different from material flow that shows transformations of objects. The subgraph notation for object types is not suitable for showing a material transformation due to limited overlap possibilities.

A model that clearly shows the related object types and cardinalities is an Object-Centric Behavioral Constraint Model [34]. These contain activities and object types. Between activities, there are constraints based on time. Between activities and object types, as well as between a pair of object types, there can be cardinalities. Again, these models are based on constraints, instead of transformations of the material flow. Furthermore, their purpose is validation, instead of discovering irregularities such as waste in the data.

In summary, none of the existing approaches concisely shows the object flow, especially the inputs and outputs of each activity, in a clear and intuitive manner. We use Petri nets instead, similar to ecological researchers who have been using Petri nets for sustainability analysis for 30 years [36]. These provide an intuitive way of combining objects with events, and the input and output places can be used to show the object types involved in a transformation, together with weighted or variable arcs for cardinalities.

One model that does show the material flow is Event Knowledge Graphs [16], although on the instance level. These graphs visualize the flow of each object, not the object type, through a graph of events. Therefore, these do not serve our purpose of providing a process model, but they can be used to inspect individual objects; for instance, if an object type has been detected as waste in our model, the flow of its instances through the process can be analyzed.

4 Method

In this chapter, we detail the object flow miner. We first formally define the model that is obtained by the object flow miner, which is an object flow net that shows the object flow and inputs and outputs of each activity. In particular, each transition in the discovered object flow net corresponds to a quantified combination of input and output object types in the event log. The algorithm depends on role functions, which we define next. We then provide the steps of the algorithm. For our original goal of waste detection, we give a method to extend the obtained model with information about waste. For that purpose, we analyze the types of waste in an OCEL. Thus, on a high level, we want to use an OCEL to discover a process model based on inputs and outputs, with annotations for waste. Our algorithm mines this model under the assumption that the input and output in each individual event is already known. As this is not necessarily given in the OCEL, we refer to our implementation in Chapter 5 for how this can be discovered automatically.

The entire method is shown schematically in Figure 4.1. The inputs are the OCEL in purple and the role functions in yellow, and the outputs are different kinds of object flow nets in green. These notions are specified in subsequent sections.

4.1 Object Flow Nets

For this thesis, we introduce a special class of Petri nets, called *Object Flow Nets* (OFNs). These are Petri nets where each place is associated with an object type. In contrast to OCPNs, in which each marking consists of object identifiers [2], OFNs use regular, non-identifiable tokens.

In an OFN, the incoming and outgoing arcs of a transition specify the exact number of tokens consumed or produced, as long as weighted arcs are used. For a fixed object type, it is guaranteed that there are at most two input or output places of that type each. This allows one to directly read how many objects of which object types are consumed or produced by a transition. Otherwise, to abstract from the exact number, variable arcs can be used, that differentiate only between one or many objects. Therefore, an OFN visualizes object flow, where each input and output of a transition occurring in the log is clearly specified.

We use two types of OFNs. First, with weighted arcs, where a specific number of tokens is consumed or produced when a transition is fired. Second, we introduce OFNs with

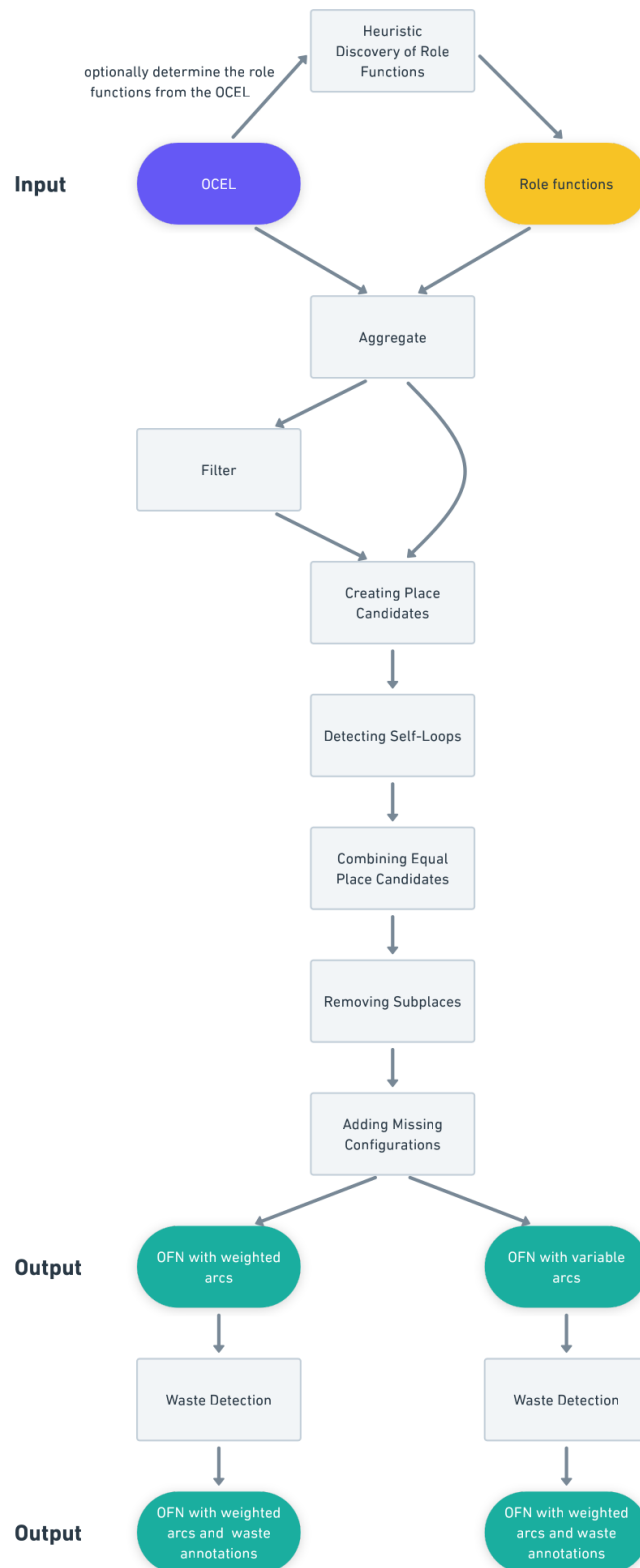


Figure 4.1: The workflow of our method. Created with Whimsical (<https://whimsical.com>).

4 Method

variable arcs. Variable arcs in OFNs do not correspond exactly to variable arcs in OCPNs, since in OFNs they consume or produce at least one token, while in OCPNs they can also consume or produce none.

In both types, places have one of three roles. *Initial* places do not have any incoming arcs, *final* places do not have any outgoing arcs, and places which have both incoming and outgoing arcs are *regular*. This is important for the context of process inputs and outputs, so that, for example, the raw materials in an LCA can be modeled as initial places.

Formally, this gives us the following two definitions:

Definition 4.1.1 (Object Flow Net with weighted arcs). An *Object Flow Net with weighted arcs* is a tuple $OFN = (N, pt, role)$, where $N = (P, T, W, l)$ is a labeled Petri net with weighted arcs, $pt : P \rightarrow OT$ assigns object types to places, and $role : P \rightarrow \{regular, initial, final\}$ maps places to their role.

Definition 4.1.2 (Object Flow Net with variable arcs). An *Object Flow Net with variable arcs* is a tuple $OFN = (N, V, pt, role)$, where $N = (P, T, F, V, l)$ is a labeled Petri net with variable arcs, $pt : P \rightarrow OT$ assigns object types to places, and $role : P \rightarrow \{regular, initial, final\}$ maps places to their role.

Semantically, the firing rules for OFNs are identical to those of the Petri nets they are based on.

An OFN is fully defined by a graphical notation, see for example Figure 1.1 in the motivation in Section 1.1. The places and transitions are as in Petri nets, including a label τ for silent transitions. The object type for each place is given by a color. The initial places have an octagon shape, and the final places are double-bordered octagons. Regular places are ellipses.

4.2 Role Functions

The role functions $role_{in}$ and $role_{out}$ for a given OCEL describe the *role* of each object in the OCEL's events. The role is input, output, initial input, final output, or none. *Initial inputs* and *final outputs* refer to inputs or outputs of the entire process. In LCA terms, they are crossing the system boundary.

While technically an initial input is still an input, for the purposes of the algorithm we differentiate between the two, so that no object can be both in the same event. The same holds for outputs and final outputs. This allows us to define the consistency of the role functions. An initial input o of an event must be truly initial and, therefore, never occur in the log before that event, and similarly, a final output o must never again occur in the log. In addition, for an input o a previous event where that o was an output

must exist, and for an output o , a next event where o is an input must exist. This is formalized as follows.

Definition 4.2.1 (Role functions). For each event and related object $(e, o) \in E2O$, the role of o in e is given by the two *role functions*

$$\begin{aligned} \text{role}_{in} : E \times O &\rightarrow \{in, init, \perp\} \\ \text{role}_{out} : E \times O &\rightarrow \{out, final, \perp\} \end{aligned}$$

init and *final* have to be consistent, that is, for each $e \in E, o \in O$ with $\text{role}_{in}(e, o) = init$ there is no previous event $e' < e$ such that $\text{role}_{out}(e', o) \neq \perp$ or $\text{role}_{in}(e', o) \neq \perp$. Similarly, for each $\text{role}_{out}(e, o) = final$ there is no next event $e' > e$ with the same condition.

If $\text{role}_{out}(e, o) = out$, there must be a next event $e' > e$ with $\text{role}_{in}(e', o) = in$, and also if $\text{role}_{in}(e, o) = in$, there must be a previous $e' < e$ with $\text{role}_{out}(e, o) = out$.

4.3 Object Flow Miner

The object flow miner works in six steps, which are also shown in the visualization of our method in Figure 4.1. As an input, it takes an OCEL and role functions for that OCEL. As an output, it gives an OFN. This OFN is mined by first aggregating events to the activity and object type level by combining events that have inputs and outputs of the same object type. After this step, the different combinations, or *configurations* of object types in inputs and outputs in an activity are known. These configurations have a different effect on the object flow, which is why distinguishing them is key to discovering the object flow. Then, the model is built. For each input and output of a configuration a place candidate is created in the second step. Third, self-loops that implicitly exist in the configurations but are not adequately represented by referring to a single, looping place candidate are handled by creating these looping places. Fourth, identical places are combined. Fifth, places from different configurations are combined by analyzing which places fulfill similar functions. Sixth, missing transitions are created. Finally, an OFN with either weighted or variable arcs is created. Furthermore, we describe how to filter infrequent behavior during step one, which requires filter parameters as additional input to the object flow miner.

To achieve our goal of waste detection, we then provide a method of annotating the resulting model with information about waste, giving a waste-annotated OFN. For that, we differentiate between system-wide, object type, explicit, and implicit waste.

The existence of the role functions for the first step is a strong assumption that is made to allow using different methods for this step. For example, assume that an OCEL

4 Method

has qualifiers on relations that immediately define whether an object is an input or output. This can be directly translated into a role function. If that is not given, we provide a way to discover them automatically using heuristics in our implementation in Section 5.1, and in our evaluation, we see that this is mostly sufficient, although there are some exceptions where the heuristic method needs to be supplemented with domain knowledge. If the automatic discovery of the role functions is used, the algorithm only requires an OCEL as input.

We use the plastic production process as our running example. The OCEL was specified in the introduction in Section 1.1, which presents the input to our algorithm. The role functions were discovered using the heuristic method. The final model that our algorithm produces on our running example is given in Section 1.1, in Figure 1.1.

4.3.1 Aggregation of Events to the Activity Level

The input for the first step is a modified OCEL $L = (E, O, EA, OA, evtype, time, objtype, eatype, oatype, eaval, oaval, E2O)$ according to the definition in Section 2.4.1 and the role functions $role_{in}$ and $role_{out}$ that provide the inputs and outputs for each event instance. The events in the OCEL are aggregated to the activity level; specifically, each distinct aggregation of the same activity represents a different way an activity is used in the log; we call this a *configuration* of that activity. This step outputs these configurations.

Consider our running example. The activity *Pack* can play two different roles in the material flow, despite being only a single activity. In the first process execution, it takes the *Mold* and turns it into a *Product*. Thus, one configuration is a single input *Mold*, and a single output *Product*. But in the second process execution, *Pack* takes this previously created *Product* in addition to another *Mold*. Here, the second configuration is a single input *Mold* and a single input *Product*, which is turned into a single output *Product*.

Inputs and Outputs of Events

In this step, the role functions are used in combination with the OCEL to find the object types of the input, initial inputs, outputs, and final outputs of each event. In addition, two functions that define the previous and next activities of these object types are introduced, so that the context of the neighboring events is kept.

By using multisets, the number of objects in a role is preserved. This later gives rise to

the weight of an arc. For an event $e \in E$:

$$\begin{aligned} In_e &:= [objtype(o) \mid (e, o) \in E2O \wedge role_{in}(e, o) = in] \\ Out_e &:= [objtype(o) \mid (e, o) \in E2O \wedge role_{out}(e, o) = out] \\ Init_e &:= [objtype(o) \mid (e, o) \in E2O \wedge role_{in}(e, o) = init] \\ Final_e &:= [objtype(o) \mid (e, o) \in E2O \wedge role_{out}(e, o) = final] \end{aligned}$$

In combination with the activity, these then form the setting of the event, so we can later aggregate events which have equal settings.

Definition 4.3.1 (Setting). The *setting* of an event $e \in E$ with $act = evttype(e)$ is $setting_e := (act, In_e, Out_e, Init_e, Final_e)$. The events that have the setting s are $Events_s := \{e \in E \mid setting_e = s\}$.

We define two functions that return the previous or next activities that occur for an object type in an event. This is necessary so that configurations that directly follow each other get a place connecting them, similar to how in the α -algorithm directly-follows causal relations result in a place[3]. The *origin* function provides all previous activities of objects of a type in an event. The same is done by the *destiny* function for the next activities. Both map to a multiset, so that the number of times an activity is a previous or next activity is preserved.

Definition 4.3.2 (Origin and Destiny functions). The *origin function* with signature $origin : E \times \mathbb{U}_{otype} \rightarrow \mathcal{B}(\mathbb{U}_{etype})$ is

$$origin(e, ot) := [evttype(prev(o, e)) \mid role_{in}(e, o) = in \wedge objtype(o) = ot]$$

The *destiny function* with signature $destiny : E \times \mathbb{U}_{otype} \rightarrow \mathcal{B}(\mathbb{U}_{etype})$ is

$$destiny(e, ot) := [evttype(next(o, e)) \mid role_{out}(e, o) = out \wedge objtype(o) = ot]$$

Aggregating to the Activity Level

The settings are used to aggregate events with the equal input and output object types and activity to the activity level. This is then combined with the *origin* and *destiny* functions to give the output of the first step of the object flow miner, namely a set of configurations that have some inputs, outputs, and next and previous events for those.

A configuration aggregates all events with equal settings, and therefore equal input and output object types, with the same multiplicities. In addition, for each object type, the possible previous and next activities complete the configuration. They combine the functions *origin* and *destiny* by aggregating the previous or next activities that occur

4 Method

in *any* event that is part of the configuration, meaning that the functions do not have to be equal for two settings.

Definition 4.3.3 (Configuration). Let $s = (act, In, Out, Init, Final)$ be a setting. For the setting s , the *preceding activities* are the combined previous activities of an object type

$$preceding_s(ot) := \biguplus_{e \in Events_s} origin(e, ot)$$

This is a function $preceding_s : \mathbb{U}_{otype} \rightarrow \mathcal{B}(\mathbb{U}_{etype})$. The *succeeding activities* are the combined next activities of an object type

$$succeeding_s(ot) := \biguplus_{e \in Events_s} destiny(e, ot)$$

The *configuration* of a setting s is $configuration_s = (s, preceding_s, succeeding_s)$. The *set of configurations* is $Configs = \{configuration_s \mid e \in E \wedge setting_e = s\}$. Let $act(c_s) := \pi_1(s)$ be the activity of the configuration.

We apply this procedure to our running example. The result is visualized in Figure 4.2. It shows a box for each configuration, which contains the activity name and the input or initial input object types with their multiplicity in the first row, and the output or final output object types in the second row. For two select configurations, the mathematical representation is also shown. The colors representing each object type are explained on the bottom left.

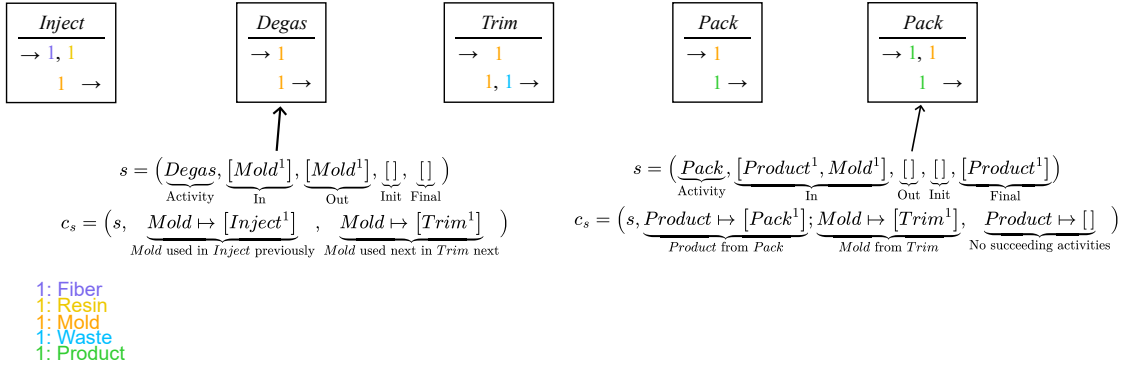


Figure 4.2: The configurations of our running example.

4.3.2 Creating Place Candidates

For every configuration in the set of configurations, we create a place candidate for each input and output. The subsequent steps of the algorithm modify these place candidates, so that the final candidates become places of the resulting object flow net.

A place candidate is a tuple of three elements. First, the object type of the input or output, second a multiset of the activities that precede the place candidate, and third a multiset of the activities that succeed the place candidate. Given a configuration c , a place candidate is created for each related object type. For input or initial input place candidates, the succeeding activity is just the activity of the configuration. The same is true for output and final output place candidates. For initial inputs or final outputs, an empty multiset $[]$ is used for the preceding or succeeding activities, respectively.

Definition 4.3.4 (Place candidates). Let $c_s = (s, preceding, succeeding) \in Configs$ be a configuration with the setting $s = (act, In, Out, Init, Final)$. The *place candidates* of c_s are

$$\begin{aligned} PIn_c &:= \{(ot, preceding(ot), [act]) \mid ot \in In\} \\ POut_c &:= \{(ot, [act], succeeding(ot)) \mid ot \in Out\} \\ PInit_c &:= \{(ot, [], [act]) \mid ot \in Init\} \\ PFinal_c &:= \{(ot, [act], []) \mid ot \in Final\} \end{aligned}$$

For a place candidate p of c_s , we define $otype(p) := \pi_1(p)$, $preceding(p) := \pi_2(p)$, and $succeeding(p) := \pi_3(p)$. Let the function $multiplicity(c, p)$ give the multiplicity of $otype(p)$ in the multiset of the setting that created it, so $In(ot)$, $Out(ot)$, $Init(ot)$ or $Final(ot)$, respectively.

We also consider all place candidates:

Definition 4.3.5 (Set of all place candidates). The *set of all place candidates* is

$$PAll := \bigcup_{c \in Configs} (PIn_c \cup POut_c \cup PInit_c \cup PFinal_c)$$

We note that the definition of each place candidate does not store any reference to the configuration that it was created from. In the following steps, a place candidate can change its preceding or succeeding activities, and the configurations that it is related to. It also implies that we can freely modify the set of place candidates and the weights, stored for each configuration, stay the same.

The result when applied to our running example is shown in Figure 4.3, where for each configuration box, we create place candidates accordingly, colored and annotated by type and showing preceding activities in the first row and succeeding activities in the second row. Initial input place candidates are drawn with an empty multiset as the preceding activities, in this case *Fiber* and *Resin*, which we draw in an octagon shape. Final output place candidates, which do not have succeeding activities, are drawn as a double-bordered octagon, which holds for *Waste* and one *Final Output*. We draw an arc from the place candidate to the configuration if the candidate is input or initial input,

4 Method

and the other way around for outputs or final outputs. For two places of the *Resin* and *Mold* type, we additionally show the representation of this place as a tuple.

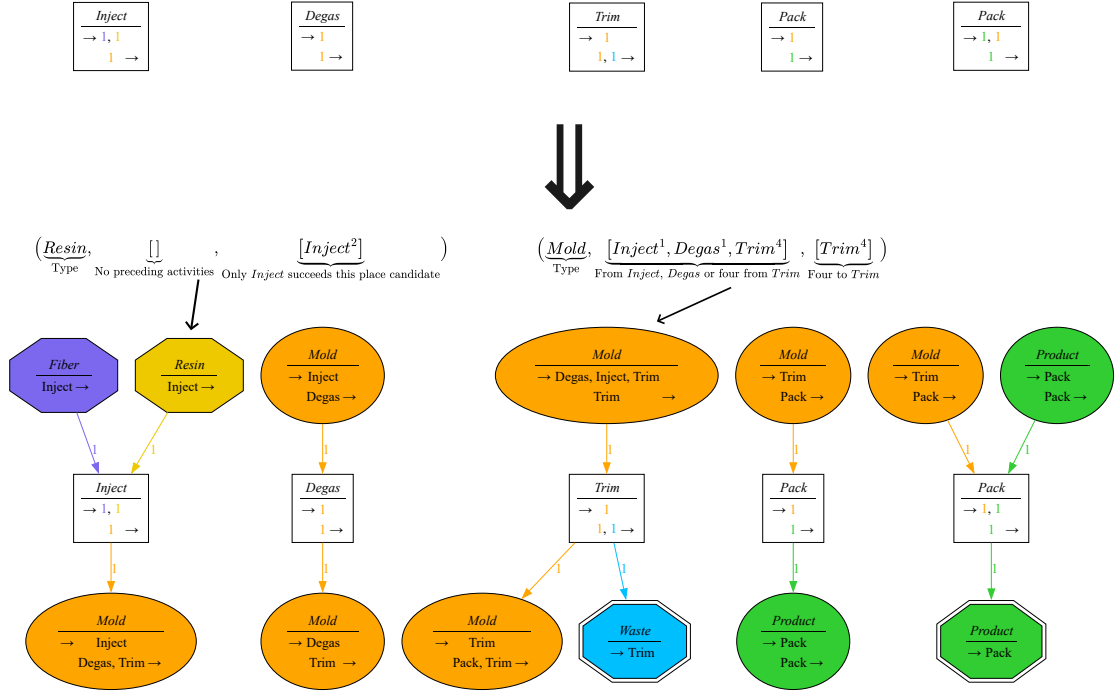


Figure 4.3: The configurations and their associated place candidates of our running example.

4.3.3 Detecting Self-Loops

A self-loop, or length-one loop, is an activity that directly follows itself on an object type. Our model should accurately visualize these self-loops by using the same place as both an input and output place, so this step takes the place candidates and combines the ones that are part of a self-loop.

Self-loop place candidates can be detected by finding the place candidates for which an activity occurs in both the preceding and succeeding activities. This means that the activity occurs twice with the same object with no other activity in between. The place candidates belonging to a self-loop must be of the same object type since that is the type that is looped. They occur in pairs, one for the input and one for the output place candidate of the loop. To create the self-loops in the model, a new place candidate is created that is both input and output, and combines the preceding and succeeding activities of the pair. Clearly, an initial or final place can never be involved in a self-loop.

These requirements are formalized in the following definition:

Definition 4.3.6 (Self-loop place candidates). The place candidates involved in a self-loop belonging to a configuration are

$$\begin{aligned} Loop := \{ (p_{in}, p_{out}) \mid c \in Configs \text{ with activity } act \text{ such that there exist} \\ \text{place candidates } p_{in} \in PIn_c \text{ and } p_{out} \in POut_c \\ \text{with } otype(p_{in}) = otype(p_{out}) \wedge act \in preceding(p_{in}) \wedge \\ act \in succeeding(p_{out}) \} \end{aligned}$$

For a pair $(p_{in}, p_{out}) \in Loop$ belonging to a configuration c , the new place candidate simply contains the previous activities of the input place, and the succeeding activities of the output place: $p_{loop} := (otype(p_{in}), preceding(p_{in}), succeeding(p_{out}))$. It replaces the two place candidates it originates from, so replace p_{in} by p_{loop} in PIn_c , and p_{out} by p_{loop} in $POut_c$.

Our running example has a single self-loop, the detection and new place creation of which are shown in Figure 4.4. Only the portion of the graph considered in this step is shown, and the remainder stays identical to the previous step.

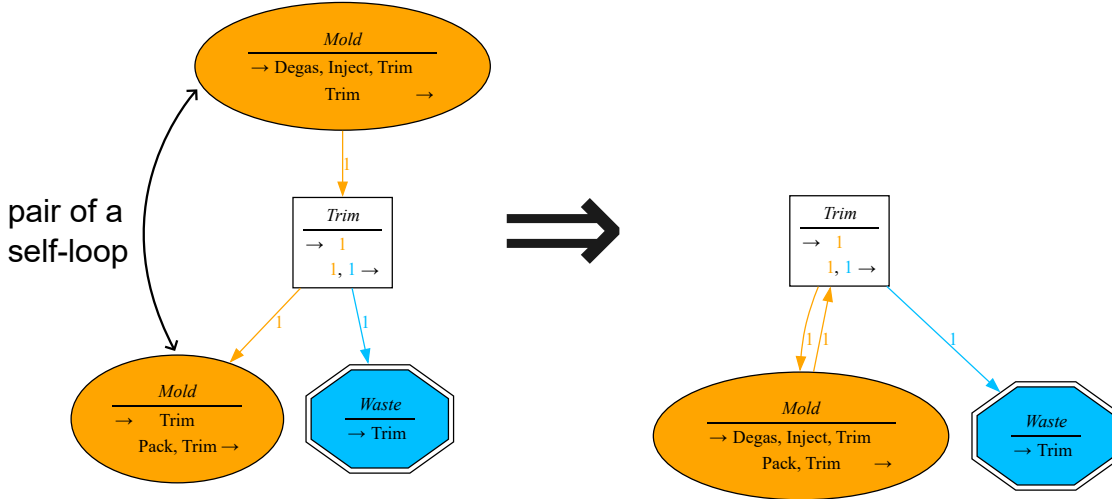


Figure 4.4: The detection and creation of a place candidate for the self-loop in our running example.

4.3.4 Combining Equal Place Candidates

It is possible that two place candidates are equal tuples of the same object type and refer to identical preceding and succeeding activities. This happens if they were created from separate configurations. They should refer to the same mathematical object, so that only one of them remains as a place candidate. This step modifies the place candidates from the previous step by combining these equal place candidates.

4 Method

The multiplicities of the preceding and succeeding multisets are not taken into account. The place candidates that are equal are defined as follows:

Definition 4.3.7 (Equal place candidates). Let $Equal$ be the equivalence relation defined on place candidates $p_1 = (ot, Prec, Succ)$ and $p_2 = (ot', Prec', Succ')$. It is $(p_1, p_2) \in Equal$ iff $ot = ot'$, $supp(Prec) = supp(Prec')$ and $supp(Succ) = supp(Succ')$.

The relation $Equal$ applied to the set of all place candidates $PAll$ partitions place candidates into equivalence classes.

Of each equivalence class, a representative place candidate is chosen and all other members are replaced with that candidate.

The portion of our example where equal place candidates exist and the same portion after their combination is shown in Figure 4.5. Again, the rest of the graph stays as-is.

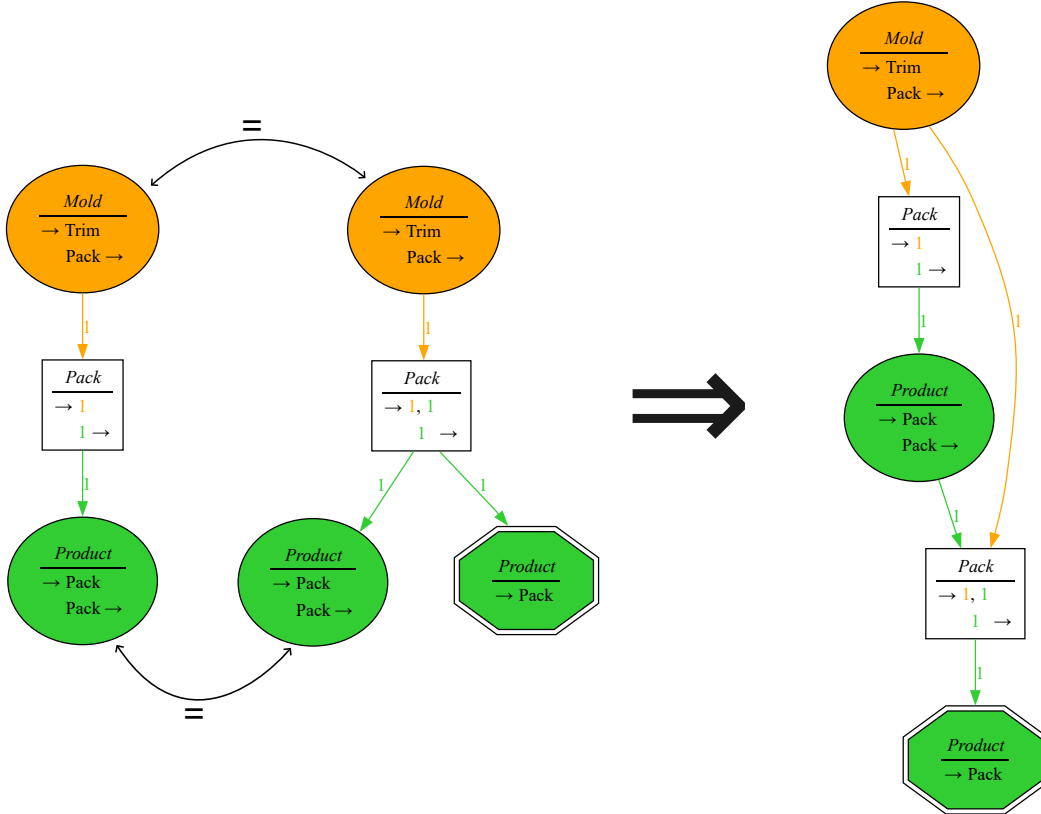


Figure 4.5: The combination of two pairs of equal place candidates in our running example.

4.3.5 Removing Subplaces

In the previous step, we modified the place candidates by combining two place candidates that played the same role in the model into a single one. In this step, we remove place candidates that are already covered by a more general place candidate. A place candidate is more general if the preceding and succeeding activities are supersets of the same activities of the less general place candidates, without multiplicities. This is similar to only keeping the maximal relation Y_W in the α -algorithm [3].

We call the general place candidate the *superplace* and the place candidate to be removed the *subplace*. The subplace and superplace must agree in role, since initial input, final output, and regular place candidates cannot be replaced by a place candidate of the other roles.

Definition 4.3.8 (Subplaces and superplaces). A place candidate p_{sub} is a *subplace* of the *superplace* p_{sup} if $otype(p_{sub}) = otype(p_{sup})$, and $supp(preceding(p_{sub})) \subseteq supp(preceding(p_{sup}))$ and $supp(succeeding(p_{sub})) \subseteq supp(succeeding(p_{sup}))$. Let $PAll$ be the set of all place candidates. Only the maximal superplace is considered in

$$Relation := \{(p_{sub}, p_{sup}) \mid p_{sub} \in PAll \text{ is a subplace of } p_{sup} \in PAll \\ \text{and } p_{sup} \text{ has no superplace}\}$$

For a pair of subplace and maximal superplace $(p_{sub}, p_{sup}) \in Relation$, we replace the subplace p_{sub} by the superplace p_{sup} .

In our running example, three places each have a single superplace, which they are merged into. This process is visualized in Figure 4.6. The entire current formalization is shown.

4.3.6 Adding Missing Configurations

In the final step, missing configurations are added. These are necessary when some activity is in the succeeding activities of a place candidate, but there is no configuration of that activity where the place candidate is an input. A silent configuration is added to accommodate this requirement. This step creates those silent configurations. The place candidates remain unchanged.

Missing configurations are defined as follows:

Definition 4.3.9 (Missing configuration). A place candidate p is *missing a configuration* for activity $act \in \mathbb{U}_{etype}$ if $act \in succeeding(p)$ but there is no configuration $c \in Configs$ with $p \in PIn_c$. The set of place candidates with a missing configuration is

$$Missing := \{(p, act) \mid p \text{ has a missing configuration for activity } act\}$$

4 Method

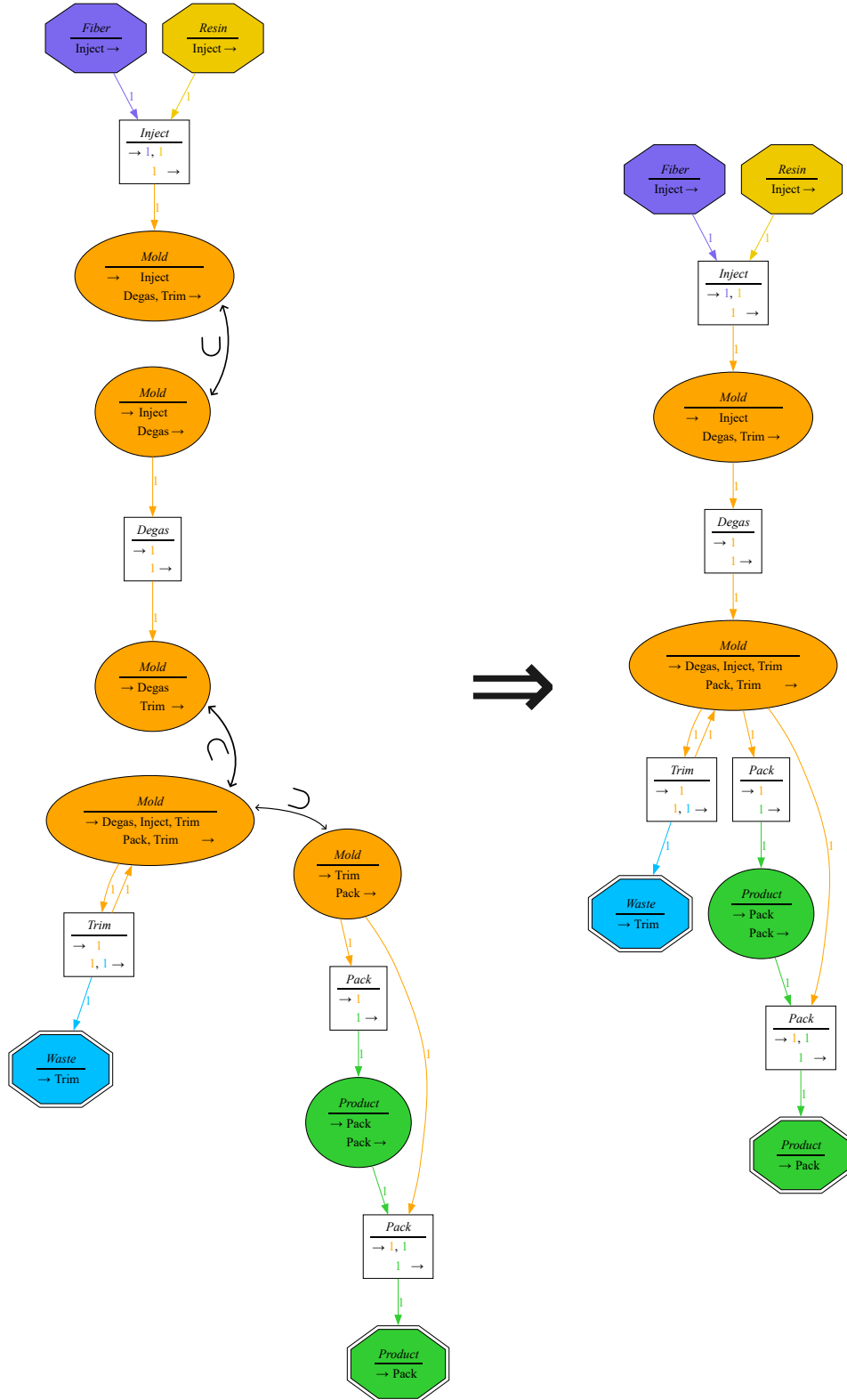


Figure 4.6: The merger of three subplaces into their superplaces in our running example.

For a place candidate that has a missing configuration for activity *act*, there can be multiple configurations of that activity *act*. Thus, to accommodate firing any *act* after a token was placed in *p*, we create a silent configuration for each of the *act*-configurations. Those silent configurations move a single token into the input places of the *act* configuration, of the correct type.

So, for each place candidate with missing configuration $(p, act) \in Missing$, we define the configurations that have an input place of the $otype(p)$ object type as

$$Target_p := \{(c, p_{targ}) \mid c \in Configs \wedge p_{targ} \in PIn_c \text{ with} \\ act(c) = act \wedge otype(p_{targ}) = otype(p)\}$$

For each target $(c, p_{targ}) \in Target_p$, we create the setting $s = (\tau, [ot], [ot], [], [])$ with only the object type *ot* as input and output. The corresponding configuration c_s has $PIn_c := \{p\}$ and $POut_c := \{p_{targ}\}$. There are no initial inputs or final outputs, so $PInit_c := \emptyset$ and $PFinal_c := \emptyset$. As we do not need the preceding and succeeding activities anymore, we use $f : \emptyset \rightarrow \emptyset$ for them in the configuration. Let *Silent* be the set of all such c_s .

In our running example, a single missing configuration is created. This process is shown in Figure 4.7. The new configuration carries the name of the activity that was the cause of its creation in parentheses, for visualization convenience. This name corresponds to the activity of the configuration *c* for $(c, p_{targ}) \in Target_p$.

4.3.7 Creating the Model

Using the place candidates, the configurations *Configs* and the silent configurations *Silent*, we create an object flow net. It is possible to use weighted arcs, where each arc weight corresponds exactly to the multiplicity of an object type, or variable arcs, where all multiplicities greater than 1 create a variable arc. If variable arcs are used, two different configurations that only differ in the arc weights can now become identical and, therefore, only create a single transition. This can prevent overfitting by abstracting away from the precise weights appearing in the log, but reduces the precision since now there are more possible results of firings.

In both cases, assume a configuration $c \in Configs \cup Silent$. For each configuration, a transition t_c is created, and $l(t_c) = activity(c)$ if $c \in Config$. Each place candidate $p \in PIn_c \cup PInit_c \cup POut_c \cup PFinal_c$ becomes a proper place *p*, with $pt(p) = otype(p)$. In addition, $role(p) = initial$ if $p \in PInit_c$, and $role(p) = final$ if $p \in PFinal_c$, otherwise $role(p) = regular$. If $p \in PIn_c \cup PInit_c$, an arc from *p* is drawn to the transition *t* of the configuration. The other way around, if $p \in POut_c \cup PFinal_c$, the arc is from *t* to *p*. For these arcs, we differentiate between the two types of models.

4 Method

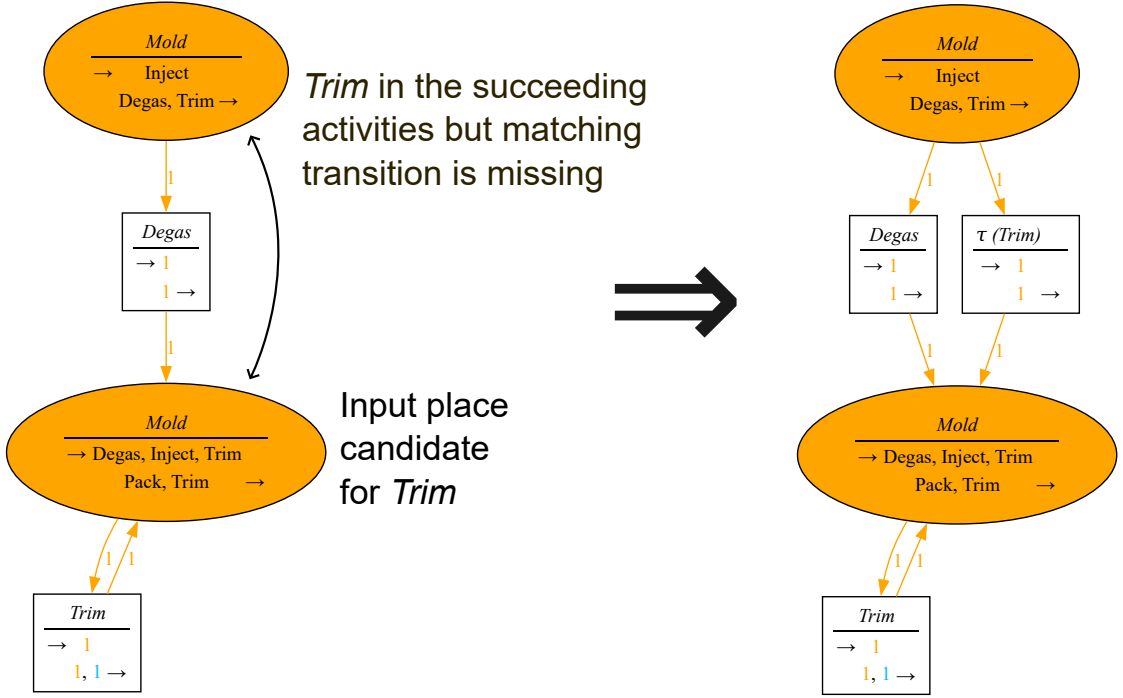


Figure 4.7: The creation of a missing configuration in our running example.

Using weighted arcs. The weight W for an arc connecting p and the transition of a configuration c is given by $multiplicity(c, p)$. The model is an object flow net with arc weights $OFN = (N, pt, role)$, where $N = (P, T, W, l)$ is a labeled Petri net with arc weights.

Using variable arcs. If $multiplicity(c, p)$ for the place to or from a transition of configuration c is greater than 1, the arc is in V , otherwise it is in F . The model is an object flow net with variable arcs $OFN = (N, pt, role)$, where $N = (P, T, F, V, l)$ is a labeled Petri net.

The Petri net model of our running example is given in Figure 4.8. It has not changed compared to the previous step, except for dropping the information on places and transitions.

As an example for variable arcs, we modified our running example so that the first activity *Inject* requires five *Fiber* and three *Resin* objects each. The corresponding arcs are simplified into variable arcs as the multiplicity is now 5 or 3, respectively. These variable arcs are represented by thick arrows. We simplified the *Pack* activity as this part is not relevant for the variable arcs. Figure 4.9 shows the output of the object flow miner on the modified running example with both weighted and variable arcs.

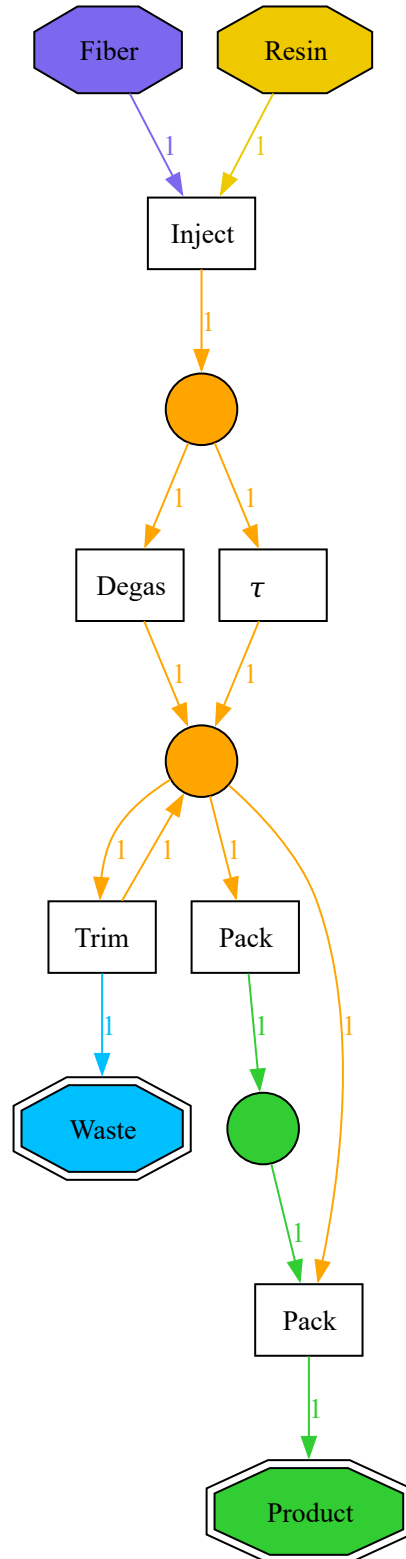


Figure 4.8: The OFN with weighted arcs discovered by the object flow miner for our running example.

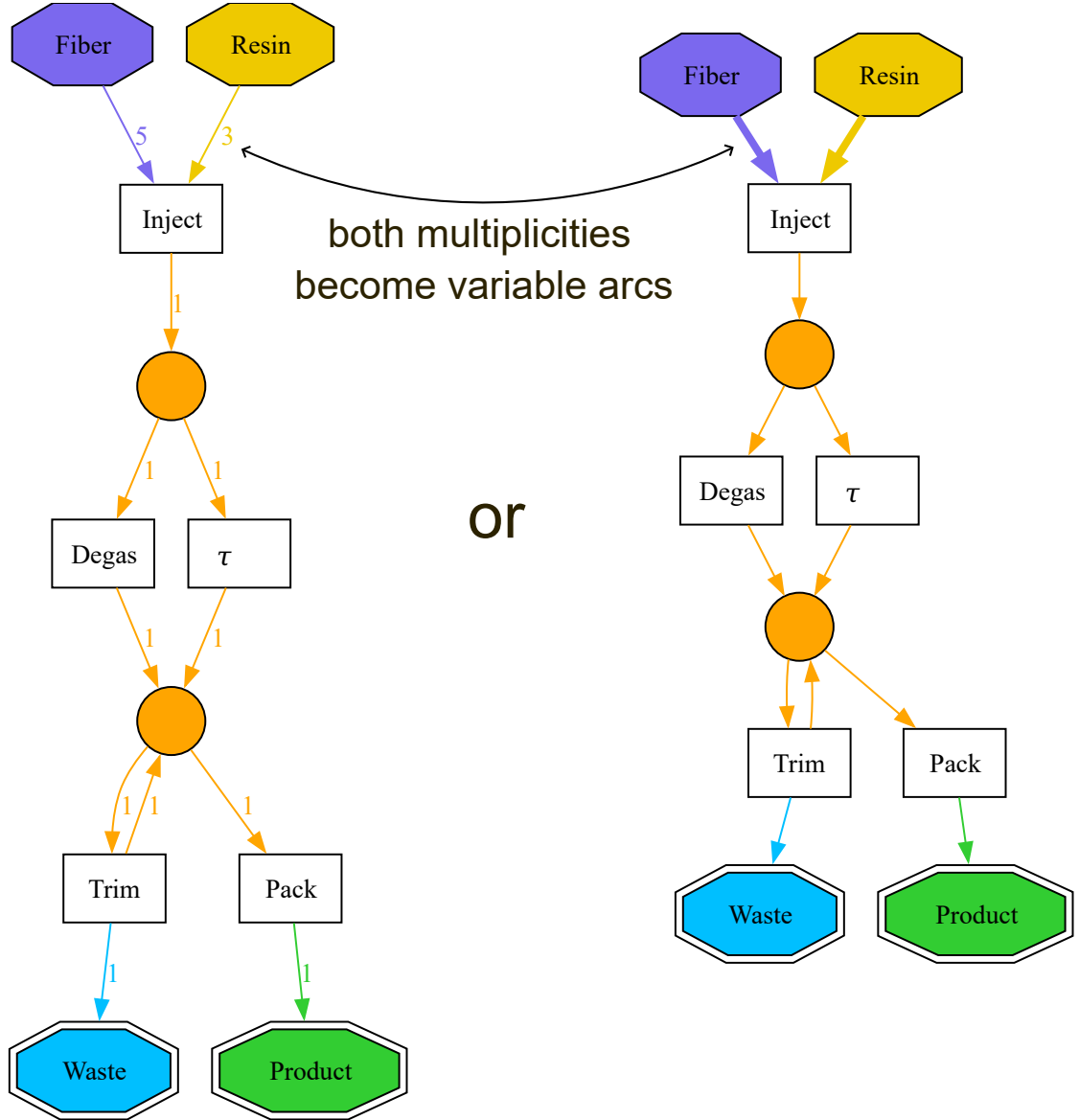


Figure 4.9: The OFNs with weighted and with variable arcs discovered by the object flow miner for a modified running example with multiple initial inputs for *Inject*.

The public OCEL on container logistics [31] demonstrates why variable arcs are useful. In Figure 4.10, we show the discovered model of the process with weighted arcs above the arrow, and the model with variable arcs and a 10% filter on configurations below the arrow. The model size is greatly reduced. How that filter works is discussed in the next section.

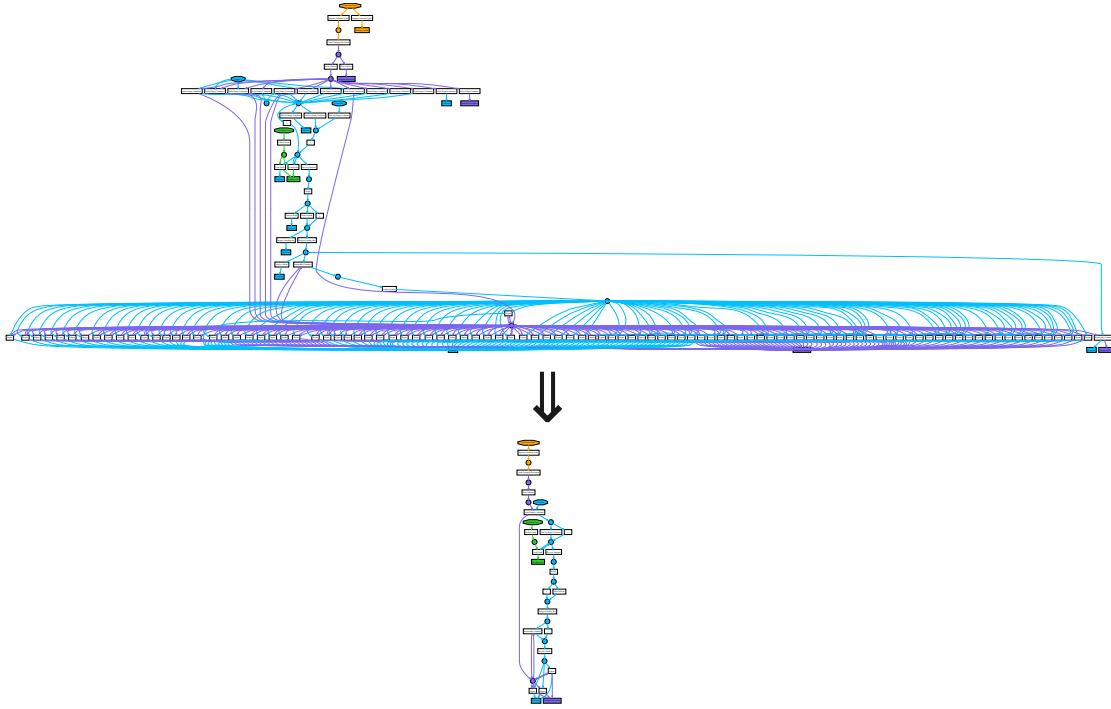


Figure 4.10: Using variable arcs and filtering by 10% on configurations in the container logistics log.

4.3.8 Reducing Model Complexity by Filtering

An event log can contain highly variable process executions, and thereby applying our method to such a log would generate many configurations for a single activity, potentially degrading the simplicity and readability of the model. To combat this issue, we filter in two possible ways.

- For an activity, configurations that occur less often are filtered out. This can filter out behavior that only occurs infrequently in the log and reduces the number of transitions in the final model by dropping less common configurations first.
- For a place, the preceding or succeeding activities that occur less frequently are filtered out. This restricts which transitions follow which places and reduces the number of arcs.

4 Method

Both filtering steps take place between the creation of the configurations and the creation of the place candidates, during the first step of the algorithm.

We define the filtered log for each activity in the following way. A parameter specifies the minimum proportion of the number of events that a configuration must make up. For example, setting it to 0.2 keeps all configurations of an activity that more than 20% of the events of that activity belong to, and filters out the rest.

Definition 4.3.10 (Filtered configurations). Let $\alpha \in [0, 1]$. The settings of an activity $act \in \mathbb{U}_{etype}$ are $Settings_{act} := \{s \mid s \text{ is a setting with activity } act\}$. Then, the *filtered configurations* are

$$Configs^{filt} := \left\{ c_s \mid s \in Settings_{act} \text{ with } |Events_s| > \alpha \sum_{s' \in Settings_{act}} |Events_{s'}| \right\}$$

$Configs$ is replaced by $Configs^{filt}$ before creating the place candidates.

The activities are filtered as follows:

Definition 4.3.11 (Filtered preceding and succeeding activities). Let $\beta \in [0, 1]$. Each configuration $c = (act, preceding, succeeding) \in Configs$ has the *filtered preceding or succeeding activities*

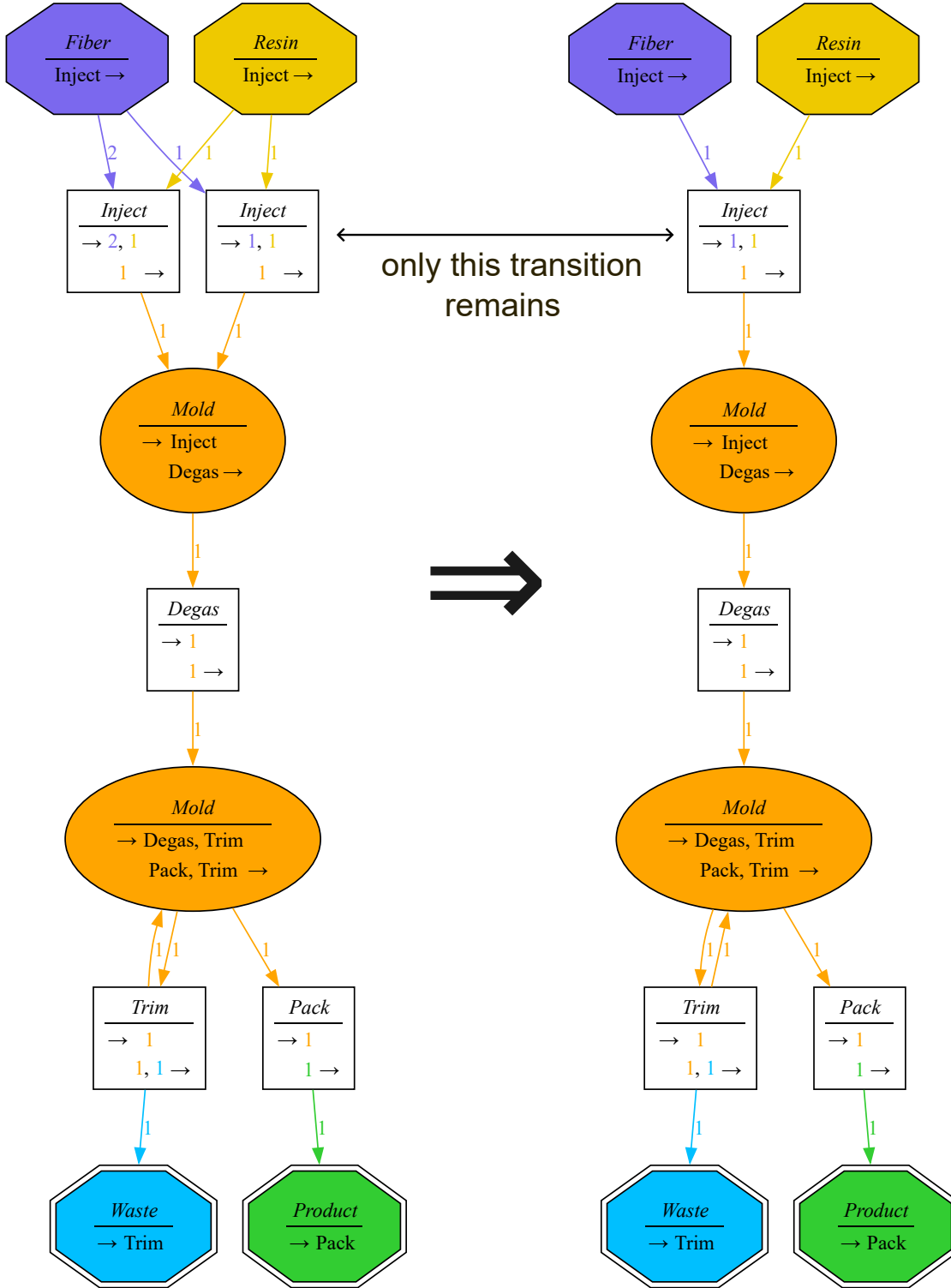
$$\begin{aligned} preceding^{filt}(ot) &:= [act \mid act \in preceding(ot) \text{ with} \\ &\quad m_{preceding(ot)}(act) > \beta |preceding(ot)|] \\ succeeding^{filt}(ot) &:= [act \mid act \in succeeding(ot) \text{ with} \\ &\quad m_{succeeding(ot)}(act) > \beta |succeeding(ot)|] \end{aligned}$$

In this definition, the absolute value of a multiset $|M|$ sums up the individual multiplicities of the elements.

The configuration c is then replaced by $(c, preceding^{filt}, succeeding^{filt})$ to apply the filter.

Note that if $\alpha = 0$ or $\beta = 0$ no filtering takes place, since every element in the multiset occurs at least once. If $\alpha = 1$ no element is kept, which clears the log, since the parameter applies to all configurations. The model consists of isolated subgraphs if $\beta = 1$, since no arcs are drawn except for the direct input and output places of each transition.

We modify our running example so that only a single *Pack* event exists, and there are eleven executions of the process, one of which uses two *Fiber* objects for *Inject*. By setting $\alpha = 0.1$, we filter that deviating configuration, since it represents only 1/11 of the *Inject* configurations. This is shown in Figure 4.11.

Figure 4.11: Filtering with $\alpha = 0.1$ in our modified running example.

In addition, we modify our running example so that ten traces *Degas* the mold, and only one bypasses it to directly go to *Trim*. We set $\beta = 0.1$, so that in the place with succeeding activities [*Degas*¹⁰, *Trim*¹], the *Trim* is filtered out and there is no silent transition in the model. Figure 4.12 shows the change.

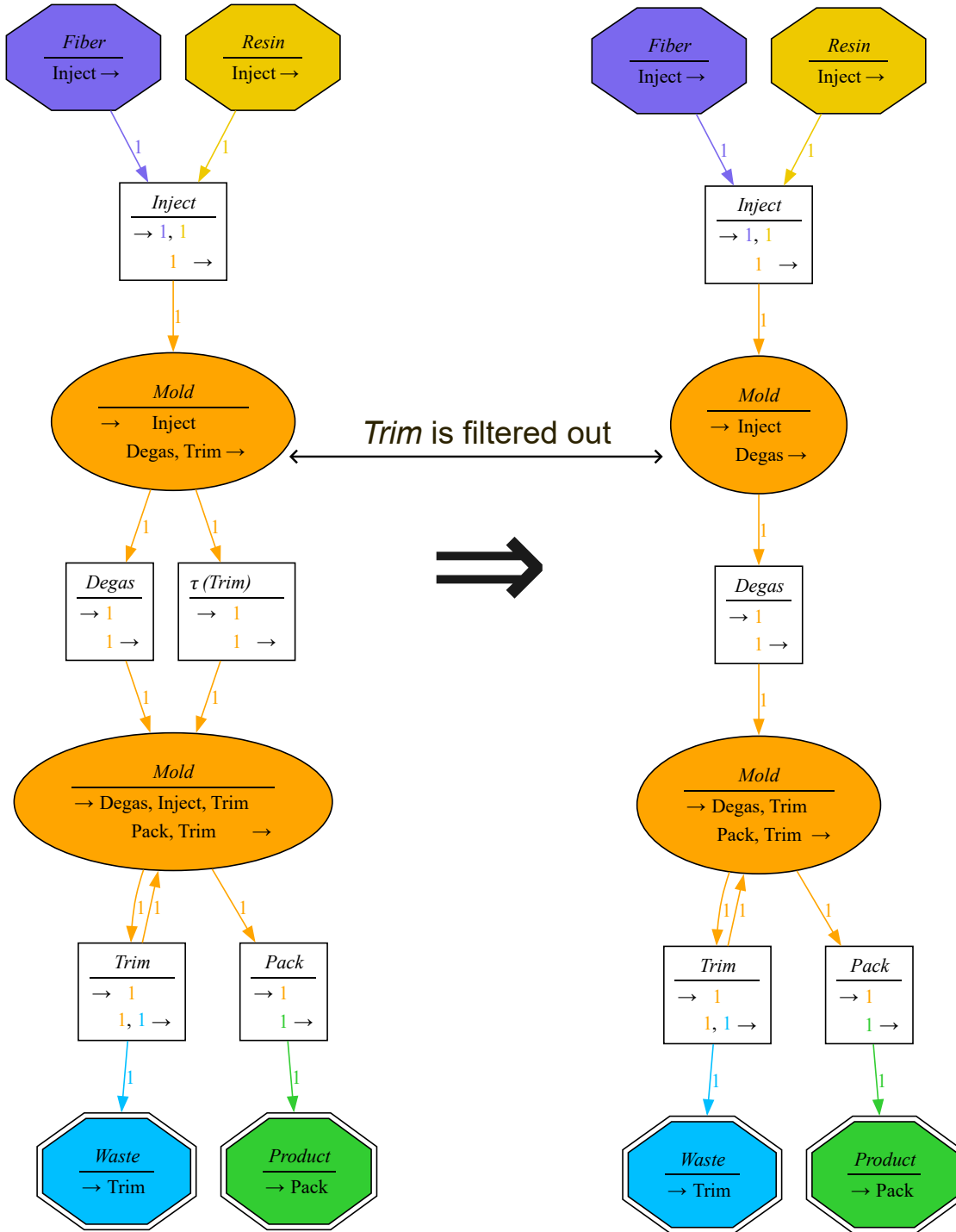
4.4 Annotating the Model with Waste

In this section, we introduce how waste, which is defined as “substances or objects which the holder intends or is required to dispose of” in ISO14040 [24], appears in an OCEL, and how these occurrences can be detected and visualized in the discovered OFN. The waste detection is the final step of our method.

4.4.1 Waste in Object-Centric Event Logs

For calculating waste in an OCEL, there are two approaches, namely by a dedicated attribute or by counting. The mass of an object or the mass emitted in an event can be given by a *mass attribute*, which is used to calculate the waste precisely. A mass attribute is some universal object attribute *mass* that quantifies an object. The exact unit and magnitude of the attribute itself are not essential for waste detection, but rather the proportion between different events and objects quantified by mass, which enables a calculation of their relative impact. If such an attribute does not exist, it can be simulated by counting the number of objects, so that at least the number of objects that end up as waste can be determined, or events where there are fewer produced than consumed objects, which might imply implicit waste creation. Although the approach utilizing mass is much more precise, it requires detailed and correct bookkeeping when creating the log, whereas the approach through counting works in any OCEL. The possibilities for detecting waste in general are the following:

1. All objects that either enter or leave the system boundary, meaning they are initial inputs or final outputs, are counted or summed by mass, and the difference between the two cases is calculated. If less total mass enters than leaves the system, mass balance is violated, and waste has been produced somewhere.
2. Within the system boundary, waste can occur at the process level, without the ability to assign it to specific events. In our running example, this can happen if a machine used for injecting is slowly used up and becomes waste after a long time and after producing a large number of outputs, so this waste cannot be attributed to individual outputs. This shows up in the mass balance.
3. Waste can be explicitly declared in the OCEL.
 - a) A final output object occurring in the log is defined to be waste. In the plastic production, this is the case for objects of the *Waste* type.


 Figure 4.12: Filtering with $\beta = 0.1$ in our modified running example.

4 Method

- b) An event is a specific activity for processing waste as input object, such as an event *CollectWaste* (which does not exist in our running example). This can occur for recycling activities, for example. As this can be simulated by giving the object as final output instead of executing the activity, we do not consider this case further.
 - c) At the event level, waste can be explicitly declared as an event attribute. For example, during *Inject* a small amount of resin is wasted, which is declared on each *Inject* event as an attribute.
4. Or waste can be implicitly present, meaning it occurs in the real process but is not recorded in the log.
- a) An object is simply not used anymore after being produced. For example, a *Mold* fails to pass a quality check and is discarded. The object flow miner already detects these as final outputs, so we do not consider them further.
 - b) The mass or number of the input objects do not add up to the mass or number of the output objects. In the process of our running example, it could happen that a *Trim* event produces multiple *Waste* objects but one is not recorded. Then, there is a difference in the input and output mass of the *Trim* event.

We point out that counting is equivalent to assuming a unitless mass of 1, which allows us to assume that a mass attribute always exists, so we do not consider the case of counting further. This leaves four types of waste that we consider separately.

Definition 4.4.1 (Types of waste). In an OCEL, waste occurs in the following ways.

1. **System-wide waste:** This is calculated by summing the masses of all input and output objects of the process.
2. **Object type waste:** This is calculated by summing the mass of all objects of a type.
3. **Explicit event waste:** This is calculated using an event attribute.
4. **Implicit event waste:** This is calculated by summing the masses of all input objects and all output objects of an event, and taking the difference between the two sums.

4.4.2 Annotating the Model with Waste

The result of the object flow miner, an OFN model showing object flow, is suitable for displaying waste since waste is a specific type of object flow. For each of the types of

4.4 Annotating the Model with Waste

waste that we defined, we give the method to calculate it, assuming a mass attribute $mass \in OA$ exists.

Starting with system-wide waste, the approach is straightforwardly calculating the difference between the mass of all initial input and final output objects.

$$syswaste := \sum_{\substack{e \in E, o \in O \\ role_{in}(e,o)=init}} oaval_{mass}(o) - \sum_{\substack{e \in E, o \in O \\ role_{out}(e,o)=final}} oaval_{mass}(o)$$

If $syswaste$ is positive, more mass entered the system than left it; this indicates either waste generation or erroneous logging.

The object type waste can be integrated directly into the Petri net, as text on each object type. This has the advantage of showing precisely where the waste was emitted, since an object type can occur multiple times as a final output place, but the waste can be attributed to each of them separately. Every final output place is associated with some configuration $c_s \in Configs$ for some setting s . The object waste of the object type of that place is given by

$$owaste(c_s, ot) := \sum_{e \in Events_s} \sum_{\substack{o \in In_e \\ objtype(o)=ot}} oaval_{mass}(o)$$

For events, there are two different types. Explicit event waste is waste that is declared in the log itself; in our case, it appears as an event attribute. We call this attribute $mass \in EA$ as well, although it is now defined for events instead of objects. If it does not exist, we assume it exists and is equal to 0. The waste given by this attribute can be shown visually in the transition belonging to the configuration $c_s \in Configs$ with the setting s . For that configuration, it is given by

$$exwaste(c_s) = \sum_{e \in Events_s} eaval_{mass}(e)$$

Finally, there is implicit waste on events. This is waste that was not intentionally put into the OCEL at creation time but can be derived from it. This is done by summing the mass of the input objects and subtracting the sum of the mass of the output objects. It is formalized for a configuration $c_s \in Configs$ with a setting s , considering all events of that setting.

$$imwaste(c_s) := \sum_{e \in Events_s} \left(\sum_{o \in In_e} oaval_{mass}(o) + \sum_{o \in Init_e} oaval_{mass}(o) - \sum_{o \in Out_e} oaval_{mass}(o) - \sum_{o \in Final_e} oaval_{mass}(o) \right)$$

4 Method

If the value is positive, there has been unrecorded waste.

By incorporating these values into our model, we get the extended model that includes waste information:

Definition 4.4.2 (Waste-Annotated Object Flow Net). Let OFN be the OFN model from the object flow miner, with a mass attribute $mass$ for objects and explicit event waste. A *waste-annotated Object Flow Net* is $WOFN = (OFN, syswaste, owaste, exwaste, imwaste)$, where $syswaste$, $owaste$, $exwaste$ and $imwaste$ are functions as defined above that return information about the waste at the process, object type, or activity level.

4.4.3 Example

Our running example defined in Section 2.4.2 contains a mass attribute with masses in Table 1.2. Using this, we show the partially annotated model in Figure 4.13. We only show a selection of the detected waste. For instance, the explicit waste is simply 0 for all events, which we omit. On *Inject* the implicit event waste is 1 in *inject_1_1* and 2 in *inject_2_1*, resulting in the average 1.5 and a total of 3 waste units detected. The total system-wide waste, which is not visualized but provided as output of our implementation is 3, from a total input mass of 50 and a total output mass of 47, consistent with the implicit waste of 3. On *Waste*, the mass of waste produced is shown. For completeness, the mass of the initial inputs is also annotated. The *Product* is only counted, which shows a single product as an output. This shows that mass and counting annotations can be combined. In addition, despite the fact that *Product* is not waste, the method for waste detection can still be applied. For the mass of the initial inputs, we only switched the *Final* to *Initial* in the *owaste* definition.

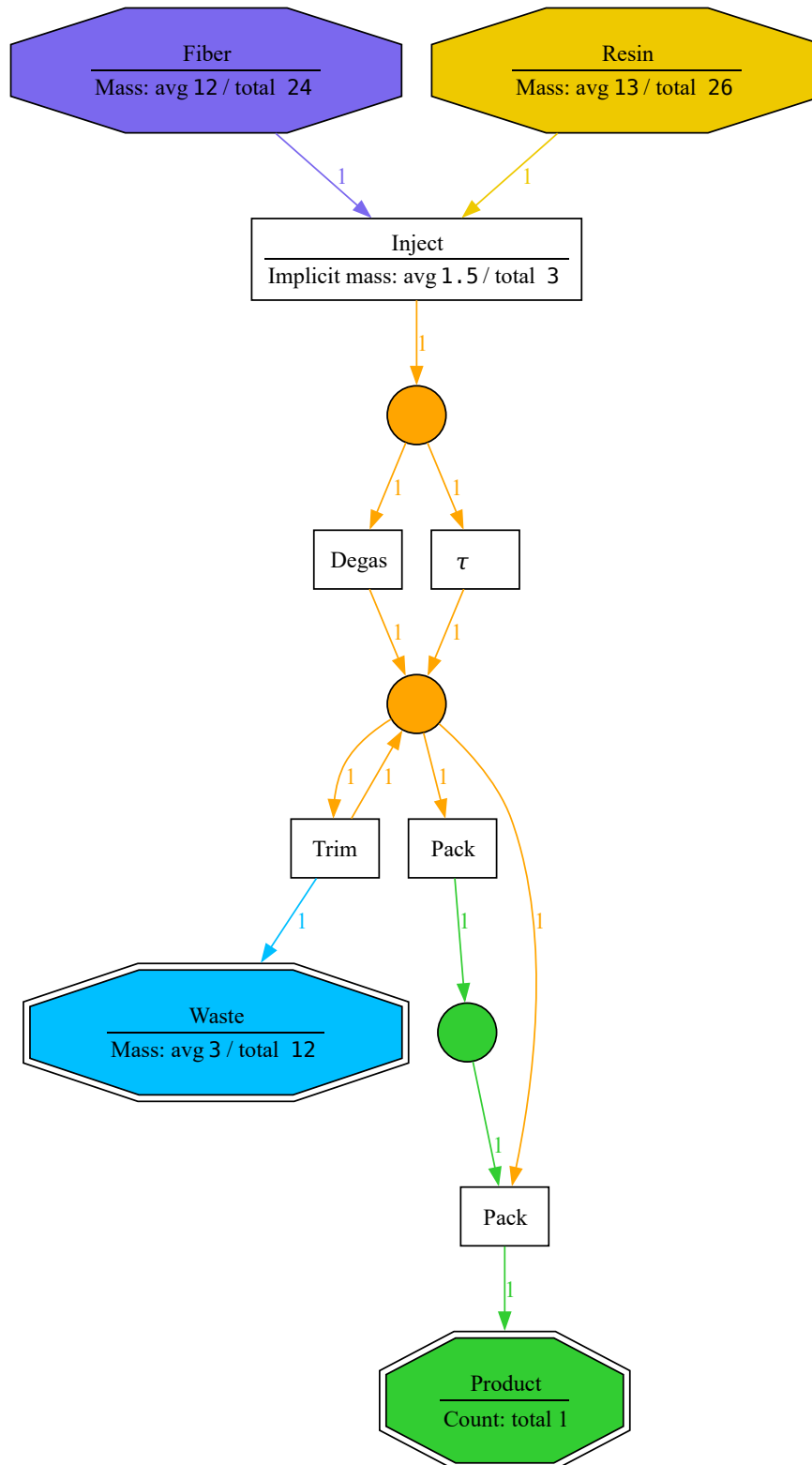


Figure 4.13: The model of our running example annotated with waste information.

5 Implementation

Our implementation consists of two parts. First, we expand on the method by detailing how the role functions required by our method can be discovered heuristically. Second, we show the implementation of our entire approach in Oceleoscope [38], a tool for using OCPM techniques.

5.1 Discovering the Role Functions Automatically

In our method, we assumed that for an input OCEL the functions $role_{in} : E \times O \rightarrow \{in, init, \perp\}$ and $role_{out} : E \times O \rightarrow \{out, final, \perp\}$ are given. We provide a method to discover them automatically. This section presents the implementation of this functionality.

This is achieved by exploiting the total order of events based on timestamps. In the simplest case, we assume that an object is an output of an event if it is related to a later event, and an input if it is related to a previous event. If there are two events $e, e' \in E$ with $time(e) < time(e')$ and an object $o \in O$ related to both, meaning $(e, o) \in E2O$ and $(e', o) \in E2O$, we set an output on the previous event, $role_{out}(e, o) = out$, and an input on the later event, $role_{in}(e', o) = in$. The total order ensures that this is valid. These definitions also immediately satisfy the consistency condition of the role functions.

To discover the initial inputs and final outputs in an event, we use a case distinction based on the nature of the event. Events are categorized as logistical or transformational.

Logistical events. An event is a logistical event if it semantically reflects movement, storage, or change of state of associated objects, all of which persist until after the event.

Transformational events. An event is a transformational event if it semantically represents a consumption of the inputs to create a new output, such that the inputs might not exist in their original form after the event.

We classify each event as one of seven cases, which are shown in Table 5.1. The columns correspond to the number of objects related to the event. The first three cases assume the event to be logistical. The next three cases assume an underlying transformational

5.1 Discovering the Role Functions Automatically

event. The final case is a fallback and is treated as transformational as well. The order is important and the cases are considered in ascending order, from top to bottom in the table.

Event Classification	Only Before	Before and After	Only After	Only in Event
Fully Logistical	0	all	0	0
Logistical with Before	> 0	≥ 0	0	0
Logistical with After	0	≥ 0	> 0	0
Transformation in Event	0	0	0	all
Transformation of Before	> 0	0	0	> 0
Transformation of After	0	0	> 0	> 0
Fallback	any	any	any	any

Table 5.1: The different cases in our classification.

Once each event is classified, the initial inputs and final outputs are set based on the case of each event. The regular inputs and outputs, according to the above definition, are assumed to already exist and are not changed. In the fallback case, it is not possible to assign initial inputs or final outputs, so the roles remain unknown. The assignment is shown in Table 5.2, and in Table 5.3 the different cases are illustrated with the structure that triggers the case distinction and what the assumed underlying structure is.

Event Classification	Initial inputs	Final Outputs	Unknown
Fully Logistical	none	none	none
Logistical with Before	none	objects that are only before	none
Logistical with After	objects that are only after	none	none
Transformation in Event	all objects	all objects	none
Transformation of Before	none	objects only in the event	none
Transformation of After	objects only in the event	none	none
Fallback	none	none	all objects

Table 5.2: The initial inputs and final outputs in the different cases.

Objects that still have an unknown role from the fallback case are rendered with a double-arrow dashed arc in the Petri net, or the user can provide their role as a hint to the algorithm.

Alternatively, the remaining objects with an unknown role can be resolved using MFA if they have a mass attribute. Accordingly, we assume that the mass-balance principle holds. Then, the objects with an unknown role are assigned initial input or final output roles in each possible combination, and the assignment with the smallest difference

Event Classification	Detected Structure	Assumed Structure
Fully Logistical		Identical to the detected structure
Logistical with Before		
Logistical with After		
Transformation in Event		
Transformation of Before		
Transformation of After		
Fallback		Identical to the detected structure

Table 5.3: Illustrations of the structures that trigger each case, and what the assumed underlying structure is.

between the total input and output mass is chosen. This corresponds to the closest match of the total input and output mass. This procedure is equivalent to minimizing the process-wide waste *procwaste* from Section 4.4.2 over the possible assignments.

5.2 Oclescope Plugin

In this section, we present how a user can use the method and implementation in Oclescope [38]. It is implemented in the *Object Flow Miner* plugin. This plugin offers four methods that run the object flow miner, which differ in using weighted or variable arcs and having waste annotations or not. This menu is shown in Figure 5.1. Each one gives an *object flow net*.

As an example, choosing “Weighted arcs” leads to the configuration screen in Figure 5.2. It offers a range of parameters:

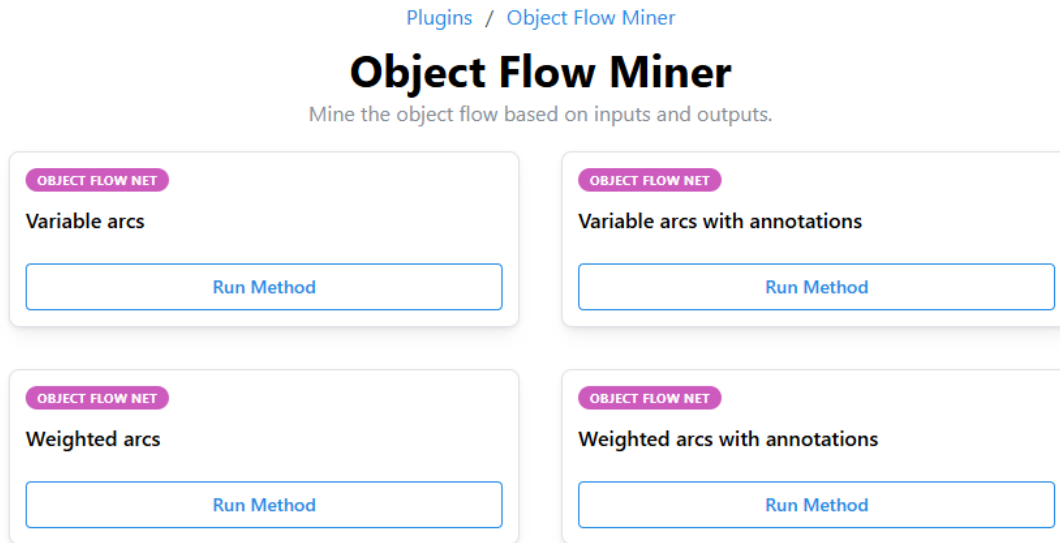


Figure 5.1: The available methods in our Ocelescope plugin.

- ① The OCEL2.0 event log.
- ② A list of object types that are filtered out prior to running the object flow miner.
- ③ The role function maps these object types to always be an initial input. This allows the user to manually set the role functions. For object types that are not given here or in the next field, the heuristic algorithm is applied to determine the role function.
- ④ The role function maps these object types to always be a final output.
- ⑤ If an attribute *mass* exists, selecting this checkbox causes an MFA to be performed as described in Section 5.1 to approximate mass balance as closely as possible.
- ⑥ The threshold α , which filters configurations of the same activity, as described in Section 4.3.8.
- ⑦ The threshold β , which filters the succeeding activities in a place and therefore reduces the number of arcs, also described in Section 4.3.8.
- ⑧ Selecting this checkbox also displays the inputs and outputs of each place and transition as additional information in each node, which is done throughout the examples in our method.

The Ocelescope application automatically populates each field with appropriate suggestions, such as exactly the object types that occur in the log. Clicking on **Submit** executes the algorithm, yielding the same OFN model that was also the result of our method in

5 Implementation

Section 4.3.7, see Figure 4.8. Oclescope shows a preview of the model, see Figure 5.3, which can be made full-screen and downloaded.

[Plugins](#) / [Object Flow Miner](#) / [Weighted arcs](#)

Weighted arcs

Event Log * ①

running_example

Filter out object types * ②

Always consider as initial input * ③

Always consider as final output * ④

☒ Use MFA to determine initial inputs and final outputs ⑤

Configurations threshold α ⑥

0

Arcs threshold β ⑦

0

☐ Show place and transition information ⑧

Submit

Figure 5.2: The available parameters for the algorithm with weighted arcs and without annotations in Oclescope.

We also explain the parameters for the “Weighted arcs with annotations” method, shown in Figure 5.4. In addition to the previously described parameters, now the interface includes the options:

- ⑨ These object types are counted in each place where they are initial input or final output.
- ⑩ The mass of these object types is calculated for each place where they are initial input or final output.
- ⑪ The attribute *mass* is averaged and summed for each configuration of the activities listed here.
- ⑫ The difference between the number of input and output objects is calculated for each configuration of the activities listed here.

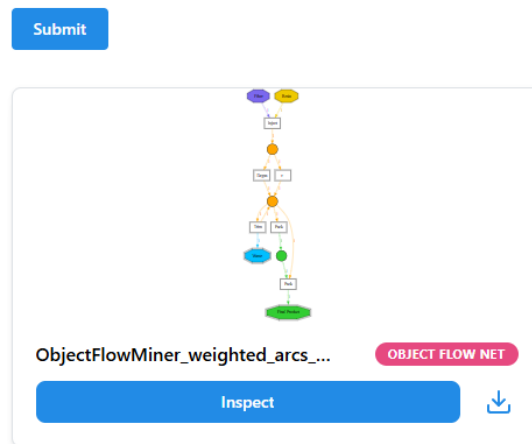


Figure 5.3: The output of Oceanscope for the parameters given in Figure 5.2.

- 13 The difference between the summed masses of input and output objects is calculated for each configuration of the activities listed here.

With the given configuration, this generates the model with waste annotations shown as an example in our method in Figure 4.13. We do not show the menus for the variable arc methods as they are completely identical.

5 Implementation

[Plugins](#) / [Object Flow Miner](#) / [Weighted arcs with annotations](#)

Weighted arcs with annotations

Event Log *
running_example

Filter out object types *

Always consider as initial input *

Always consider as final output *

☒ Use MFA to determine initial inputs and final outputs

Configurations threshold α
0

Arcs threshold β
0

☒ Show place and transition information

Annotate object types with count * ⑨
Final Product X

Annotate object types with mass * ⑩
Waste X

Annotate activities with explicit mass * ⑪
Inject X

Annotate activities with implicit count * ⑫

Annotate activities with implicit mass * ⑬
Trim X Pack X

Submit

Figure 5.4: The available parameters for the algorithm with weighted arcs and with annotations in Ocelescope.

6 Evaluation

This chapter provides an evaluation of the object flow miner with the aim of assessing both its theoretical foundation and practical applicability. We divide this assessment into five sections that evaluate different aspects:

1. The results of the object flow miner when discovering basic constructs found in process logs, such as exclusive choice.
2. The quality of the object flow model discovered on publicly available OCELS.
3. The usability of the model for waste detection.
4. The runtime performance of the object flow miner.
5. The applicability of the entire approach to LCA.

6.1 Detecting Basic Constructs

In this section, we evaluate whether the proposed object flow miner detects some fundamental routing constructs encountered in practice [3]. The handling of these constructs, namely exclusive choice, concurrency, loops, and long-term dependencies, establishes a baseline for what to expect in subsequent evaluations. We use mock object-centric logs with only a single object type and, therefore, we use simple event log notation defined in [1]. Therefore, this section simultaneously indicates the feasibility of the object flow miner as a classical, non-object-centric mining algorithm. We run the algorithm without filtering, and with the heuristic algorithm for initial inputs and final outputs.

6.1.1 Exclusive choice

In an exclusive choice construct, there are at least two transitions leaving a place, and choosing one excludes the other. The input log is $[\langle A, B, D \rangle, \langle A, C, D \rangle]$. Figure 6.1 shows the discovered OFN.

However, we note that, while there is a split at an exclusive choice, there is not necessarily a join of the paths later. This is because the split is created locally, from two transitions that have similar input place candidates, and no measure is taken to combine the paths again. As an example, the log $[\langle A, B \rangle, \langle A, C \rangle]$ results in Figure 6.1.

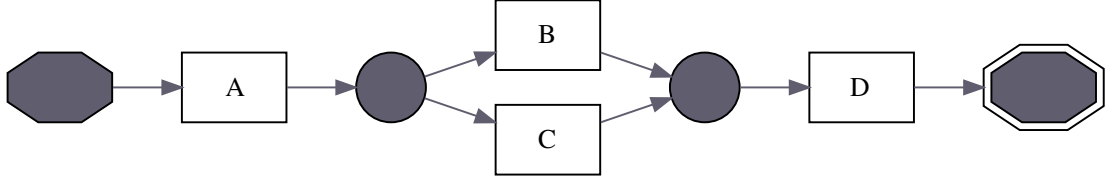
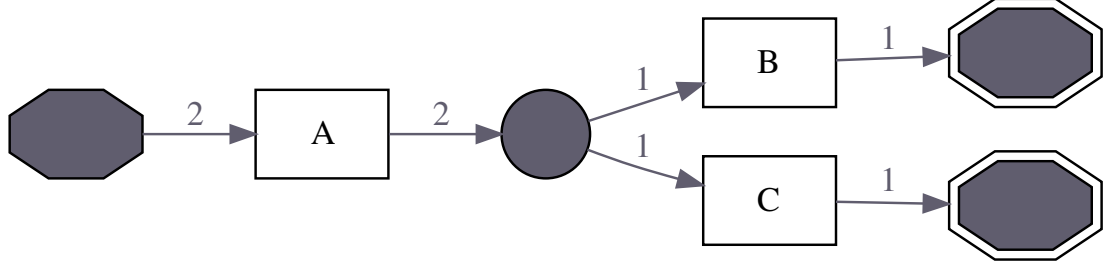
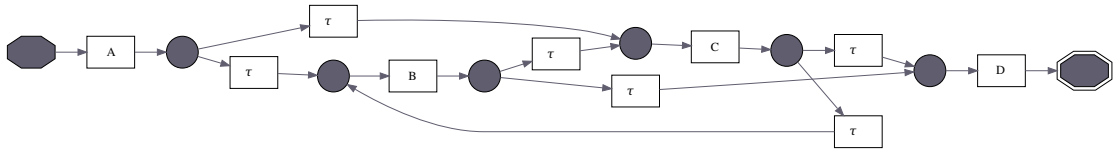
Figure 6.1: The discovered model, containing an exclusive choice between B and C .

Figure 6.2: A model with a split but no join.

6.1.2 Concurrency

In a concurrent construct two activities can be executed in any order, but both must have been executed exactly once before continuing with the next activity. For $[\langle A, B, C, D \rangle, \langle A, C, B, D \rangle]$, the result is shown in Figure 6.3. This underfitting model still supports every concurrent execution by using a silent transition after an activity fires. However, the model is less precise, as it allows for skipping activities B or C and also enables infinite loops. The reason is that there is no built-in mechanism for detecting concurrency; instead, silent transitions are generated because activity B has succeeding activities $\{C, D\}$, so due to the addition of missing transitions in Section 4.3.6, a path from B to D is created. In addition, because the preceding activities of B are $\{A, C\}$, there is a silent transition incoming from C .

Figure 6.3: The discovered model, containing concurrent activities B and C .

6.1.3 Loops

There are two types of short loops [3], which have length 1 or 2. A length-1 loop, or self-loop, occurs when an activity immediately follows itself. These are detected correctly

by the algorithm because of the specific step ensuring their handling.

In a length-2 loop, two activities alternate, as in $[\langle A, B, C, B, D \rangle]$, where B both begins and exits the loop. We again expect the algorithm to model these correctly, and it does, as shown in Figure 6.4, since the input and output place candidates of activity C are absorbed into the correct superplaces.

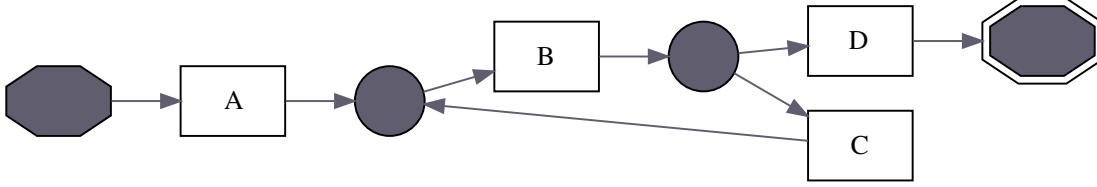


Figure 6.4: The discovered model of a length-2 loop involving B and C .

Let us also consider a loop of length 3. In the example log $[\langle A, B, C, D, B, C, D, B, E \rangle]$, $B \rightarrow C \rightarrow D$ is executed twice (but more often does not change the argument), and then executes B once again before the loop exits to E . The object flow miner finds this construct, see Figure 6.5, a result that generalizes to more sophisticated loops that themselves contain constructs.

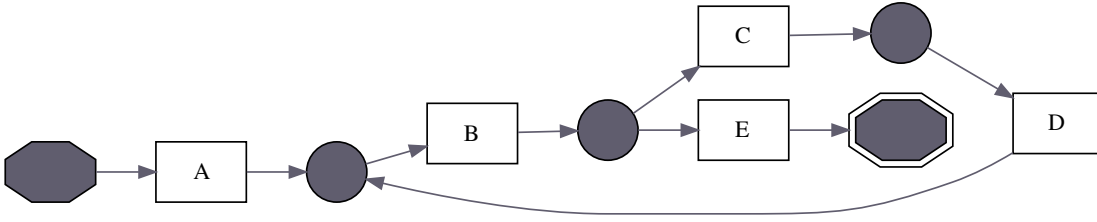


Figure 6.5: The discovered model for a loop involving B , C and D , with a final B .

Let us also consider that the loop is exited in its final activity D instead of running B once more, so that the log becomes $[\langle A, B, C, D, B, C, D, E \rangle]$. The object-centric inductive miner implemented in `pm4py` [8], the result of which is shown in Figure 6.6, does not give a precise result for such a log; instead, a central place is created that enables going to C immediately after A , or even looping C . In contrast to that, the object flow miner in Figure 6.7 correctly mines the intended Petri net.

6.1.4 Non-free Choice Constructs

One type of a non-free choice construct is a long-term dependency [1], which occurs when a transition depends on another transition that does not directly precede it. Consider the log $[\langle A, C, D \rangle^{45}, \langle B, C, E \rangle^{42}]$ [1]. We assume that the high frequencies mean that if A is executed, D must be executed after C , and also that if instead B is executed first, then E follows C . These dependencies can be realized through implicit places, which

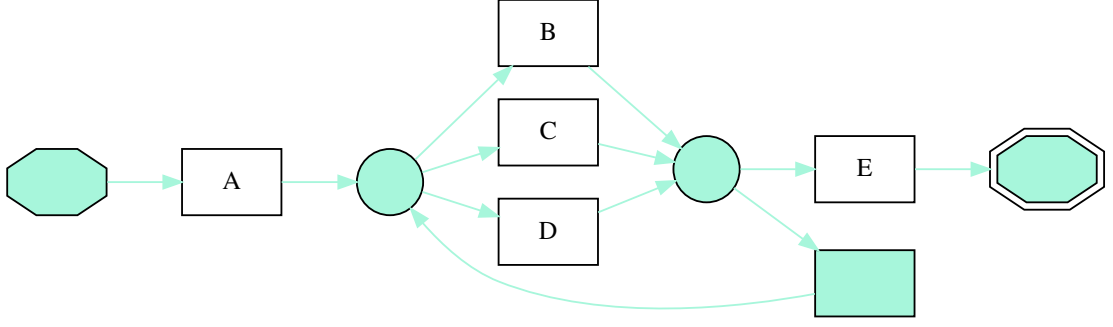


Figure 6.6: The discovered model using the object-centric inductive miner for a loop involving B , C and D , without a final B . We modified the visualization of the Petri net to use the same style as OFNs to make comparison easier.

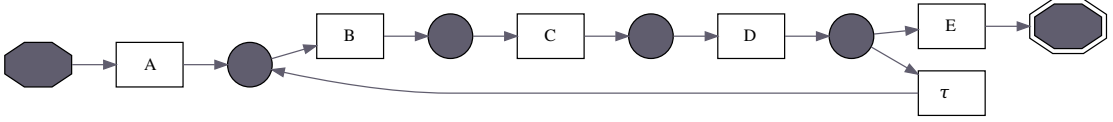


Figure 6.7: The discovered model using the object flow miner for a loop involving B , C and D , without a final B .

are input places to the later transition. However, since the purpose of the object flow miner is to ensure that every target place of a transition corresponds to a specific output object type, the algorithm can never produce an implicit place that would be necessary to ensure this behavior. Furthermore, this raises the question of what object type an implicit place should have in general. Nonetheless, the result of the object flow miner in Figure 6.8 can still replay the log, albeit with lower precision.

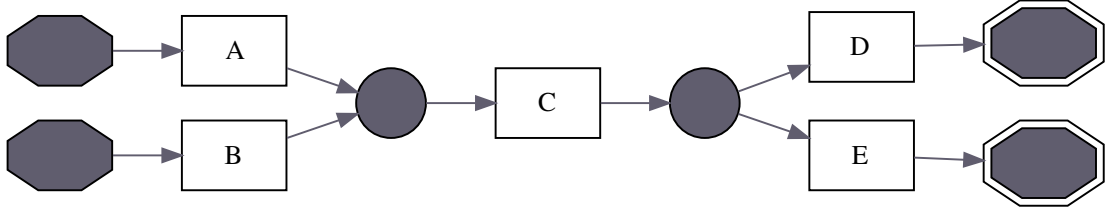


Figure 6.8: The discovered model which does not respect the long-term dependency.

We recognize that for the general object flow, information carried through an implicit place would be relevant, including in the context of waste detection. It would enable a more direct tracing of waste objects by visualizing that the waste in an activity having a long-term dependency is only created if previously another transition was taken. Therefore, we suggest an extension of the algorithm as future work, which detects long-term dependencies and gives them a special object type.

Another type of non-free choice construct is the mix of concurrency and exclusive choice, as in $[\langle A, B, C, D \rangle, \langle A, C, B, D \rangle, \langle A, E, D \rangle]$ [1]. The object flow miner discovers an object flow net that contains the exclusive choice, and represents the concurrency between B and C by the underfitting concurrent construct already introduced in Section 6.1.2. The model is shown in Figure 6.9.

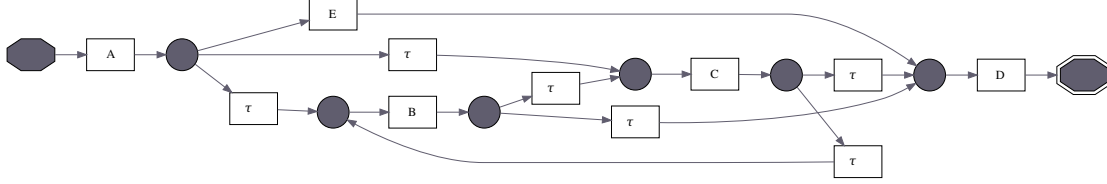


Figure 6.9: The discovered model for the non-free choice construct.

6.1.5 Conclusion

In summary, the object flow miner detects many constructs found in practice. However, as just discussed, it lacks ideal modeling of concurrency and long-term dependencies. This is a potential improvement for future work.

Since the object flow miner requires only timestamps and no qualifiers, it can be applied to classic event logs if only a single object type exists, which is interpreted as the case identifier. We therefore suggest research into employing the object flow miner specifically on classic event logs. In contrast to the α -miner, it incorporates silent transitions and multiple labels, although an extension to the α -miner, the α^{++} algorithm [45], can discover certain long-term dependencies, which the object flow miner never manages.

6.2 Discovered Models on Public Logs

Having established the handling of the theoretical constructs, in this section we evaluate models discovered by the object flow miner on publicly available OCEs. We analyze the output with different levels of filtering and included resources, and we are especially interested in the replay fitness and simplicity of the models. This is the quantitative perspective of our evaluation. Our model should inherently limit generalizability, since although loops should generalize, our goal is to visualize the existing log and the existing material flow accurately, not offering a general model at the cost of imprecise descriptions of reality. We are also interested in where logs result in unexpected, confusing, or hard-to-interpret models, and what the cause of that is. This is the qualitative perspective.

We use four public OCEs. These are the container logistics, order management, procure-to-pay, and hinge production log. We evaluate the hinge production log separately in the evaluation, with a special focus on waste detection.

6.2.1 Quantitative Results

Setup

For the four logs, our method for heuristically finding initial inputs and final outputs given in the implementation in Section 5.1 is used with the object flow miner. Furthermore, we use variable arcs for all models. This makes them smaller and match the fitness calculation better. In addition, we vary the following parameters:

1. **Resources:** We run the miner with or without resources filtered out.
2. **Filter:** The α and β parameters from our section on filtering.
3. **Max. Comp.:** The maximum size at which a connected component of objects is split. This is necessary to reduce the computational complexity of the calculation. For details, see the description of how fitness is calculated.

For the logs, we give two values regarding its size:

4. **#OT:** The number of object types.
5. **#E2O:** The size of the *E2O* relation.

Using these inputs, we run the object flow miner, which produces an OFN model. The workflow is illustrated in Figure 6.10. The OCEL is the input in blue. In purple, the numerical indicators that we quantify are shown. These are the following:

6. **Fitness:** The replay fitness, which is computed using non-object-centric alignments. To get a classic event log, the input OCEL is flattened to the connected components of each object type. The discovered OFN is filtered to contain only places of that object type and no places with the unknown role. An initial marking is created for every object that is an initial input, and a final marking for the final output objects. The weight of each arc in the filtered OFN is set to 1. This allows replaying variable arcs by moving each token individually, but also allows for more behavior. The transformed OFN is interpreted as a classic Petri net. The fitness of the connected component as a single case is then calculated using alignments of the flattened log on the Petri net. However, as some logs have large connected components that result in the computation taking too long or requiring too much memory, the *Max Comp.* parameter splits components at the cutoff size. This reduces the fitness, and is the reason why the unfiltered container logistics logs have a fitness very close to 1, but slightly lower. In rare cases, it was not possible to calculate an alignment using the `pm4py` library because the filtered Petri net was not an easy-sound net. In that case, we set the fitness of the trace to 0.

The *Fitness* is calculated by an equally weighted average of the average fitness of each flattened object type. This is done in order to prevent object types that occur more often from having a greater influence. The computation of the fitness for the

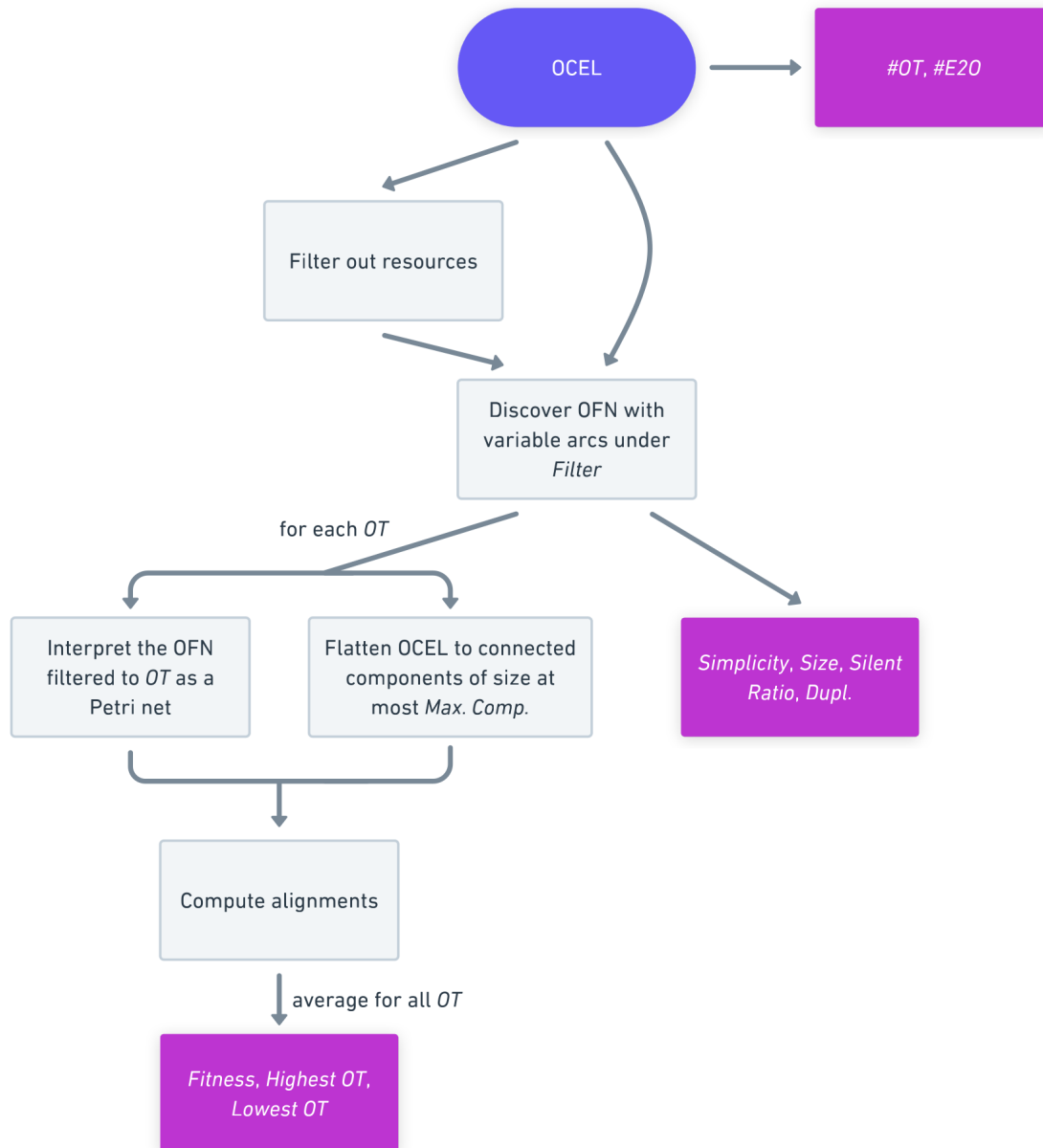


Figure 6.10: The workflow of our evaluation. Created with Whimsical (<https://whimsical.com>).

6 Evaluation

order management log with resources was too resource-intensive for our machine, which is why these rows have missing values marked with a hyphen.

7. **Highest OT**: The average fitness of the flattened object type with the highest average fitness.
8. **Lowest OT**: The average fitness of the flattened object type with the lowest average fitness.
9. **Simplicity**: An indicator between 0 and 1 based on the inverse arc degree [10].
10. **Size**: The total number of transitions and arcs.
11. **Silent Ratio**: The ratio of silent to all transitions.
12. **Dupl.**: As the object flow miner discovers transitions with the same label, this is the highest number of transitions labeled the same.

Results

In Table 6.1, we present the results. Fitness ranges from 1 to 0, simplicity from 0.7551 to 0.1470, and size from 902 to 80. This underlines that our evaluation contains sufficient variation. In the following, we analyze each log in detail, with the same parameters that were used in the table.

6.2.2 Container Logistics

The container logistics log [31] contains a flow of goods, specifically how these are ordered, loaded onto containers, and shipped. On the OCEL with resources and no filtering, the object flow miner achieves a fitness of nearly 100%. Filtering out configurations that make up less than 10% of their activity results in decreasing the size from 278 to 136, but the fitness decreases to 43.93%. This model, depicted in Figure 6.11, shows the material flow read from top to bottom, beginning by creating an order, then booking vehicles, ordering a container, filling it with goods while loading it onto a truck, which then drives to the terminal, where the container is weighed. After that, it is either placed in stock or brought directly to the loading bay, visualized through the silent τ (*Bring to Loading Bay*) transition, and afterward optionally rescheduled or directly loaded, and then it finally departs. This demonstrates that the object flow visualized in the model is clear, indicating that even a model with a low fitness can serve as a process model.

Here, the object types *Forklift* (pink), *Vehicle* (orange) and *Truck* (red) have a lot of self loops, so we identify these as resources. By filtering them out, we get Figure 6.12, which shows the intangible material flow. This has a size of only 90, and a fitness of 69.91%. It demonstrates that filtering out resources is beneficial, and the object flow miner can discover models of a small size with a good fitness.

Parameters			Log Size		Results						
Log Name	Filter	Max Comp.	#OT	#E2O	Fitness	Highest OT	Lowest OT	Simplicity	Size	Silent ratio	Dupl.
Container Logistics, with resources	-	20	7	74272	0.9934	1	0.9535	0.3286	278	0.10	7
Container Logistics, with resources	$\alpha = 0.1$	20	7	74272	0.4393	0.99	0	0.3750	136	0.21	3
Container Logistics, no resources	-	20	4	51137	0.9885	1	0.9540	0.4861	177	0.07	7
Container Logistics, no resources	$\alpha = 0.1$	20	4	51137	0.6991	0.9995	0.2513	0.5758	90	0.15	3
Procure-to-Pay	-	50	7	35927	1.0000	1	1	0.1733	285	0	20
Procure-to-Pay	$\alpha = 0.1$	50	7	35927	0.9182	1	0.5351	0.36	104	0	2
Order Management, with resources	-	8	6	147463	-	-	-	0.1470	902	0.46	17
Order Management, no resources	-	8	6	147463	-	-	-	0.3258	349	0.63	8
Order Management, no resources	$\alpha = 0.1$	8	3	71196	0.4271	0.6410	0.0119	0.4359	270	0.82	2
Order Management, no resources	$\alpha = 0.1, \beta = 0.2$	8	3	71196	0.0000	0	0	0.5122	104	0.35	2
Hinge Production, no resources	-	50	9	67800	1.0000	1	1	0.7551	80	0	3

Table 6.1: The results of the evaluation on public OCELS.

6 Evaluation

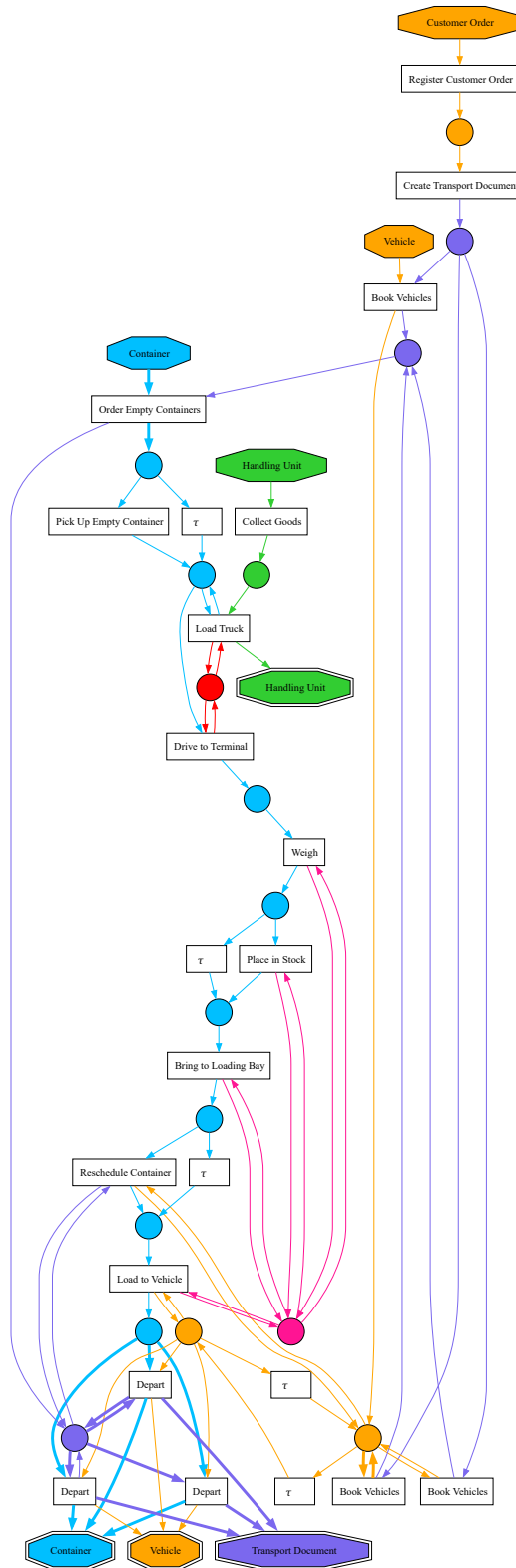


Figure 6.11: Output of the object flow miner on the container logistics log, with 10% filtering on configurations.

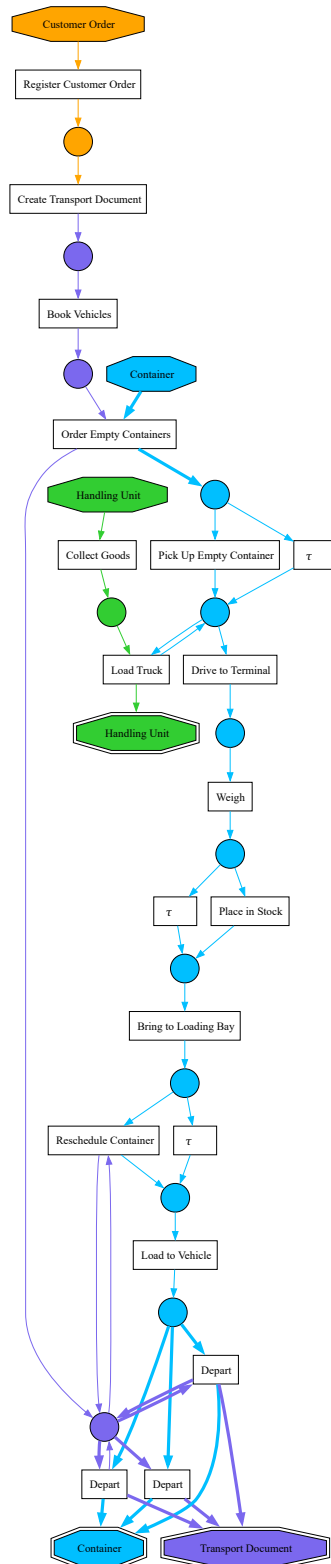


Figure 6.12: Output of the object flow miner on the container logistics log, with 10% filtering on configurations and filtering out *Forklift*, *Vehicle* and *Truck*.

6.2.3 Procure to Pay

In the procure-to-pay [39] log, the object flow is the flow of a purchase order through the procurement division of a company. The Petri net model without filtering is shown in Figure 6.13. It has perfect fitness. Even though the lower part of the Petri net is hard to read, it still serves the purpose of telling the user of the algorithm that there are many configurations of *Execute Payment*, which can be used to revise the complexity of the process and whether this is intended. The quantitative evaluation in Table 6.1 indicates that there are 20 identically labeled transitions in this model. Thus, even when the object flow miner creates a model that is too complex to handle by hand, this model can carry valuable information about the process, specifically which transitions contribute to the complexity. After inspecting the *Execute Payment* configurations, much of their variability comes from different multiplicities on input or output objects, since two configurations that are identical except for one having cardinality 1 and the other >1 on a single object are considered separate by the algorithm. Therefore, as a future improvement of the algorithm, we suggest optional abstractions over variable arcs, such as absorbing a configuration with a 1-cardinality into an otherwise identical configuration with a >1 -cardinality on the same object type, turning it into a ≥ 1 -cardinality.

By filtering out 10% of configurations, the discovered OFN in Figure 6.14 becomes readable and usable for the purpose of visualizing the process. This filtering step doubled the simplicity of the model and reduced the size from 285 to 104, but the fitness is still high at 91.82%. Only two *Execute Payment* transitions remain, whereas unfiltered there were 20. One of these takes multiple *purchase_orders* and *goods_receipts*, while the other takes one each. Thus, this is an example of an OCEL where a simple yet fit model is discovered by the object flow miner.

We point out that our model can aid in the detection of anomalies such as maverick buying. The model shows this by the lower *purchase_requisition* that is an initial input, to the right of the intended purchase requisition that is the output of *Approve Purchase Requisition* or *Delegate Purchase Requisition Approval*.

6.2.4 Order Management

The order management log [30] describes the handling of orders in a warehouse, where an order is issued that can contain multiple items, each of a specific product, that are shipped to a customer who pays for the order. This log is an example in which the model discovered by the object flow miner is very large, with a size of 902, the highest in our evaluation. It was too complex to calculate the fitness. We filter out the resource *products*, the handling unit *items*, as well as *employees* and *customers* because these are classic resources that deal with many different orders. Still, the fitness computation even without resources did not terminate.

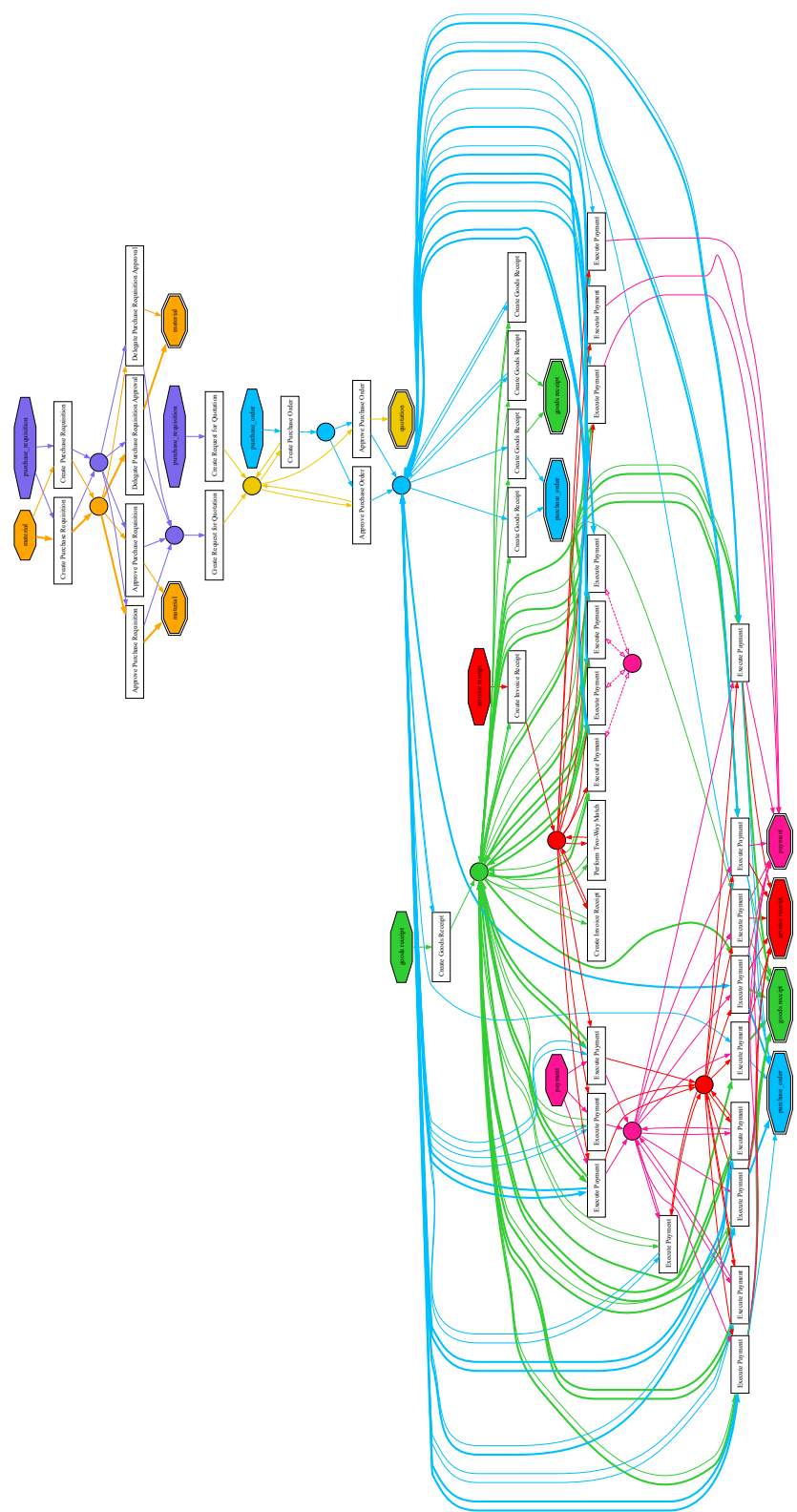


Figure 6.13: Output of the object flow miner on the procure-to-pay log, without filtering.

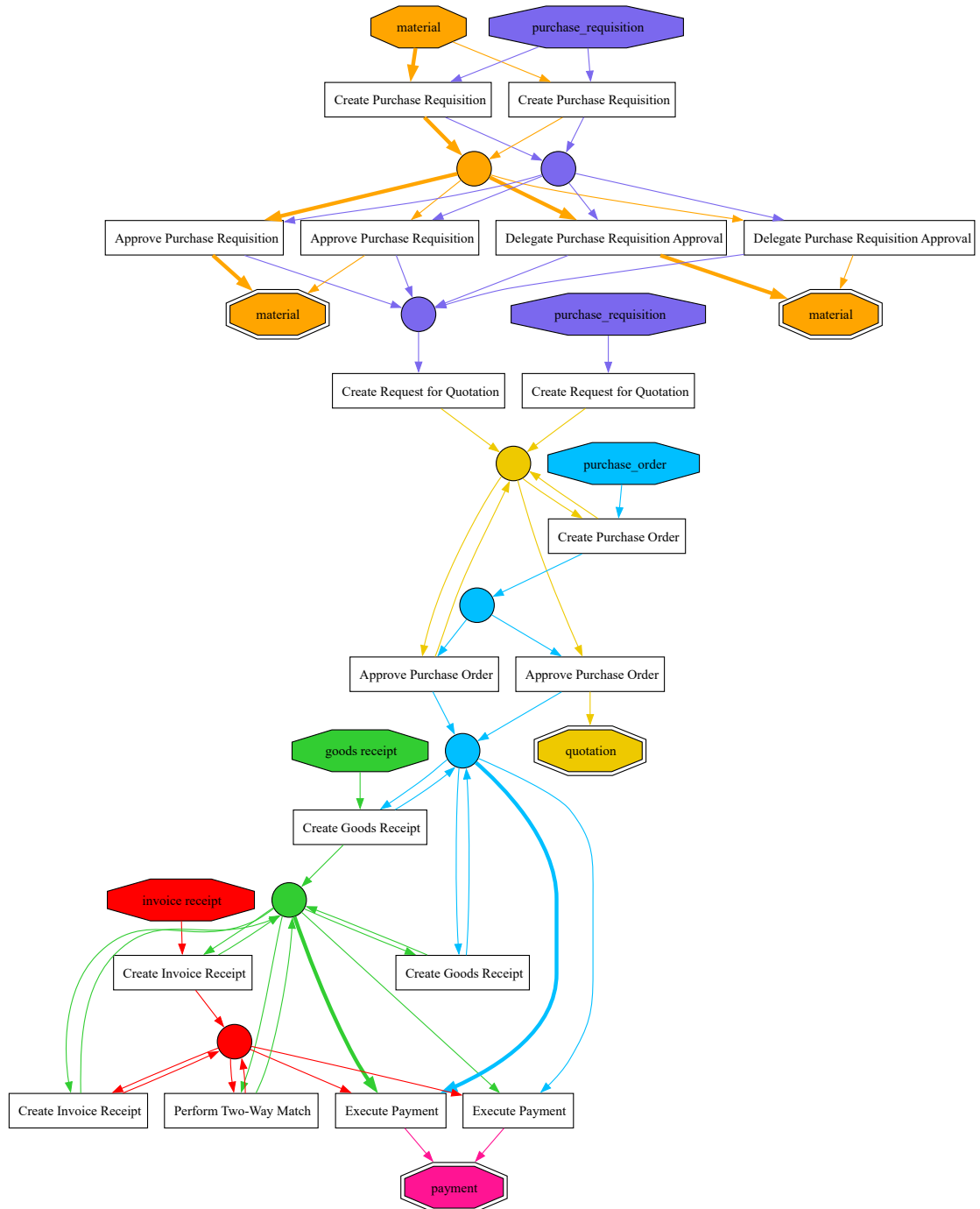


Figure 6.14: Output of the object flow miner on the procure-to-pay log, using 10% filtering on configurations.

If we additionally enable filtering out configurations below 10% prevalence, the created model, depicted in Figure 6.15, has a size of 270, but is still hard to read. One reason is that all items of an order are related to activities *confirm order*, *payment reminder* and *pay order*, even though conceptually these activities should only be related to *Orders*, not the *Items* of each order as well. Because these activities can occur concurrently with the item handling activities such as *pick item* and *create package*, the result is that there are many concurrent activities with regard to the *Item* type. As discussed previously, this concurrency then results in constructs with silent transitions.

Another reason for the large model is that there is the further difficulty that some events occur simultaneously, only conforming to a partial order of events, while we assumed a total order. For example, *pick item* and *create package* of the same item are sometimes recorded at the same timestamp, where the algorithm implementation is non-deterministic about which activity is considered first. This then causes more silent transitions to be created, which connect the two activities. The depicted model has a high ratio of 82% silent transitions for these two reasons.

In addition to the 10% filter on configurations, we can use the filter on preceding and succeeding activities to reduce silent transitions, since if an activity is filtered out of the list of next activities, no silent transition is drawn for it. Filtering by 20%, we obtain a simple and readable model; see Figure 6.16. This model has only 35% silent transitions, and only a size of 104, in contrast to a size of 349 for the full model without resources. However, one can note that at the very top, an orange *items* place does not have any incoming arcs, but is not marked as an initial input. This occurs because the incoming arcs were filtered out. It demonstrates that even though there exist filtering capabilities for logs as complicated as this one, the structure of the model can be impacted. This is underlined by the fact that this heavily filtered model has a fitness of 0%. In summary, the level of concurrency is not handled well by the object flow miner, and violating an assumption is also the cause of a complex model. Filtering works, but fitness drops to 0 as a result.

6.2.5 Conclusion about the Model Discovery

Overall, without filtering, the public OCEs generated very large models that are difficult to handle for a human. This is an inherent trait of the object flow miner, since every different configuration, even if they differ only by a single multiplicity, generates a new transition and its corresponding places and arcs. Although this hinders the usability of the model as a process reference, we argue that the purpose of the model is showing the inputs and outputs of each configuration in the log, and, thereby, visualizing the as-is process more than finding an abstract, underlying model. The unfiltered models achieved a fitness equal to or close to 100% at the cost of a large size and low simplicity.

There may be a large number of silent transitions, since each silent transition only consumes and produces a single object type, with multiplicity one. Some logs with

6 Evaluation

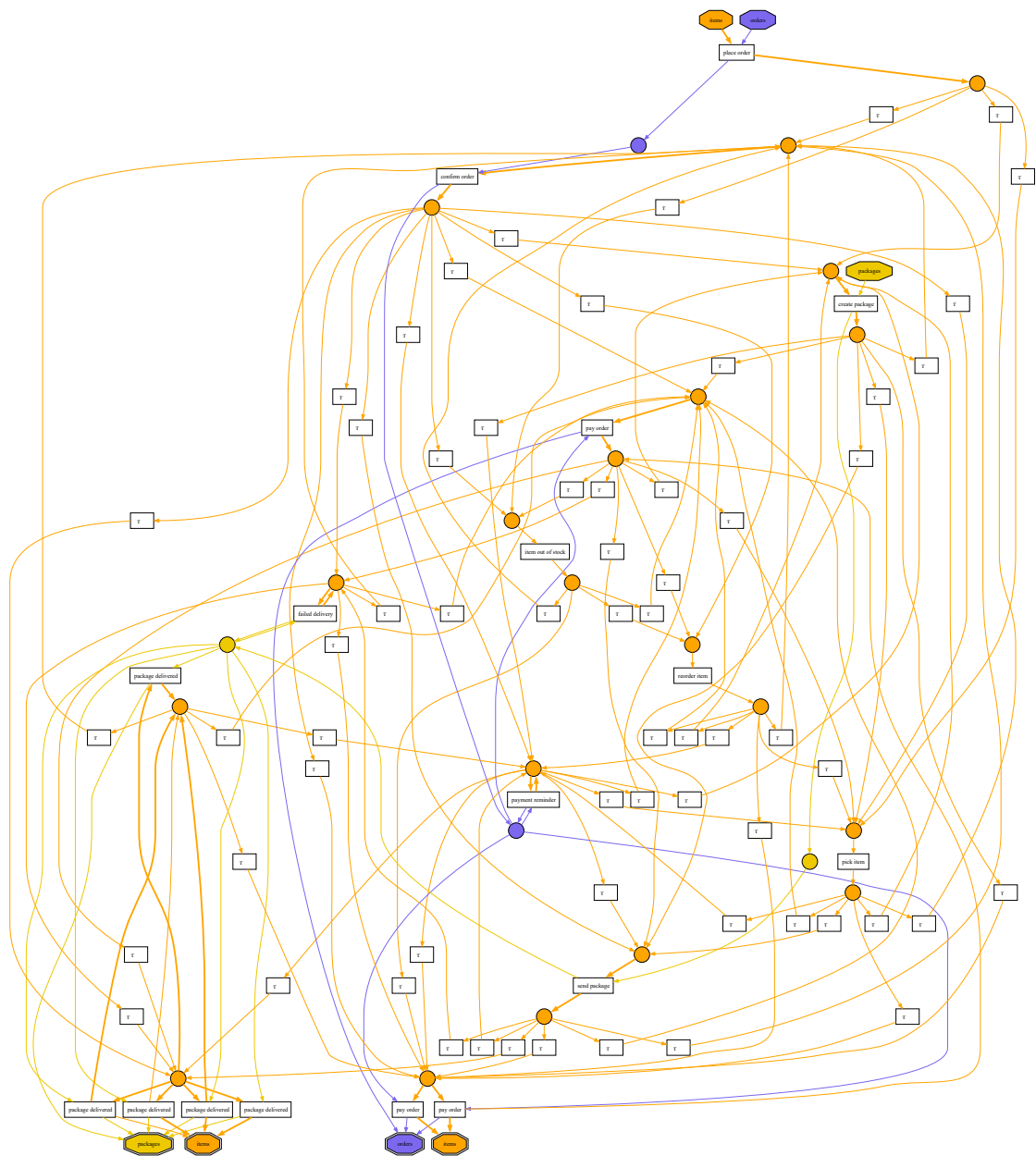


Figure 6.15: Output of the object flow miner on the order management log, filtering out *products*, *employees* and *customers* and using 10% filtering on configurations.

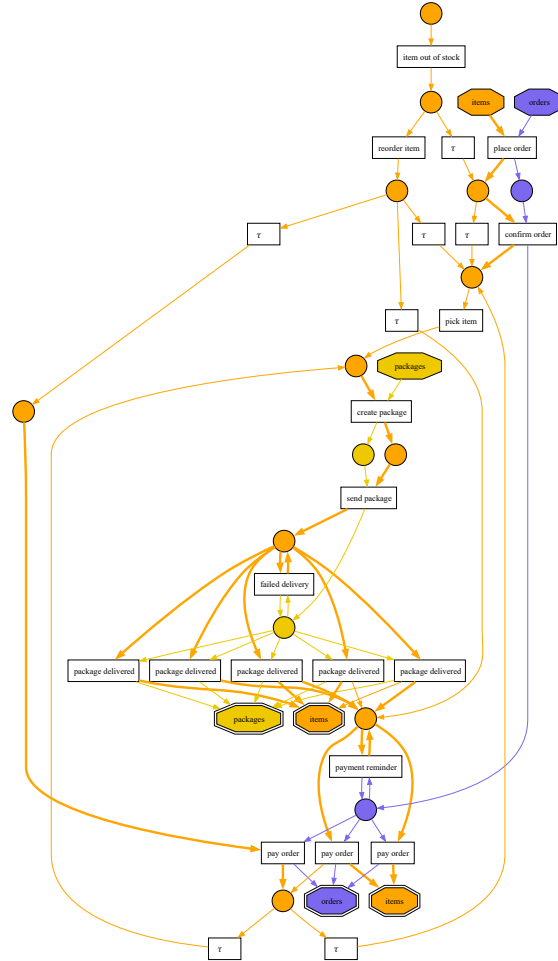


Figure 6.16: Output of the object flow miner on the order management log, filtering out *products*, *employees* and *customers*, and using 10% filtering on configurations and 20% on preceding and succeeding activities.

6 Evaluation

many relationships for each event, such as the order management log, generate many places with slightly different inputs and outputs, which cannot be simplified. As a result, many silent transitions are created to move between the places and achieve a fit model, which generates clutter and reduces readability. For future work, we suggest improving silent transitions to handle multiple object types and correct multiplicities.

If, instead of in-depth process analysis, the object flow miner is applied to discover a visual model of the object flow, the filtering methods are powerful tools. We managed to discover usable models through appropriate filtering for every log. In that case, fitness is sacrificed for simplicity.

The evaluation shows that our algorithm is suitable not only for strictly material flow, but for all kinds of handling units such as orders and shipments. This shows that while the object flow miner was developed with material flow analysis in mind, it functions as a general process discovery algorithm, providing an alternative model to other object-centric discovery algorithms. This may be especially useful in situations where a classic Petri net is desired as an output. In summary, based on our testing OCELs, the object flow miner achieves its goal of showing the material flow of the inputs and outputs of the process and each configuration. In addition, it can also be used for the purpose of discovering a general process model.

6.2.6 The Role of Resources

In three logs, we filtered out resources prior to applying the object flow miner, because they can cause issues such as complicated models and very long runtimes (see Section 6.4). However, resources carry information, and waste detection could be improved by associating waste with the resources related to it. For example, suppose that a *Machine* produces parts that are checked for abnormalities, and one specific machine produces parts that are much more likely to be malformed. Detecting this *cause* of waste can be done by taking resources better into account, which we suggest for future work.

6.3 Waste Detection

The logs considered in the previous section did not contain waste or a mass attribute. In this section, we evaluate the waste annotation with the hinge production log. First, it is necessary to discover an appropriate process model, in a manner similar to the previous sections, but with the addition of MFA that takes advantage of the mass attribute that is present. Then, on this model, we annotate various quantifications of the waste either as a count of objects or as mass. The result shows that the implicit system-wide waste and waste on output object types can be visualized in the model. Furthermore, this works for both the mass and count of waste. In order to have a valid baseline, we modified

the hinge production log in such a way that every object has a mass attribute and the mass-balance principle holds.

6.3.1 Discovering the Model

The hinge production log [21] is a simulated log that describes the production of hinges from steel in a factory. The conceptual model is given in Figure 6.17, where the factory and the workstations that visually span other activities are resources, and the worker and machine names to the left of an activity are also resources. The remaining gray boxes are handling units, which are transformed by the activities in white boxes.

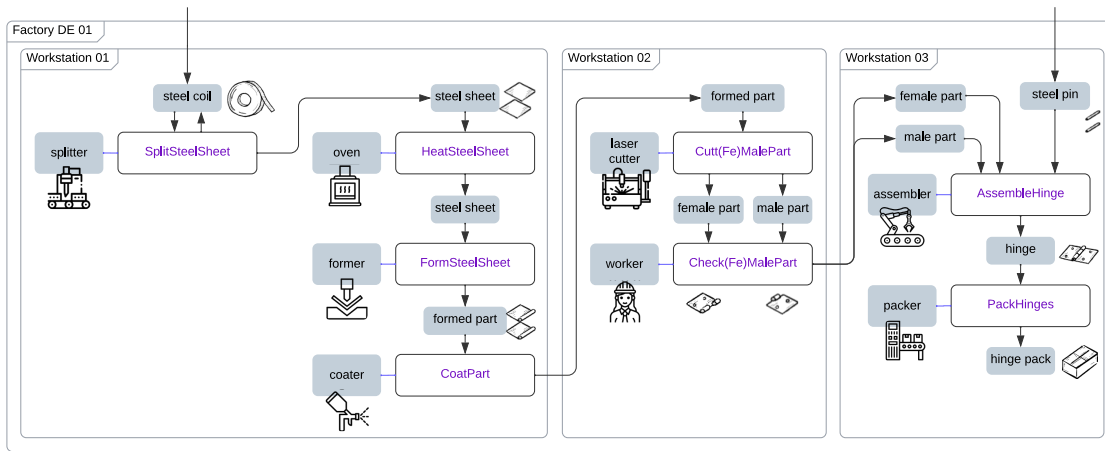


Figure 6.17: Conceptual model of the hinge production log [21].

First, we build a process model. We filter out the object types that are resources and apply the object flow miner with heuristics, which gives Figure 6.18. The model accurately reflects the material flow shown in the conceptual model, and further filtering is not necessary.

There are dashed double arcs for the *SplitWaste* (yellow), *CutWaste* (red), and *SteelPin* (dark blue) object types, which indicate that the role of these is unknown. The transitions related to these places fall under the fallback case in our heuristics due to a transformation involving different inputs and outputs. From the perspective of the algorithm, it is not possible to know whether *SteelPin* is an initial input or final output, because it only sees the transformation of a *FemalePart* and a *MalePart* into a *Hinge*.

We can use MFA to determine the roles of these unknown object types. In addition to that, we give two hints to the algorithm, which partially defines the role functions. First, we hint that the *SplitWaste* object type is always a final output, since it is currently falsely classified as an initial input. Second, we hint that *SteelSheet* is not an initial input, which is necessary when combined with the previous suggestion.

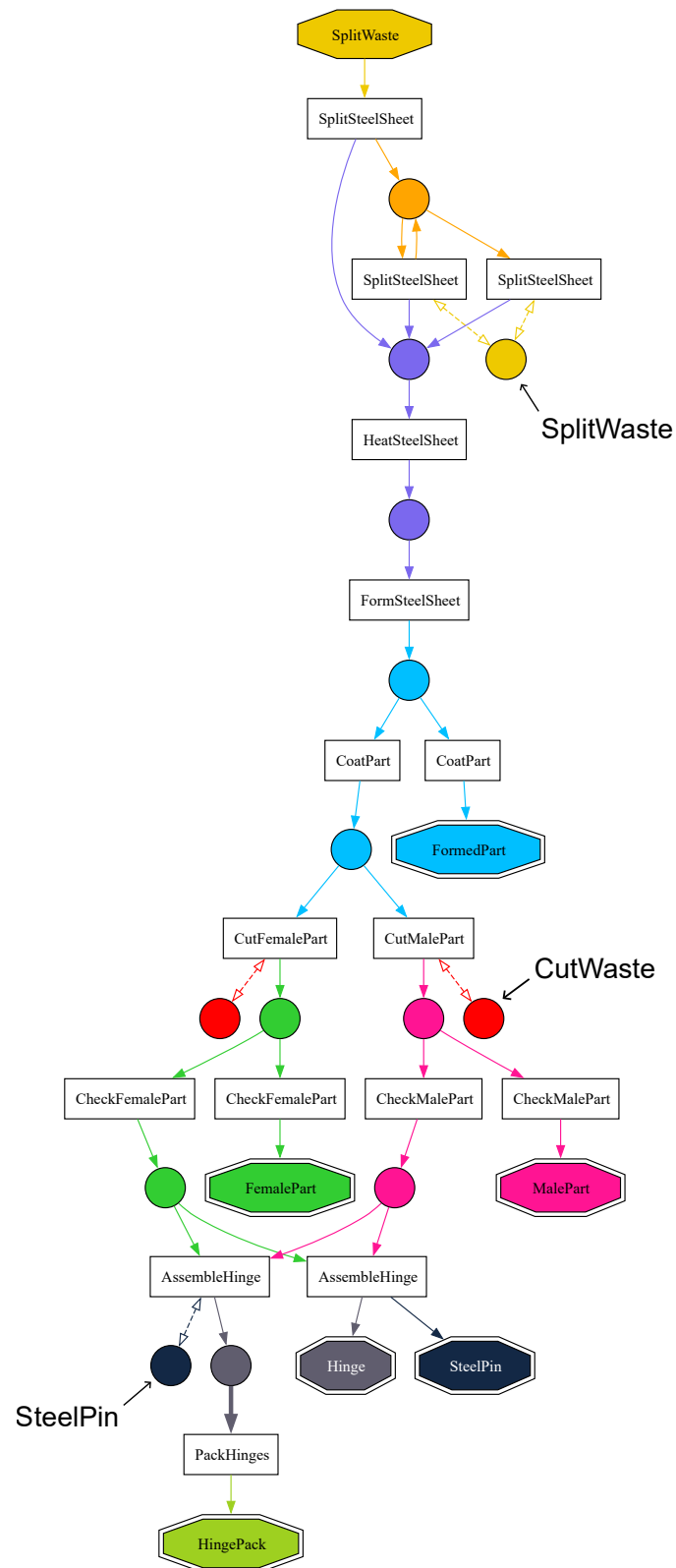


Figure 6.18: Output of the object flow miner on the hinge production log, without re-
sources *Facility*, *Workstation*, *Machine* and *Worker*.

We set the two hints and perform MFA to resolve the roles. It assigns *final output* to *CutWaste* and *initial input* to *SteelPin*, both correctly. Overall, it calculates an input mass of 130.54304 kg and an output mass of 130.16622 kg. This difference of less than 0.3%, which in strict terms violates the mass-balance principle, is due to the *SteelCoil* retaining some mass after all sheets are cut. This remaining mass is not added to the total output mass because *SteelCoil* is classified only as an initial input. Another reason is that *SteelPin* is a final output on the bottom right, although it should be an initial input. This represents an anomaly where the hinge is not packed in any *HingePack*. But since these mistakes made by the algorithm occur only a few times, the resulting difference in mass is small. The model after applying MFA is shown in Figure 6.19.

The discovered OFN has a simplicity of 75.51%, the highest in our evaluation, a low size of 80, and perfect fitness.

In the creation of the model, it was necessary to give two hints to the algorithm. After that, the algorithm resolves unknown roles based on the mass-balance principle. This shows that the heuristic algorithm is sometimes insufficient, but it is possible to intervene by providing a number of correct assumptions to take, and the remainder of the work is still done correctly and automatically. Edge cases such as the *SteelCoil* that contributes both to the input and output mass might not be handled incorrectly. As a result, in this example that fulfills mass balance in theory, the algorithm does not fully achieve it. We do not consider this invalidating for our method, since the mass difference is small and the mass-balance principle is seldom fulfilled in practice in any case [5].

6.3.2 Waste Annotation

Next, we annotate the model with waste. In the hinge production log, waste occurs explicitly in the waste object types *SplitWaste* and *CutWaste*. There are also objects that are not usually waste but can be if they are not used further after their creation, which is the case for *FormedPart*, *FemalePart*, *MalePart*, *SteelPin* and *Hinge*. This can be seen by the final output places of these object types, while the process is not finished yet. In addition, the *SteelCoil* object type is transformed into *SteelSheets*, but a small rest of the coil that is too small to be split remains, which is not explicitly given as waste in the log.

To detect this implicit waste, we show implicit waste in the *SplitSteelSheet* transitions, since that is where the steel coil is used. Additionally, we display the mass of the *CutWaste* and *SplitWaste* object types. To find out more details about the objects that are not used further, we show the total count of objects for the *FormedPart*, *FemalePart*, and *MalePart*.

We also show the mass on the initial inputs *SteelCoil* and *SteelPin*. This is achieved by modifying the definition of *owaste* to use the *Initial* set. While this is not waste, we include this annotation in order to get a model of the entire inventory, to automate the LCI analysis.

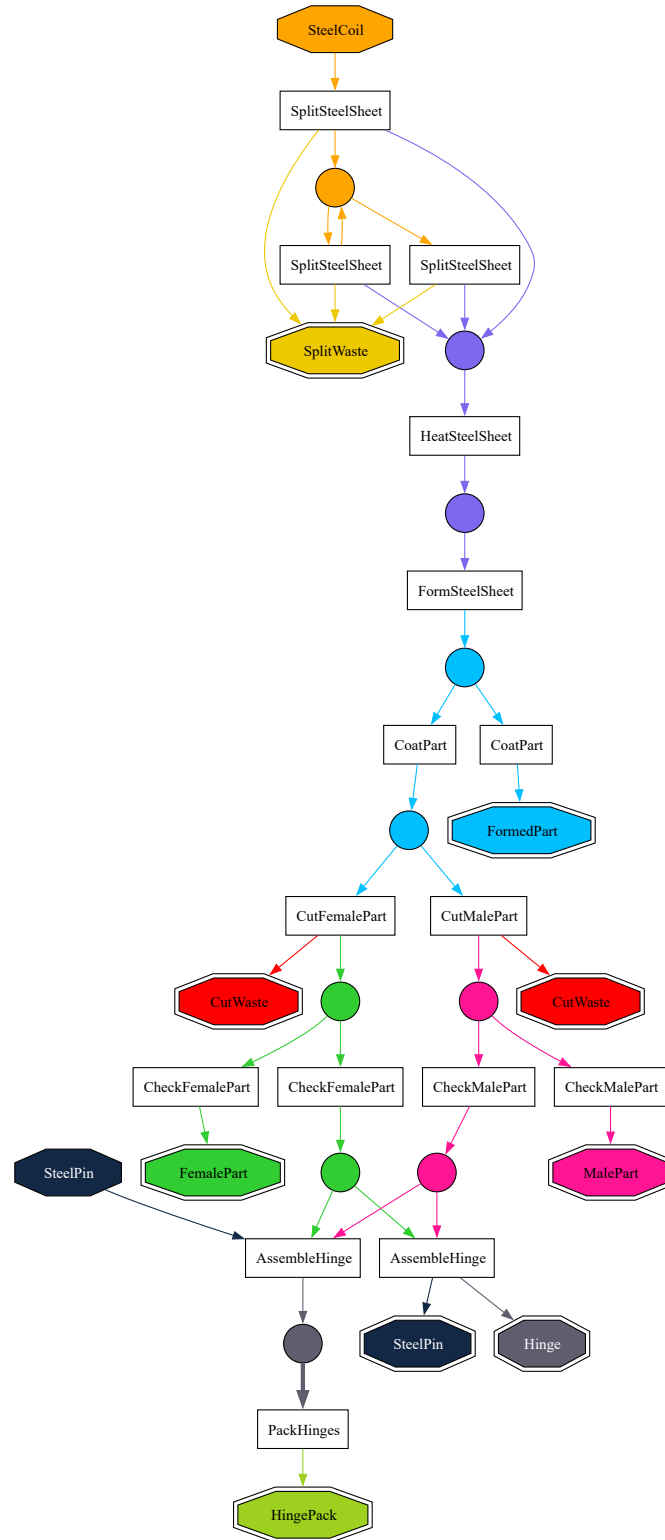


Figure 6.19: Output of the object flow miner on the hinge production log, without resources and with hints and MFA applied.

Contrary to our assumption, this log has object attribute changes. We had to adapt our code slightly to use the changed object attribute instead of statically using the object attribute.

The resulting annotated object flow net is shown in Figure 6.20. In the *SplitSteelSheet* transitions, we can see that only the transition that ends the loop of *SteelCoil* contains implicit waste, showing the average per event and the total for the configuration, which is expected since that is where the unused remainder of the steel coil is relevant. Both numbers are given in the unit of our mass (kg), but which is not rendered because the approach is unit-agnostic. Without the discovered object flow it would not be possible to detect this implicit waste as it is not directly given in the log.

The average and total waste mass is shown for the waste object types *CutWaste* and *SteelWaste*. In addition, we can see that 8 and 7 *FemalePart* and *MalePart*, respectively, are filtered out during their check. These numbers are given as absolute frequencies, since they are just counts of the object instances, so the average would simply be 1. The same type of count is shown for *FormedPart*, so it can be seen how often a *FormedPart* is produced but not used anymore — which is 82 times.

The masses are given on the initial inputs *SteelCoil* *SteelPin* as well. This shows that our algorithm is not limited to waste or even final outputs, but in general all inputs and outputs of the process, either simply by count or quantified by some attribute.

This flexibility makes the annotated hinge production model a suitable solution to the LCI analysis phase. After giving the hints, the model is an accurate representation of the material flow, and transformations of materials. The average and total mass of the waste object types *CutWaste* and *SteelWaste* is shown, which could be used in environmental reporting, or to plan the necessary transport to waste facilities. One can see the anomaly that 82 *FormedParts* instances are not cut, which is information about probably unintended waste, which is clearly visible in the model, which therefore makes it possible to deal with this issue. For future work, we suggest that it becomes possible to view the exact properties of these 82 formed parts, in particular object attributes and the path they took through the process. This could reveal commonalities, such as a supply of inferior quality. By also showing information about the initial input mass for *SteelCoil* and *SteelPin*, the model incorporates data that is useful for the LCI analysis.

It should not be understated that, assuming no major process changes, the model can be run on updated data at any time and without further adjustments and will immediately reflect the new data. If the LCI analysis were performed by hand, the new data would require manual recalculation of the waste. In fact, even process changes do not pose a significant amount of work, since the detection of the model is mostly automatic.

6 Evaluation

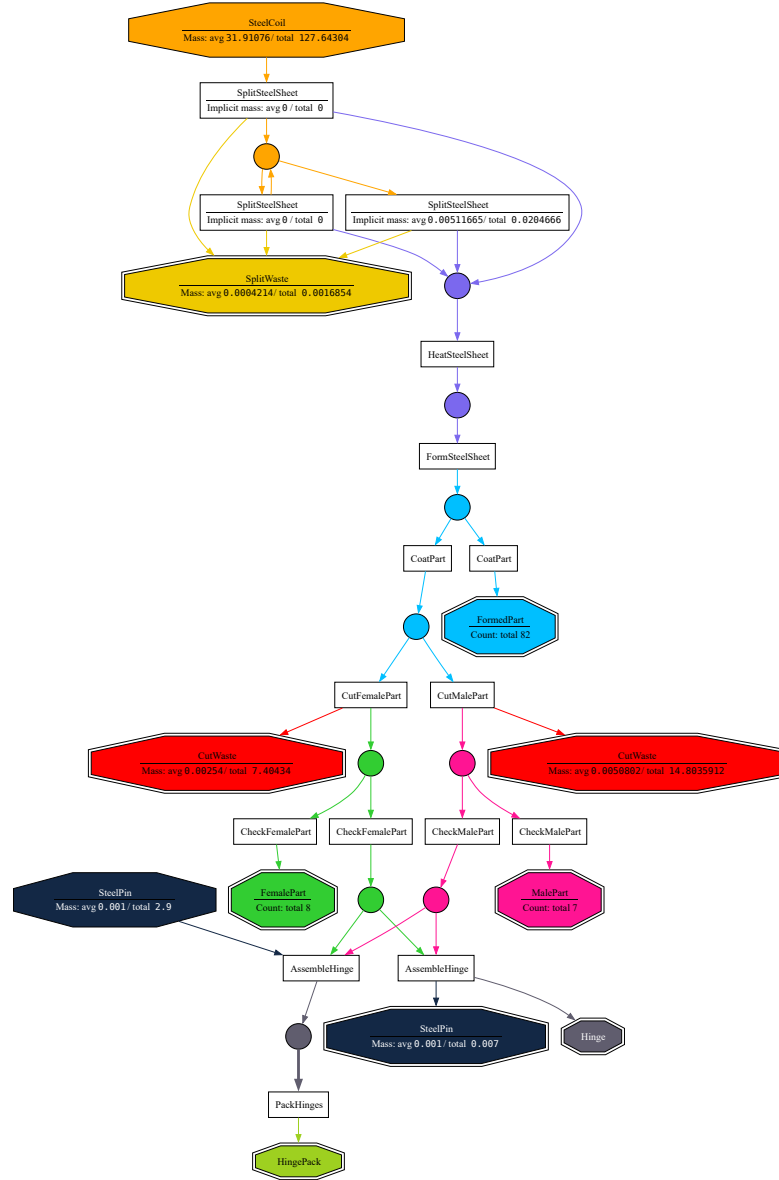


Figure 6.20: The waste annotated model of the hinge production log.

6.4 Runtime Performance

We measure the runtime of the object flow miner for the previously discovered models, to evaluate how feasible it is to run in practice. Executing the program in that scenario involves three steps. First, the log is read from a file. Second, the implementation described in Section 5.1, which finds initial inputs and final outputs, computes the role functions. Third, the algorithm is run on the log. For the hinge production log, we include another step for resolving unknown roles through applying MFA, before the algorithm is run. We hypothesize that the runtimes are acceptable for research software.

We first measure the runtime without any modification of the log as a baseline. For determining realistic runtimes, we use the parameters described for each OCEL in the previous chapter. Filtering out object types that are resources is part of the log reading, since these are filtered out prior to giving the log to the object flow miner. The filters on configurations and on preceding and succeeding activities is part of the algorithm runtime.

The experiments were run on a computer with an AMD Ryzen 7 5800H CPU, with Python version 3.13.7, and the `pm4py` [8] package on version 2.7.15. Every log was first converted to the SQLite format [7] using `pm4py`, since prior experiments revealed that this format is the fastest when reading the log. Before running the measurements, there was a preparation run of the program in case later executions of the same program are faster because of Python optimizations. For the end result, every program was run ten times and the runtime averaged, after checking for significant outliers, of which there were none. The results for unmodified OCELS without filtering are shown in Figure 6.21, and the results for the realistic parameters in Figure 6.22.

We can see that in both scenarios, the calculation of the role functions took up the majority of time, while reading the log was similar, and the object flow miner itself was very fast, taking at most 1.2 seconds for the unmodified order management log. Overall, except for the unmodified hinge production with 47.042 seconds, the total runtimes do not exceed 15.244 seconds. The reason that the hinge production log takes the longest is the presence of resources, some of which are related to every single event of an activity, and therefore thousands in total. Filtering them out to reduce global dependencies decreases the calculation to less than 20% of the unfiltered runtime. Thus, in summary, we consider the runtime to be acceptable.

The heuristic method for computing the role functions $role_{in}$ and $role_{out}$ given in our implementation takes up most of the processing time in our evaluation. These functions can be calculated separately for each event, and do not depend on each other in any form. This independent nature of our implementation allows good performance, but information could be gained from comparing similar events. In an event, it may be possible to infer the role functions by analyzing the roles in similar events. This could

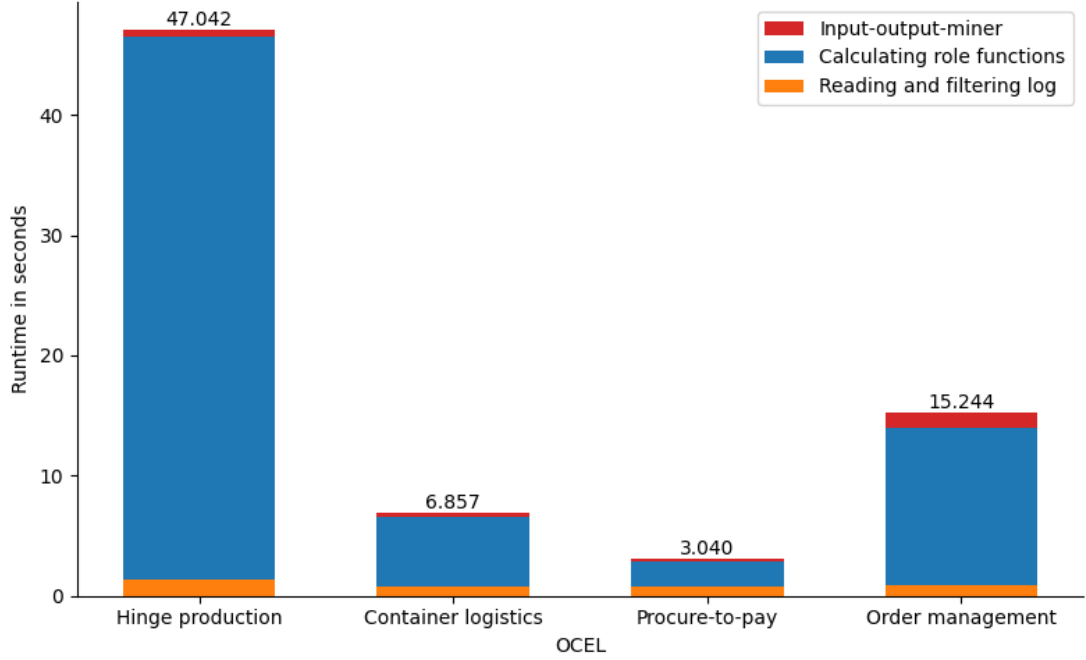


Figure 6.21: Runtime of the algorithm and setup using the unmodified public OCELs.

be especially useful for automatically amending errors in the log, and is left for future work.

We saw that large models were an issue multiple times in our evaluation, mainly caused by small differences that result in creating new configurations. It was possible to remove these configurations through basic filtering, but improvements at the algorithm level could produce a better result. For example, suppose that an activity *take order* takes *product* objects as an input. There can be many configurations of *take order*, as each number of related products in the log generates a different configuration when using arc weights. Because the number of products does not change the handling of the order, it would make sense to merge the configurations into one. This would sacrifice fitness for simplicity, as it shows an idealized process model. The current implementation is still necessary for visualizing the process as-is.

6.5 Applicability to LCA

The original goal and direct application of the object flow miner is to aid in life cycle assessment. This section synthesizes the previous points on model quality, usability and performance to provide a holistic assessment of how well the object flow miner and waste annotation method achieve this goal. For that, we have pointed out how the main use of

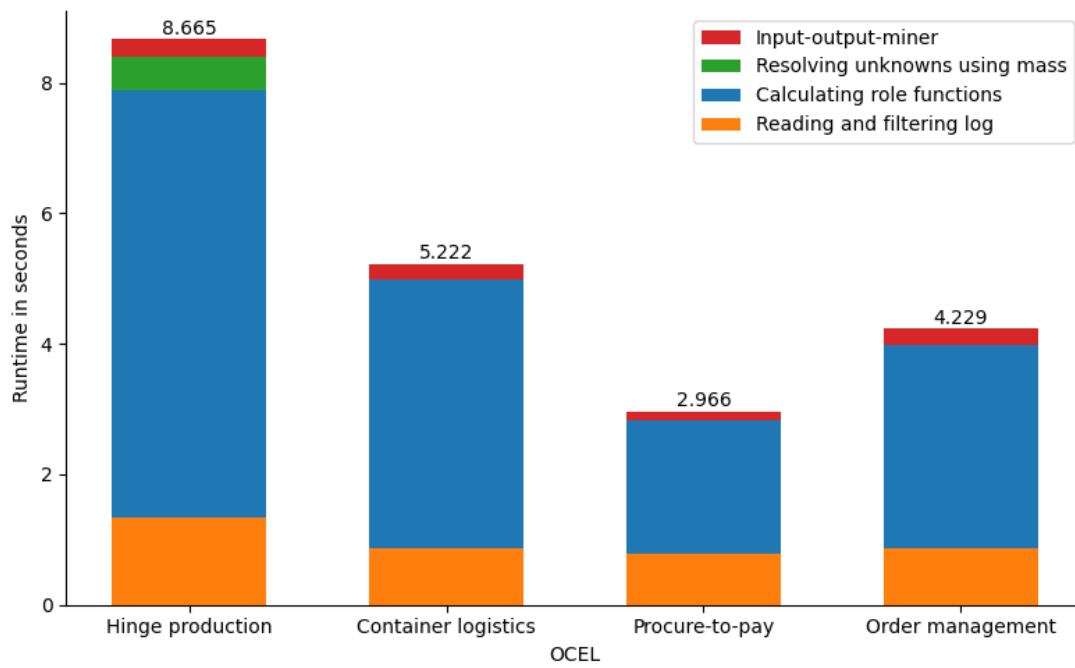


Figure 6.22: Runtime of the algorithm and setup using realistic parameters as described in Section 6.2 for each OCEL.

6 Evaluation

process mining in an LCA is in the data-driven LCI analysis phase. During this phase, the inputs and outputs of the process are collected, and optionally a model relating them is created. Often, a material flow analysis is used for this phase, which includes the mass-balance principle to ensure consistency.

As mentioned in Section 2.2, in the LCI analysis phase, it is the standard procedure to calculate the environmental impact in proportion to a single unit of the process output, the functional unit (FU). Because the strength of object-centric process mining allows for a detailed analysis of every object, we do not base our analysis around functional units. This is beneficial especially when considering irregularly occurring waste that cannot easily be distributed over the intended process output. Our model allows for a more fine-grained specification of the emission locations, whereas in a standard LCA, the emissions are calculated relative to only the FU. A problem with our approach is that making comparisons with existing LCAs that use FUs becomes difficult.

As an example of this, see the 82 unused intermediary *FormedParts* our model showed in the evaluation of the hinge production log in Section 6.3.1. Normally, such materials that contribute less than 1% to the FU are not considered [29], although these rare anomalies can carry important information. Our method both detects and quantifies them, and can show the quantification in a visual model that makes it clear where in the process they are created. In this way, the use of OCPM makes the analysis transparent and manageable, since algorithms can quickly search the entire data set, unlike humans.

Another strength of our method is the ease of maintenance, since the generation of the model is mostly automatic and can simply be rerun to incorporate updated data. The algorithm is fast, even for large logs, as our analysis of the runtime showed. This is normally difficult to do in a manual LCA [37], as an expert must perform the recalculation. Instead, once a method for getting the data is set up, updating the model with new knowledge is quick and inexpensive. Changes in the process are detected automatically by the object flow miner, which would otherwise require manual intervention by an expert. By partitioning the data based on windows of time and comparing the automatically generated models, it is easy to see evolutions and concept drift in the process. Doing this by hand would be time-consuming [37]. For future work, we suggest an extension that visualizes changes in the model and in generated waste.

In our object flow model, it is not generally possible to see the path that a final output has taken or an initial input will take, which could be important information for waste analysis as this can uncover its root cause. However, the fine-grained event log data does support such an analysis, by drilling down into the path that an object takes in the model. Therefore, for future work, we suggest a method to visualize the path of an object through the model. Until then, we suggest Event Knowledge Graphs [16] for this purpose.

7 Discussion

In this chapter, we discuss our method, taking into account the evaluation results from Chapter 6. We reflect on the choice of using object flow nets as outputs, and on the assumptions that the object flow miner relies upon. Afterward, we consider the design of our heuristic method for discovering role functions. In addition, we discuss the usability of waste annotations on an object flow net. Finally, we review the strengths and weaknesses of our entire approach to automating LCA.

7.1 Object Flow Nets

The model discovered by the object flow miner is an object flow net. This is a Petri net with weighted or variable arcs, where a variable arc consumes at least one token, whereas variable arcs in OCPNs can also consume or produce zero objects. In addition, places are typed and some places are marked as initial or final. Therefore, OFNs have commonalities with OCPNs, but their markings are tokens instead of object identifiers. The main benefit of OFNs over OCPNs is that they are a simpler model which satisfies the requirement that quantified inputs and outputs of a transition are visualized explicitly.

Using tokens simplifies the semantics of OFNs, as well-known algorithms for Petri nets can be used. For example, we used classic alignments together with flattened event logs in our evaluation to evaluate fitness. The purpose of an OFN is to show the inputs and outputs and their multiplicity for each transition. For this reason, for every object type, there are at most two input places, a regular and an initial input. This avoids the situation where object identifiers are necessary to consume the proper objects from different input places of the same type. Thus, anonymous tokens are sufficient for our purpose because in an OFN, a transition represents a transformation of tokens, while in an object-centric Petri net, a transition represents a synchronization point in the control flow of objects, which all persist before and after the transition. For these reasons, we chose to use OFNs in this thesis.

However, using an OCPN would allow the tracing of each individual object through the model, which is especially useful for finding the path of concrete waste objects. In contrast to this qualitative waste analysis, in our evaluation we focused on the quantitative aspect, where the mass or count of waste was shown on transitions or places. For this purpose, OFNs are a useful tool.

7.2 Object Flow Miner

The key step in our method is the object flow miner in combination with the heuristics for the role functions. This approach can be used to discover not only as-is models, but also general process models by using filtering. If no filtering is applied, the discovered OFN has perfect fitness with our alignments-based computation. This makes sense as for each possible combination of events and objects in the log, the appropriate transitions and places are discovered.

Our fitness computation uses alignments only on flattened event logs. This potentially overestimates true fitness. Suppose that two objects of different types have perfect alignment in their filtered Petri net. In the entire model, there is a transition that depends on both of them to be in the input places. However, this marking is unreachable, meaning the transition cannot fire when replaying the log. Then, our approach calculates the fitness as 1, even though it should be lower.

For the container logistics and procure-to-pay logs, filtering reduced the size of the model at the cost of a lower fitness. For the order management log, filtering to get a readable model reduced the fitness to 0. Thus, the object flow miner works well to discover what is in the underlying data, but is not always readily applicable as a general-purpose miner.

In the following, we discuss the assumptions underlying the object flow miner, and the limitations to which they lead. For that, we analyze the object flow miner itself, and also the heuristic discovery of the role functions.

7.2.1 Limitations of the Object Flow Miner

Assumption of no relation qualifiers. This assumption is easily met by preprocessing the log.

Underfitting concurrency. In our evaluation in Section 6.1.2, we observed that the object flow miner underfits concurrent constructs. Usually, Petri nets are a good choice if there is concurrency in the underlying data, as they support it natively. We also take advantage of this, but allow additional behavior. We chose Petri nets as a representation since they provide an intuitive distinction of places for material conditions and transitions for transformations. For future work, we suggest an additional algorithm step to properly model concurrency without underfitting.

Restriction to the local context. Both the heuristics for the role functions and the discovery of configurations in our method are local operations. They do not investigate the larger context further than the immediate preceding or succeeding activity. This is also the reason for our assumption of a total order of events, since this ensures a deterministic local context. It is not clear whether this approach is sufficiently powerful to model all necessary processes that arise in an LCI analysis. The current approach worked for all example logs but order management in Section 6.2.4, where a very large model with a high ratio of silent transitions was discovered. This was caused by many objects being related to logically distinct but concurrently occurring events. For these events, the object flow miner generates silent transitions connecting the activities, which implies a causal relation that does not actually exist. We evaluated our method only on four logs, so more research is needed to determine the types of logs where the object flow miner produces these insufficient models. In general, it is not possible to trivially transform an object-centric event log with simultaneous events into a log with a total order of time. Identical timestamps may occur in logs where operations are effectively instantaneous. For example, the creation of a package and the instantiation of the picking process often occur at the same timestamp in the order management process. Our approach cannot handle this case.

Disregard of object attribute changes. In addition, we address our handling of object attribute changes. Although the OCEL 2.0 standard allows objects to modify their attribute values over time, we assumed that all object attributes remain static. We use this restriction because supporting attribute changes would introduce considerable complexity, particularly the challenge of unambiguously determining which value an attribute holds at the specific timestamp of an event. For example, suppose that an event e changes a weight attribute, but it requires some time to take effect. The event occurs at $time(e)$, yet the attribute is only changed at $time(e) + 5$. Then, it is not clear what the input and output weight of the object in e is, since the object flow miner does not know that the change of the object attribute later is caused by e . Therefore, this design choice prioritizes the stability of the waste annotations over the full flexibility of the OCEL specification.

7.2.2 Limitations of the Heuristic Discovery of the Role Functions

Assumption of an underlying material process. For discovering the role functions automatically, we presented a heuristic method based on a case distinction. The logic of this case distinction is based on assuming that the process contains different patterns for logistical and transformational processes. Although our evaluation showed that it worked well on our example logs, this design decision requires the log to exhibit a straightforward pattern. The hinge production log, evaluated in Section 6.3.1, showed that an input object can be used multiple times, which does not match this pattern and thereby required us to manually correct the role functions. Furthermore, the assumptions about

7 Discussion

logistical and transformational activities are based on material processes, and could be violated for processes on intangible products where physical transformation is not required. For future work, we suggest researching improved methods to discover these role functions, for example by taking a larger context of events into account.

No distinction between intermediate and final objects. If there are unresolved roles after applying the heuristics and a mass attribute is present, this attribute can be used to determine the unknown roles by approximating mass balance. Applying MFA in this way assumes that for the entire log, the total input mass is equal to the total output mass. This is not necessarily correct, as there might be intermediate products if the log is extracted at a time when the process is still running. The hinge production in our evaluation might have exhibited this phenomenon, as some *FormedParts* and *Hinges* were detected as waste, even though they might just be intermediate products at the time of the log creation. Since OCELs do not have a case notion, it is difficult to decide whether an object is only intermediate or whether it is waste. We suggest that the distinction between waste and intermediate products be researched in future work, as currently these products are assumed to be waste.

In our evaluation, the proposed combination of using heuristics to determine the inputs and outputs and applying the object flow miner on the result generally performed well and only few interventions were necessary, in particular in the hinge production log. The filtering capabilities were useful to reduce the complexity of the model. Nevertheless, we recognize that there are several suggestions for improvement, such as taking context into account, handling object attribute changes, and allowing simultaneous events. Therefore, we answered our research question on modeling the process based on the object flow and presented a tool for this purpose.

7.3 Waste Annotation

In the evaluation with the hinge production log, we saw that our method is capable of performing an MFA. Inflows and outflows of the process are computed and visualized with their mass. Waste flows on object types, as a specific outflow, are quantified in the same manner. Explicit waste given by event attributes can also be detected. By calculating system-wide waste and mass differences in events, even implicit waste can be determined on both the process and event level. Thus, with regard to our research questions about the occurrence and visualization of waste, our method effectively detects and visualizes waste in an OCEL. A limitation is that the annotated information is kept brief.

Limited information visualization. Although the current implementation of the waste-annotated OFN succeeds in visualizing waste directly in the process model, the existing

annotations are rudimentary. Currently, the model only visualizes the average and total waste on object types or activities. Although this is sufficient for a high-level LCI analysis, it does not offer the depth required for a thorough analysis of process inefficiencies. For example, the model does not provide a specific list of the individual objects that constitute the discovered waste, which prevents a user from analyzing the specific attributes or histories of those items. Furthermore, additional visualizations can be added, such as distributions of the waste mass, and metrics such as the median and standard deviation. This would improve the applicability especially in the intended domain of waste detection.

7.4 Applicability to LCA

We established that for the hinge production log, the waste flows were annotated on the discovered process model. In addition, we showed that the waste calculation for object types can be applied to final outputs and to initial inputs, which demonstrates that our method detects not only waste outflows, but regular outflows and inflows as well. This completes the requirements for the MFA part of the LCI analysis. The algorithm runs in under a minute, even on large example logs. However, there are limitations. Our algorithm assumes high-quality data, and does not deal with the data acquisition of the LCI analysis. Additional limitations of our approach include the lack of an evaluation of a complete life cycle process and usability. In summary, in regard to our research question of how effectively our approach automates the LCI analysis phase, we conclude that the approach is a promising automation tool that discovers a model, inflows, outflows, and waste, but depends on ideal data sets. Nonetheless, it contributes to filling the research gap in applying object-centric process mining in the domain of sustainability. In the following, we elaborate on the limitations.

7.4.1 Limitations

High-quality data. Throughout this thesis, we assumed high-quality data. The object flow miner does not sanity-check or estimate missing values, which is possible when evaluating manually. Therefore, our approach disregards the fact that real data often has issues [5]. Poor data quality can result in a complicated process model unless filtering is applied. For instance, wrong timestamps can result in a large and complicated model, similar to the output on the order management log. In general, data acquisition presents one of the greatest complexities in conducting LCA [29], which is not made easier by our method. It is also unlikely that the recorded data is in the OCEL 2.0 format, which has only existed since 2023.

Lack of reference LCA A significant limitation of the current evaluation is the absence of a direct comparison between the results of the object flow miner and the LCI analysis conducted with traditional manual methodologies. While the approach successfully quantified waste streams and demonstrated mass-balance consistency for the hinge production log, this is a simulated log. In addition, none of the logs used in our evaluation cover the entire life cycle of their objects. In a manual assessment, practitioners often apply abstractions, whereas the object flow miner takes all data into account. Consequently, the results discovered by our method might differ from traditional reports, and it is not clear that completely disregarding abstractions leads to better results. For these reasons, it would be important to compare the object flow miner applied to a real-world log with a reference LCI analysis for the same data. This can also serve as an experiment where the object flow miner is applied to data of a lower quality.

Usability. The intention of our method is that even people unfamiliar with process mining can interpret the model to find out where waste is created. The Oculoscope plugin interface exists so that no technical knowledge is required; however, it can be difficult to understand the entire workflow, including filtering out resources and understanding what the filter parameters do, without any knowledge about process mining. Furthermore, it must be explained how Petri nets work. Consequently, it would be useful to conduct a usability study with LCA practitioners. This would also shed light on the applicability to real-world logs.

8 Conclusion

This thesis explored the application of object-centric process mining to life cycle assessment. It contributes to the automation of LCI analysis, a critical phase in LCA responsible for modeling and quantifying the inputs and outputs of a system. As a particular use case, we focused on waste detection, as waste can be derived from the inventory. We motivated our research by highlighting that traditional LCI analysis is often a manual, time-consuming, and costly procedure and typically results in static models that are difficult to update.

For this purpose, we proposed the object flow miner, a discovery algorithm that constructs an object flow net directly from object-centric event logs. These nets visualize the transformation of input objects into output objects for every activity, together with the corresponding object types and multiplicities. In addition, the inputs and outputs of the entire process are marked. The advantage of this approach is its ability to distinguish between different configurations of the same activity. In contrast to existing process mining algorithms that focus on control flow, the OFN explicitly visualizes the object flow based on inputs and outputs. We further extended our approach by annotating an OFN with waste data derived from quantitative mass attributes or object counts, achieving waste detection.

As a prerequisite, we required high-quality OCEs with certain restrictions, such as a total order of events, and the absence of object attribute changes. Optionally, filtering parameters and the desired waste detection can also be specified. On these inputs, the role functions are computed heuristically, which are used for the object flow miner that discovers an OFN and annotates it with waste. This effectively automates the LCI analysis.

The evaluation of our approach showed its effectiveness in both theoretical and practical scenarios. The algorithm discovered the basic constructs such as exclusive choice, loops, and non-free choice found in process models, but the discovery of parallelism is poor, and the OFN semantics do not allow for discovering implicit places. Our method generated as-is models of the process with close to perfect fitness on four public OCEs. When applied, filtering successfully reduced the size of process models to concise representations of the underlying processes. The application to the hinge production log validated the applicability of the method for waste detection. The annotated OFN revealed waste information on object types, as well as implicit waste on activities. This information is derived from a provided mass attribute or counting automatically. By applying the waste detection in reverse to the input object types as well, our approach

8 Conclusion

semi-automated the LCI analysis phase, requiring only two manual hints. The runtime performance was shown to be acceptable for practical use. However, a major limitation is that we did not compare our approach with a reference LCI analysis on real-world data.

8.1 Future Work

The limitations of our algorithm suggest various directions for future work. Our approach assumes that events never occur simultaneously, which is typically not the case, as we observed in our evaluation. Therefore, handling simultaneous events would allow using the method on more OCEs. Additionally, supporting object attribute changes would also make the approach more general, as currently attributes are assumed to be static. The heuristic discovery of role functions, which is currently a case distinction based only on the preceding and succeeding activity of the object traces, could be improved by analyzing a wider context of events. Further research into detecting parallelism and implicit places would enhance the precision of the discovered object flow nets. Finally, visualizing the paths of individual objects through the model would allow for a deeper root-cause analysis of the detected waste.

Bibliography

- [1] W. M. P. van der Aalst. *Data science in action*. Springer, 2016. ISBN: 978-3662570418. DOI: 10.1007/978-3-662-49851-4.
- [2] W. M. P. van der Aalst and A. Berti. “Discovering object-centric petri nets.” In: *Fundamenta informaticae* 175.1-4 (2020), pp. 1–40. DOI: 10.3233/FI-2020-1946.
- [3] W. M. P. van der Aalst, T. Weijters, and L. Maruster. “Workflow mining: Discovering process models from event logs.” In: *IEEE transactions on knowledge and data engineering* 16.9 (2004), pp. 1128–1142. DOI: 10.1109/TKDE.2004.47.
- [4] M. K. Ansah, X. Chen, H. Yang, L. Lu, and P. T. Lam. “Developing an automated BIM-based life cycle assessment approach for modularly designed high-rise buildings.” In: *Environmental Impact Assessment Review* 90 (2021), p. 106618. DOI: 10.1016/j.eiar.2021.106618.
- [5] R. U. Ayres. “Life cycle analysis: A critique.” In: *Resources, Conservation and Recycling* 14.3 (1995). Life Cycle Management, pp. 199–223. DOI: 10.1016/0921-3449(95)00017-D.
- [6] G. O. Barrionuevo, J. Andrade-Garzón, L. Llatance-Guevara, and F. E. Alarcón. “Life Cycle Assessment of Tensile Specimens of Stainless Steel Obtained by Additive Manufacturing versus Conventional Manufacturing.” In: *Advanced Materials & Sustainable Manufacturing* 2 (2025), p. 10004. DOI: 10.70322/amsm.2025.10004.
- [7] A. Berti, I. Koren, J. N. Adams, G. Park, B. Knopp, N. Graves, M. Rafiei, L. Liß, L. T. G. Unterberg, Y. Zhang, C. Schwanen, M. Pegoraro, and W. M. P. van der Aalst. *OCEL (object-centric event log) 2.0 specification*. 2024. DOI: 10.48550/arXiv.2403.01975.
- [8] A. Berti, S. van Zelst, and D. Schuster. “PM4Py: A process mining library for Python.” In: *Software Impacts* 17 (2023), p. 100556. DOI: 10.1016/j.simpa.2023.100556.
- [9] A. Bjørn, M. Owsianiak, C. Molin, and A. Laurent. “Main Characteristics of LCA.” In: *Life Cycle Assessment: Theory and Practice*. Ed. by M. Z. Hauschild, R. K. Rosenbaum, and S. I. Olsen. Springer International Publishing, 2018, pp. 9–16. ISBN: 978-3-319-56475-3. DOI: 10.1007/978-3-319-56475-3_2.
- [10] F. R. Blum. “Metrics in process discovery.” In: *University of Chile, Santiago, Chile, Report* (2015), pp. 1–21.

Bibliography

- [11] L. Brehm, J. Slamka, and A. Nickmann. “Process Mining for Carbon Accounting: An Analysis of Requirements and Potentials.” In: *Digitalization Across Organizational Levels: New Frontiers for Information Systems Research*. Ed. by J. Dibbern, J. Förderer, T. Kude, F. Rothlauf, and K. Spohrer. Springer International Publishing, 2022, pp. 209–244. ISBN: 978-3-031-06543-9. DOI: 10.1007/978-3-031-06543-9_9.
- [12] P. H. Brunner and H. Rechberger. *Handbook of Material Flow Analysis: For Environmental, Resource, and Waste Engineers*. 2nd ed. CRC Press, 2016. DOI: 10.1201/9781315313450.
- [13] J. Carmona, J. Cortadella, and M. Kishinevsky. “New Region-Based Algorithms for Deriving Bounded Petri Nets.” In: *IEEE Transactions on Computers* 59.3 (2010), pp. 371–384. DOI: 10.1109/TC.2009.131.
- [14] I. P. O. C. Change. *AR5 Climate Change 2014: Mitigation of Climate Change*. Cambridge University Press, 2014. URL: <https://www.ipcc.ch/report/ar5/wg3/>.
- [15] A. K. Christfort, A. Rivkin, D. Fahland, T. T. Hildebrandt, and T. Slaats. “Discovery of Object-Centric Declarative Models.” In: *2024 6th International Conference on Process Mining (ICPM)*. 2024, pp. 121–128. DOI: 10.1109/ICPM63005.2024.10680680.
- [16] D. Fahland. “Process Mining over Multiple Behavioral Dimensions with Event Knowledge Graphs.” In: *Process Mining Handbook*. Ed. by W. M. P. van der Aalst and J. Carmona. Cham: Springer International Publishing, 2022, pp. 274–319. ISBN: 978-3-031-08848-3. DOI: 10.1007/978-3-031-08848-3_9.
- [17] A. Fritsch. “Sustainability Analysis Patterns for Process Mining and Process Modelling Approaches.” In: *Process Mining Workshops*. Ed. by A. Delgado and T. Slaats. Cham: Springer Nature Switzerland, 2025, pp. 725–737. ISBN: 978-3-031-82225-4. DOI: 10.1007/978-3-031-82225-4_53.
- [18] N. Graves, A. Fritsch, R. Hensen, I. Koren, and W. M. P. van der Aalst. “Object-Centric Process Mining for Semi-Automated and Multi-Perspective Sustainability Analyses.” In: *2025 11th International Conference on ICT for Sustainability (ICT4S)*. 2025, pp. 143–152. DOI: 10.1109/ICT4S68164.2025.00023.
- [19] N. Graves, I. Koren, and W. M. P. van der Aalst. “ReThink Your Processes! A Review of Process Mining for Sustainability.” In: *2023 International Conference on ICT for Sustainability (ICT4S)*. IEEE. Apr. 2023, pp. 164–175. DOI: 10.1109/ICT4S58814.2023.00025.
- [20] P. Haun, P. Müller, and M. Traverso. “Improving automated Life Cycle Assessment with Life Cycle Inventory model constructs.” In: *Journal of Cleaner Production* 370 (2022), p. 133452. DOI: <https://doi.org/10.1016/j.jclepro.2022.133452>.
- [21] M. Heinisch, N. Graves, and W. M. P. van der Aalst. *Socel 2.0: A Sustainability-enriched OCEL of a Hinge Production Process*. Sept. 2024. DOI: 10.5281/zenodo.13638681.

- [22] R. Hensen. “OCEAn: A Web Application for Object-Centric Emission Analysis Using Process Mining.” Master’s Thesis. RWTH Aachen University, 2024.
- [23] M. A. Huijbregts, Z. J. Steinmann, P. M. Elshout, G. Stam, F. Verones, M. Vieira, M. Zipp, A. Hollander, and R. Van Zelm. “ReCiPe2016: a harmonised life cycle impact assessment method at midpoint and endpoint level.” In: *The international journal of life cycle assessment* 22.2 (2017), pp. 138–147. DOI: 10.1007/s11367-016-1246-y.
- [24] ISO 14040:2006. *Environmental management – Life cycle assessment – Principles and framework*. International Organization for Standardization, 2006. URL: <https://www.iso.org/standard/37456.html>.
- [25] ISO 14044:2006. *Environmental management – Life cycle assessment – Requirements and guidelines*. International Organization for Standardization, 2006. URL: <https://www.iso.org/standard/38498.html>.
- [26] J. Kleinekorte, L. Fleitmann, M. Bachmann, A. Kätelhön, A. Barbosa-Póvoa, N. von der Assen, and A. Bardow. “Life Cycle Assessment for the Design of Chemical Processes, Products, and Supply Chains.” In: *Annual Review of Chemical and Biomolecular Engineering* 11. Volume 11, 2020 (2020), pp. 203–233. DOI: <https://doi.org/10.1146/annurev-chembioeng-011520-075844>.
- [27] F. Klessascheck, Y. Seibert, and L. Pufahl. “Capturing Environmental Impacts of Business Processes with Camunda and the SOPA Framework.” English. In: *CEUR Workshop Proceedings* 4032 (2025), pp. 176–183. ISSN: 1613-0073. URL: <https://portal.fis.tum.de/en/publications/capturing-environmental-impacts-of-business-processes-with-camunda>.
- [28] F. Klessascheck, I. Weber, and L. Pufahl. “SOPA: a framework for sustainability-oriented process analysis and re-design in business process management.” In: *Information Systems and e-Business Management* 23.2 (Jan. 2025), pp. 489–537. DOI: 10.1007/s10257-024-00695-x.
- [29] W. Klöpffer and B. Grahl. *Life Cycle Assessment (LCA)*. John Wiley & Sons, Ltd, 2014. ISBN: 9783527655625. DOI: 10.1002/9783527655625.
- [30] B. Knopp and W. M. P. van der Aalst. *Order Management Object-centric Event Log in OCEL 2.0 Standard*. Sept. 2023. DOI: 10.5281/zenodo.8428112.
- [31] B. Knopp and N. Graves. *Container Logistics Object-centric Event Log*. Aug. 2023. DOI: 10.5281/zenodo.8428084.
- [32] B. Köck, A. Friedl, S. Serna Loaiza, W. Wukovits, and B. Mihalyi-Schneider. “Automation of life cycle assessment—A critical review of developments in the field of life cycle inventory analysis.” In: *Sustainability* 15.6 (2023), p. 5531. DOI: 10.3390/su15065531.

Bibliography

- [33] A. Levasseur, P. Lesage, M. Margni, L. Deschênes, and R. Samson. “Considering time in LCA: dynamic LCA and its application to global warming impact assessments.” In: *Environmental science & technology* 44.8 (2010), pp. 3169–3174. DOI: 10.1021/es9030003.
- [34] G. Li, R. M. de Carvalho, and W. M. P. van der Aalst. “Automatic Discovery of Object-Centric Behavioral Constraint Models.” In: *Business Information Systems*. Ed. by W. Abramowicz. Cham: Springer International Publishing, 2017, pp. 43–58. ISBN: 978-3-319-59336-4. DOI: 10.1007/978-3-319-59336-4_4.
- [35] L. Liss and W. M. P. van der Aalst. “Process Area Extraction by Multilevel Resource Detection for Object-Centric Process Mining.” In: *Business Process Management*. Ed. by A. Senderovich, C. Cabanillas, I. Vanderfeesten, and H. A. Reijers. Cham: Springer Nature Switzerland, 2026, pp. 197–214. ISBN: 978-3-032-02867-9.
- [36] A. Möller and A. Rolf. “Methodische Ansätze zur Erstellung von Stoffstromanalysen unter besonderer Berücksichtigung von Petri-Netzen.” In: *Stoffstromanalysen: in Ökobilanzen und Öko-Audits*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1995, pp. 33–58. ISBN: 978-3-642-79793-4. DOI: 10.1007/978-3-642-79793-4_3.
- [37] C. Ortmeier, N. Henningsen, A. Langer, A. Reisch, A. Karl, and C. Herrmann. “Framework for the integration of Process Mining into Life Cycle Assessment.” In: *Procedia CIRP* 98 (2021). The 28th CIRP Conference on Life Cycle Engineering, March 10 – 12, 2021, Jaipur, India, pp. 163–168. DOI: 10.1016/j.procir.2021.01.024.
- [38] G.-E. Öztürk. *Ocelescope: An Extensible Web Framework for Object-Centric Process Mining*. Bachelor’s Thesis. Advisors: Nina Graves, Aaron Küsters. Aachen, Germany, Dec. 2025.
- [39] G. Park and L. T. genannt Unterberg. *Procure-to-payment (P2P) Object-centric Event Log in OCEL 2.0 Standard*. Oct. 2023. DOI: 10.5281/zenodo.8412920.
- [40] D. Schäfer, F. Klessascheck, T. Kampik, and L. Pufahl. “Can We Leverage Process Data from ERP Systems for Business Process Sustainability Analyses?” In: *Process Mining Workshops*. Ed. by A. Delgado and T. Slaats. Cham: Springer Nature Switzerland, 2025, pp. 764–777. ISBN: 978-3-031-82225-4. DOI: 10.1007/978-3-031-82225-4_56.
- [41] D. Schneider, P. Jordan, J. Dietz, M. F. Zaeh, and G. Reinhart. “Concept for Automated LCA of Manufacturing Processes.” In: *Procedia CIRP* 116 (2023). 30th CIRP Life Cycle Engineering Conference, pp. 59–64. DOI: 10.1016/j.procir.2023.02.011.
- [42] I. Steen. “Phosphorus availability in the 21st century: management of a non-renewable resource.” In: *Phosphorus & Potassium* 217 (1998), pp. 25–31.
- [43] J. N. Van Detten, P. Schumacher, and S. J. J. Leemans. “Discovering Compact, Live and Identifier-Sound Object-Centric Process Models.” In: *2024 6th International Conference on Process Mining (ICPM)*. 2024, pp. 113–120. DOI: 10.1109/ICPM63005.2024.10680659.

- [44] T. Viere. “State of the Art and Future Developments in LCA.” In: *Progress in Life Cycle Assessment* (2018), pp. 3–6. DOI: 10.1007/978-3-319-92237-9_1.
- [45] L. Wen, W. M. P. van der Aalst, J. Wang, and J. Sun. “Mining process models with non-free-choice constructs.” In: *Data Mining and Knowledge Discovery* 15.2 (2007), pp. 145–180. DOI: 10.1007/s10618-007-0065-y.

We used Google Gemini for proofreading, improving grammar and readability, and generating Latex. In addition, parts of our code implementing our work were generated using Gemini.